

ENERGETIKAI TECHNIKUM ÉS KOLLÉGIUM

Vizsgaremek

Retro Realm

2026.

Szakma:
SZOFTVERFEJLESZTŐ ÉS -TESZTELŐ
5 0613 12 03

Készítették:
Furduj Csenge Viktória
Hegedüs Dávid
Lőrinc Márton

Tartalomjegyzék

1.	Bevezetés	5
2.	Témaválasztás indoklása	6
2.1	Indoklás és aktualitás	6
2.2	Kutatások, követelmények	7
2.3	Eszközök és eljárások	8
3.	Fejlesztői dokumentáció	12
3.1	Specifikáció	12
3.2	Adatmodell és adatbázis leírása	12
3.3	Táblák és modelljeik	13
3.4	API végpontok	25
3.5	Egyéb Backend Algoritmusok	54
3.6	Frontend játékok	59
3.7	Frontend oldalak	92
4.	Tesztelési dokumentáció.....	122
4.1	Bevezetés	122
4.2	Tesztelési módszerek	122
4.3	Tesztesetek bemutatása.....	123
4.4	API tesztelés	124
4.5	Tesztelés során talált hibák	125
5.	Felhasználói dokumentáció	126
5.1	Játékok felhasználói dokumentációja	126
5.2	Az oldalak felhasználói dokumentációja	149
6.	A program általános specifikációja, célja.....	173
7.	Rendszerkövetelmények.....	175
7.1	Hardver követelmények.....	175
7.2	Szoftver követelmények	176

8.	A program telepítése.....	177
9.	Összegzés	178
10.	Ábrajegyzék.....	180
11.	Irodalomjegyzék, hivatkozásjegyzék	185

Alulírott **Furduj Csenge Viktória** büntetőjogi felelősségem teljes tudatában nyilatkozom arról, hogy az itt szereplő vizsgaremek csoportmunka eredménye, és sem részeiben, sem egészében nem került még kereskedelmi forgalomba, ill. publikálásra, a GPL licenszelésű programrészek kivételével.

Paks, 2026. április 22.



.....
Furduj Csenge Viktória

Alulírott **Hegedüs Dávid** büntetőjogi felelősségem teljes tudatában nyilatkozom arról, hogy az itt szereplő vizsgaremek csoportmunka eredménye, és sem részeiben, sem egészében nem került még kereskedelmi forgalomba, ill. publikálásra, a GPL licenszelésű programrészek kivételével.

Paks, 2026. április 22.



.....
Hegedüs Dávid

Alulírott **Lőrinc Márton** büntetőjogi felelősségem teljes tudatában nyilatkozom arról, hogy az itt szereplő vizsgaremek csoportmunka eredménye, és sem részeiben, sem egészében nem került még kereskedelmi forgalomba, ill. publikálásra, a GPL licenszelésű programrészek kivételével.

Paks, 2026. április 22.



.....
Lőrinc Márton

1. Bevezetés

Csapatunk egy olyan problémára keresett megoldást, amely a könnyen hozzáférhető, mégis közösségi élményt nyújtó online játékok hiányából indult ki. Ennek eredményeként egy webalapú platform fejlesztésébe kezdtünk, amely egyszerre kínál minijátékokat, teljesítményalapú kihívásokat és egy interaktív virtuális teret a felhasználók számára. A rendszer célja, hogy egységes környezetben biztosítson szórakozást, fejlődési lehetőségeket, miközben a felhasználói élményt digitális jutalmak támogatják.

A platform négy klasszikus, böngészőben futtatható játékot tartalmaz: Wordle (szó kitaláló), Memória játék, Bunny Run és Flappy Bird. Ezek mindegyike különböző kihívásokkal egészül ki, amelyek teljesítése után a felhasználók „Coin” jutalmat kapnak. A megszerzett jutalmak felhasználhatók a rendszer beépített Cosmetic Shop felületén, ahol avatárok vásárolhatók. Mindez egy átlátható, előrehaladást követő rendszerrel egészül ki, amely rögzíti a játékosok eredményeit és fejlődését.

A játékokat kiegészíti a Retro Realm nevű közösségi tér, amely a platform egyik legfontosabb elemeként működik. Itt a felhasználók szabadon mozoghatnak, kommunikálhatnak egymással, különböző standokhoz vagy épületekhez léphetnek, és akár játékokat is indíthatnak. A Retro Realm célja egy olyan digitális környezet kialakítása, amely a hagyományos minijátékok mellé közösségi élményt is társít, ezáltal növelve a rendszer interaktivitását és hosszú távú vonzerejét.

A projekt forráskódja két különálló GitHub repository-ban érhető el, amelyek eltérő célt szolgálnak a fejlesztési folyamat során. A fejlesztési repository tartalmazza a teljes forráskódot, beleértve a fejlesztés alatt álló funkciókat és a projekt minden komponensét. Ezzel szemben a kiadási repository kizárólag a végleges, stabil állapotot reprezentálja, és egyetlen mappában tartalmazza magát a kész programot, beleértve az indításhoz szükséges futtatható (.exe) fájlt. Az oldal alapértelmezett elérhetősége a `https://localhost:5001`. A repository-k az alábbi linkeken érhetők el:

Fejlesztési repository: <https://github.com/Davidk0Fiddler/Technikusi-Projekt>

Kiadási repository: <https://github.com/Davidk0Fiddler/RetroRealm>

2. Témaválasztás indoklása

2.1 Indoklás és aktualitás

A platform fejlesztésének ötlete abból a személyes tapasztalatból indult ki, hogy fiatal felhasználókként rendszeresen találkozunk az online játékok közösségformáló erejével. A modern játékelmény már nem csupán a játékmenetről szól, hanem a közösségi interakciókról, a vizuális testreszabásról és a teljesítmény folyamatos követéséről is. Úgy láttuk, hogy kevés olyan könnyen hozzáférhető, böngészőalapú megoldás létezik, amely egyszerre nyújt minőségi minijátékokat és egy közös virtuális teret, ahol a játékosok kapcsolatba léphetnek egymással. Ezért döntöttünk egy olyan platform létrehozása mellett, amely mind a szórakozási igényt, mind pedig a közösségi élményt képes kielégíteni – külön telepítés nélkül, bármilyen böngészőben elérhető.

A játékok asztali számítógépre történő optimalizálása tudatos döntés volt. Mivel magunk is rendszeresen játszunk online játékokat, pontosan tudjuk, hogy bizonyos típusú minijátékok – különösen a gyors reakcióidőt, precíz irányítást és pontos billentyűzetkezelést igénylő játékok – élménye számottevően jobb nagyobb kijelzőn, stabilabb hardveres környezetben. A Wordle, a Memória játék, a Bunny Run és a Flappy Bird esetében a kényelmes irányíthatóság, a jó átláthatóság és a folyamatos teljesítmény különösen fontos szempontok. A mobilkészülékek változó kijelző mérete, érintésalapú vezérlése és korlátozott teljesítménye sok esetben rontaná ezeknek a játékoknak a minőségét és élvezhetőségét. Ugyanakkor úgy alakítottuk ki a platformot, hogy a kiegészítő szolgáltatások – például a ranglisták, a Cosmetic Shop és a profiloldalak – mobilon is zökkenőmentesen használhatók legyenek, így a felhasználók bárholnan hozzáférhetnek a közösségi funkciókhoz, miközben a játékelmény továbbra is a legideálisabb eszközön marad.

2.2 Kutatások, követelmények

A fejlesztés kezdetén részletesen átnéztük a vizsgakövetelményeket annak érdekében, hogy pontosan meghatározzuk, milyen kötelező elemeknek kell megfelelnie a RetroRealm projektnek. A feladat egy olyan alkalmazás elkészítése volt, amely egy frontend felületen keresztül lehetővé teszi a felhasználók számára a különböző retro játékok böngészését, megjelenítését és az azokhoz tartozó adatok elérését. Ennek megvalósításához szükség volt egy backend rendszerre is, amely kezeli az adatok tárolását, az adatbázisban található játékok és kapcsolódó információk struktúráját, valamint az API-végpontokat, amelyek biztosítják a kommunikációt a frontend és a backend között.

A felhasználói felület kialakításához HTML5, CSS3 és JavaScript technológiákat használtunk. Ezek segítségével hoztuk létre a weboldal megjelenését és azokat az interaktív elemeket, amelyek lehetővé teszik a felhasználók számára a játékok listázását, adatainak megtekintését és a felületen történő navigációt. A szerveroldali működés megvalósításához az ASP.NET Core Web API keretrendszert választottuk, amely C# programozási nyelven készült, és megbízható alapot biztosít az alkalmazás adatkezelési és kommunikációs folyamataihoz.

Az alkalmazás adatainak tárolására SQLite adatbázist alkalmaztunk, amelyet az Entity Framework segítségével kapcsoltunk össze a backend rendszerrel. Ez a megoldás különösen praktikus kisebb projektek vagy fejlesztési környezetek esetén, mivel nem igényel külön adatbázis-szervert, mégis hatékonyan kezeli az alkalmazásban szereplő adatokat, például a játékok adatait és az azok közötti kapcsolatokat.

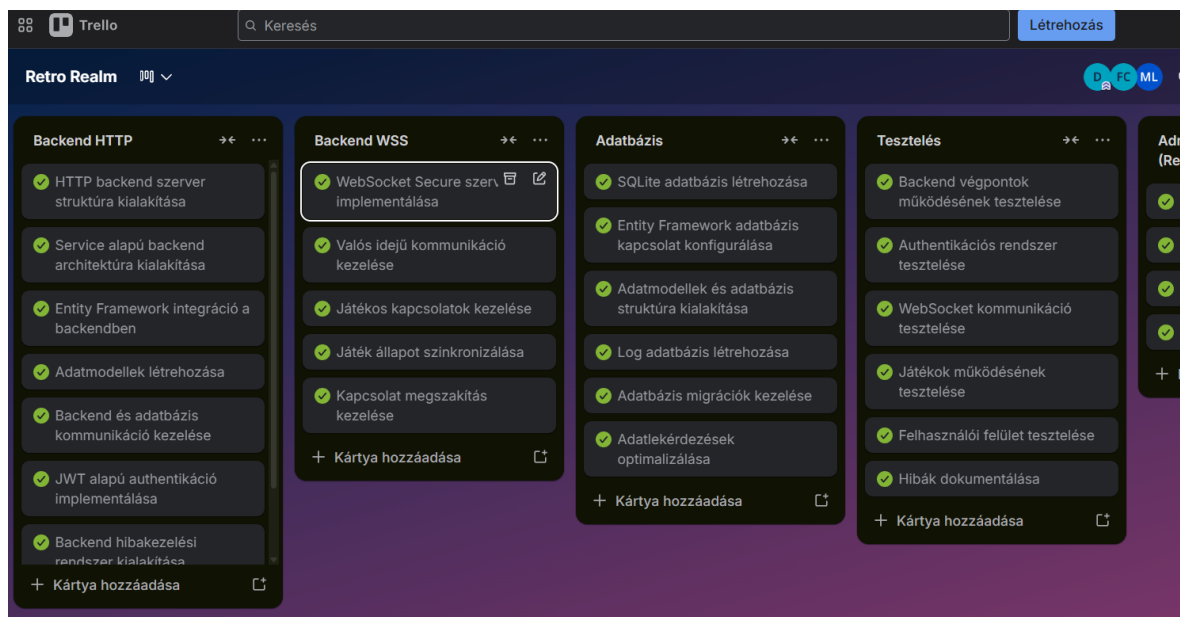
A rendszer támogatja az alapvető adatkezelési műveleteket, vagyis a CRUD műveleteket. Ez magában foglalja az adatok létrehozását (*Create*), lekérdezését (*Read*), módosítását (*Update*) és törlését (*Delete*). Ezek a műveletek a backend oldalon a megfelelő HTTP-metódusok (*GET*, *POST*, *PUT*, *DELETE*) segítségével valósulnak meg, amelyeken keresztül a frontend képes kommunikálni a szerverrel.

A fejlesztés során nagy hangsúlyt fektettünk a Clean Code elveinek betartására, hogy a projekt kódja átlátható, könnyen érthető és később is egyszerűen karbantartható legyen. Ennek érdekében a különböző programozási nyelvekhez tartozó elnevezési konvenciókat alkalmaztuk: a backendben, ahol C# nyelvet használtunk, és a frontend oldalon, a JavaScript kódban camelCase formát követtünk, míg a HTML5 és a CSS3 file-ok dash-case formát követik. Emellett egy logikus és jól strukturált mappaszerkezetet alakítottunk ki, amely megkönnyíti a Retro Realm projektben található fájlok és komponensek rendszerezését.

2.3 Eszközök és eljárások

Trello

A Trello egy online projektmenedzsment eszköz, amely segít a feladatok rendszerezésében és a projektek áttekinthető kezelésében. A rendszer táblákból, listákból és kártyákból áll, amelyek segítségével könnyen követhető egy projekt munkafolyamata. A kártyák egy-egy feladatot jelölnek, amelyekhez leírás, határidő vagy mellékletek is hozzáadhatók. A Retro Realm projekt fejlesztése során a Trello segített a feladatok tervezésében és a munka folyamatának nyomon követésében.



1. ábra - Trello munkafelület

Github

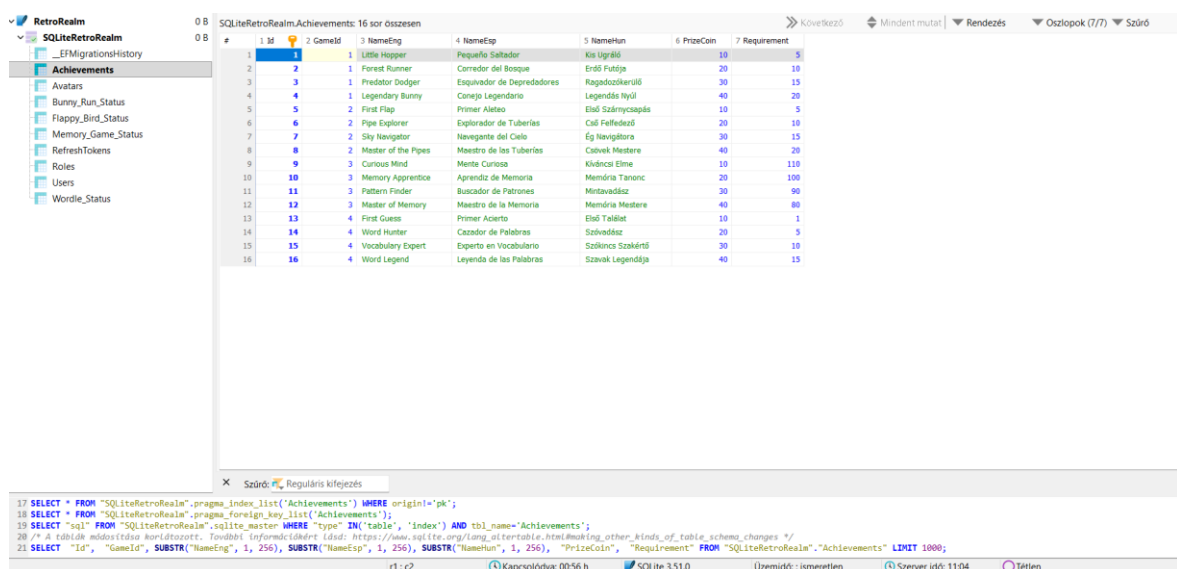
A GitHub egy online platform, amely verziókezelést és közös szoftverfejlesztést tesz lehetővé. A rendszer a Git verziókezelő rendszeren alapul, amely lehetővé teszi a kód változásainak nyomon követését és kezelését. Segítségével több fejlesztő is dolgozhat ugyanazon a projekten anélkül, hogy felülírnák egymás munkáját. A Retro Realm projekt során a GitHubot a forráskód tárolására, a verziók követésére és a közös fejlesztési folyamat támogatására használtuk.

Visual Studio, Visual Studio Code

A Microsoft Visual Studio és a Visual Studio Code fejlesztői környezeteket a Retro Realm projekt készítése során a programkód írására és szerkesztésére használtuk. A Visual Studio elsősorban a backend fejlesztéséhez volt hasznos, mivel jól támogatja a C# és az ASP.NET Core technológiákat. A Visual Studio Code könnyű és gyors szerkesztőként segítette a frontend fájlok, például a HTML, CSS és JavaScript kódok készítését. A két fejlesztői eszköz együttes használata hatékonyabbá tette a fejlesztési folyamatot.

HeidiSQL

A HeidiSQL egy adatbázis-kezelő program, amely lehetővé teszi adatbázisok egyszerű kezelését és módosítását grafikus felületen keresztül. Segítségével könnyen létrehozhatók táblák, valamint megtekinthetők és szerkeszthetők az adatbázisban tárolt adatok. A program támogatja az SQL lekérdezések futtatását, ami megkönnyíti az adatok kezelését és ellenőrzését. A Retro Realm projekt során a HeidiSQL-t az adatbázis szerkezetének és a játékokhoz tartozó adatok kezelésére használtuk.



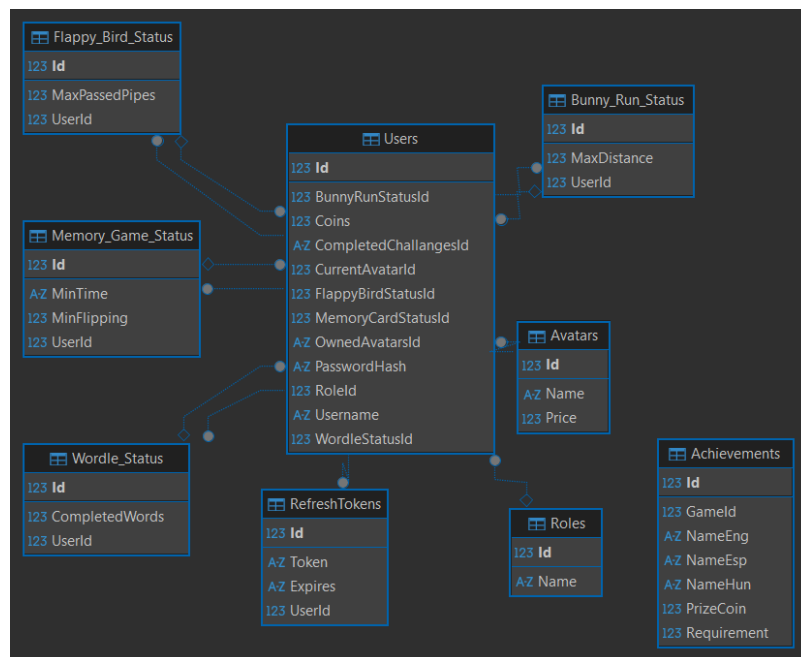
The screenshot shows the HeidiSQL interface with the RetroRealm database selected. The left pane shows the database structure, including tables like _EFMigrationsHistory, Achievements, Avatars, Bunny_Run_Status, Flappy_Bird_Status, Memory_Game_Status, RefreshTokens, Roles, Users, and Wordle_Status. The main pane displays the 'Achievements' table with 16 rows. The table has columns: #, Id, GameId, NameEng, NameEsp, NameHun, PrizeCoin, and Requirement. The data includes various achievements like 'Little Hopper', 'Forest Runner', 'Predator Dodger', etc.

#	Id	GameId	NameEng	NameEsp	NameHun	PrizeCoin	Requirement
1	1	1	Little Hopper	Pequeño Saltador	Kis Ugráló	10	5
2	2	1	Forest Runner	Corredor del Bosque	Erdő Futója	20	10
3	3	1	Predator Dodger	Esquivador de Depredadores	Ragadozkerülő	30	15
4	4	1	Legendary Bunny	Conejo Legendario	Legendás Nyúl	40	20
5	5	2	First Flap	Primer Alazo	Első Szárnycsapás	10	5
6	6	2	Pipe Explorer	Explorador de Tuberías	Cső Felfedező	20	10
7	7	2	Sky Navigator	Navegante del Cielo	Ég Navigátora	30	15
8	8	2	Master of the Pipes	Maestro de las Tuberías	Csővek Mestere	40	20
9	9	3	Curious Mind	Mente Curiosa	Kíváncsi Elme	10	110
10	10	3	Memory Apprentice	Aprendiz de Memoria	Memória Tanonc	20	100
11	11	3	Pattern Finder	Buscador de Patrones	Mintavadász	30	90
12	12	3	Master of la Memoria	Maestro de la Memoria	Memória Mestere	40	80
13	13	4	First Guess	Primer Acerto	Első Találat	10	1
14	14	4	Word Hunter	Cazador de Palabras	Szavadvész	20	5
15	15	4	Vocabulary Expert	Experto en Vocabulario	Szókincs Szakértő	30	10
16	16	4	Word Legend	Leyenda de las Palabras	Szavak Legendája	40	15

2. ábra - Adatbázis szelete a HeidiSQL-ben

DBeaver

A DBeaver egy adatbázis-kezelő és -vizualizáló eszköz, amely különböző adatbázisok kezelésére és áttekintésére használható. Lehetővé teszi a táblák, adatok és kapcsolatok egyszerű böngészését, valamint vizuális diagramok készítését az adatbázis szerkezetéről. A program egyik hasznos funkciója az ER (Entity-Relationship) diagram generálása, amely segítségével a táblák közötti kapcsolatok grafikus formában jeleníthetők meg. A RetroRealm projekt során a DBeaver szoftvert kizárólag az adatbázis ER modelljének képiítésére használtuk, így könnyebben átláthatóvá vált a rendszer struktúrája és a táblák közötti összefüggések.



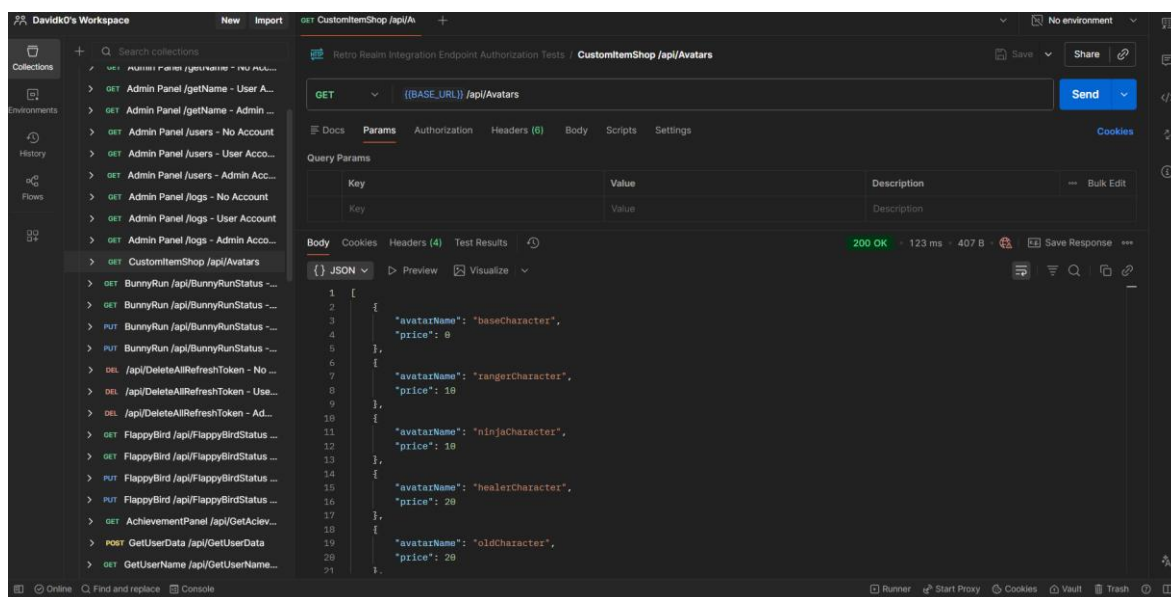
3. ábra - Adatbázis ER-modell változata DBeaver-ben

Swagger

A Swagger egy fejlesztői eszköz, amely az API-k dokumentálására és tesztelésére szolgál. Segítségével a backend által biztosított végpontok könnyen áttekinthetők és kipróbálhatók egy webes felületen keresztül. A fejlesztők így gyorsan ellenőrizhetik, hogy az egyes HTTP-kérések megfelelően működnek-e. A RetroRealm projekt során a Swagger segítségével teszteltük és ellenőriztük az API-végpontok működését.

Postman

A Postman egy API-fejlesztő és tesztelő eszköz, amely lehetővé teszi HTTP-kérések egyszerű létrehozását, küldését és vizsgálatát. Segítségével a backend szolgáltatások végpontjai könnyen tesztelhetők különböző metódusokkal (GET, POST, PUT, DELETE), valamint kezelhetők az autentikációs folyamatok is, például token alapú azonosítás. A fejlesztők így gyorsan ellenőrizhetik a válaszokat, státuszkódokat és az adatforgalmat. A RetroRealm projekt során a Postman szoftvert az integrációs és autorizációs tesztek elkészítésére használtuk, különösen a bejelentkezési és token-kezelési folyamatok ellenőrzésére.



4. ábra - Végpontok autorizációs tesztelése Postman-el

KaplayJS játékefejlesztő könyvtár

A KapaJS egy JavaScript alapú könyvtár, amelyet böngészőben futó játékok fejlesztésére lehet használni. Segítségével egyszerűen megvalósíthatók különböző játékelemek, például karakterek, mozgások és ütközések. A könyvtár támogatja az interaktív játékelemek gyors fejlesztését. A Retro Realm projekt során a KapaJS-t a weboldalon futó retro játékok megvalósításához használtuk.

3. Fejlesztői dokumentáció

3.1 Specifikáció

A RetroRealm online játéktér egy webalapú platform, amelynek célja, hogy a felhasználók számára könnyen elérhető és szórakoztató online játékelményt biztosítson. A rendszer kialakítása során fontos szempont volt az egyszerű kezelhetőség, az átlátható felhasználói felület, valamint a stabil és megbízható működés. A fejlesztés során figyelmet fordítottunk arra, hogy a felhasználói hibákból vagy figyelmetlenségből adódó problémák ne okozzanak működési zavarokat a rendszerben. Ennek érdekében a program különböző ellenőrzési és hibakezelési megoldásokat alkalmaz, amelyek segítenek megelőzni a hibás működést és biztosítják a rendszer folyamatos használhatóságát. A rendszer kialakításakor szempont volt az is, hogy a platform a jövőben könnyen bővíthető legyen új játékokkal és további funkciókkal. A fejlesztés során a program működését teszteléssel ellenőriztük annak érdekében, hogy stabil működést biztosítson. A RetroRealm online játéktér célja egy olyan digitális környezet létrehozása, ahol a felhasználók különböző játékokat próbálhatnak ki egy egyszerűen kezelhető és megbízható platformon.

3.2 Adatmodell és adatbázis leírása

A projekt során az alkalmazás működéséhez szükséges adatokat relációs adatbázisban tároljuk. Az adatbázis a *SQLite* rendszerre épül, amely egy könnyen kezelhető, fájl alapú adatbázis-megoldás, így ideális kisebb és közepes méretű alkalmazások fejlesztéséhez.

Az adatbázis kialakítása úgynevezett *Code First* megközelítéssel történt, amely az *Entity Framework Core* technológia egyik fontos jellemzője. A Code First lényege, hogy az adatbázis struktúráját nem közvetlenül SQL parancsokkal hozzuk létre, hanem az alkalmazásban definiált osztályok (modellek) alapján generáljuk. Ez azt jelenti, hogy először a C# osztályok kerülnek megtervezésre, amelyek az egyes entitásokat (táblákat) reprezentálják, majd ezekből az Entity Framework automatikusan létrehozza az adatbázist és annak szerkezetét migrációk segítségével.

Ennek a megközelítésnek az előnye, hogy:

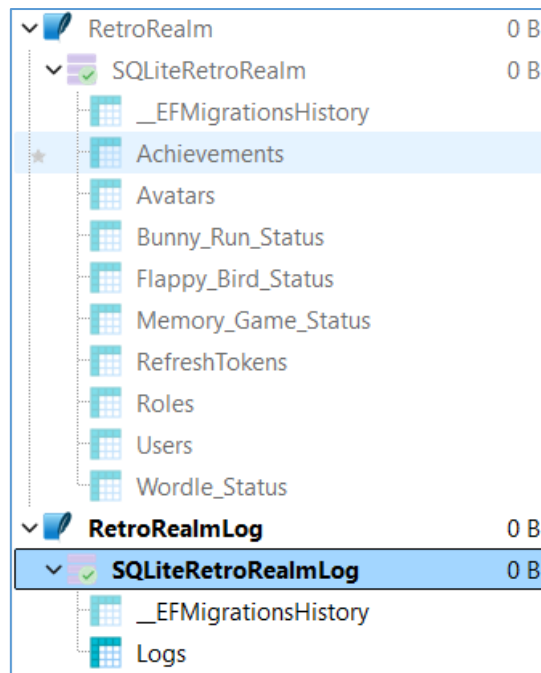
- a fejlesztés során az adatmodell és az alkalmazás logikája szorosan együtt kezelhető,
- könnyebb a módosítás és a karbantartás,
- csökken az adatbázis és a kód közötti eltérésekből eredő hibalehetőségek száma.

3.3 Táblák és modelljeik

A backend felépítésében bevett gyakorlat, hogy a modellek neve angol nyelven, egyes számban szerepel (például User), mivel ezek egyetlen entitást írnak le. Ezzel szemben az adatbázisban a DbSet típusú tulajdonságok elnevezése többes számban történik (Users), mivel ezek az adott entitás rekordjainak gyűjteményét reprezentálják. Ez alól kivételt képeznek a státuszokat tartalmazó táblák, ahol az elnevezés eltérhet ettől a konvenciótól.

Ez a fajta elnevezési konvenció széles körben elterjedt, mert következetességet visz a kódba, és egyértelművé teszi, hogy mikor beszélünk egyetlen objektumról, illetve mikor egy kollekciónról. Emellett megkönnyíti a fejlesztők számára a kód olvasását és karbantartását, hiszen a név alapján azonnal látható az adott elem szerepe. Az ilyen egységes szabályok használata hosszabb távon átláthatóbbá és rendezettebbé teszi a teljes projekt struktúráját.

A rendszer adatkezelése két külön adatbázisra van bontva: az egyik a fő üzleti adatok tárolására szolgál, míg a másik kizárólag naplózási (log) célokat lát el. Ennek az elkülönítésnek az a fő oka, hogy a naplózás folyamatosan és megszakítás nélkül működhessen, még abban az esetben is, ha a fő adatbázis valamilyen hiba miatt ideiglenesen nem elérhető vagy leáll. Így a rendszer működésével kapcsolatos események és hibák minden körülmények között rögzítésre kerülnek, ami nagyban segíti a hibakeresést, a monitorozást és az utólagos elemzést.



5. ábra - Adatbázisok és táblák HeidiSQL-ben

Achievement / Achievements

Ebben a táblában a játékokhoz tartozó kihívások kerülnek tárolásra. A játékosok a teljesített kihívások azonosítóit listában tárolják, és az ezekhez tartozó részletes adatokat az azonosítók alapján ebből a táblából kérdezik le. Minden kihívás rendelkezik egy egyedi azonosítóval, egy játékaazonosítóval, a megszerezhető Coin jutalom értékével, valamint egy szükséges teljesítési feltétellel, amely játékonként eltérő lehet. Emellett minden kihíváshoz három különböző nyelven megadott név tartozik.

```
public class Achievement
{
    [Key]
    public int Id { get; set; }
    public string NameEng { get; set; }
    public string NameEsp { get; set; }
    public string NameHun { get; set; }
    public int GameId { get; set; }
    public int Requirement { get; set; }
    public int PrizeCoin { get; set; }
}
```

6. ábra - Achievement modell

Achievements			
	Típus	Mezőnév	Megjegyzés
	INTEGER	Id	Auto increment
	TEXT	NameEng	A kihívás angol neve
	TEXT	NameEsp	A kihívás spanyol neve
	TEXT	NameHun	A kihívás magyar neve
	INTEGER	GameId	A kihíváshoz tartozó játék azonosító
	INTEGER	Requirement	A kihívás szükséglete
	INTEGER	PrizeCoin	A kihívás jutalma

Avatar / Avatars

Ebben a táblában a felhasználókhöz tartozó avatártípusok adatai kerülnek tárolásra. Ezek az avatárok a játékosok számára lehetőséget biztosítanak arra, hogy virtuális karaktereiket egyedivé tegyék és személyre szabják. Az egyes avatárok különböző megjelenéseket képviselnek, amelyeket a felhasználók a játék során megvásárolhatnak.

Minden avatar rendelkezik egy egyedi azonosítóval, egy névvel, valamint egy meghatározott árral. Az alapértelmezett avatar a rendszerben a *baseCharacter* nevet viseli, és 1-es azonosítóval van ellátva, amely minden felhasználó számára alaphál elérhető. A tábla feladata az avatárok adatainak egységes kezelése és tárolása, valamint annak biztosítása, hogy a játék során bármikor egyszerűen lekérdezhetők és felhasználhatók legyenek.

```
public class Avatar
{
    [Key]
    public int Id { get; set; }
    public string Name { get; set; }
    public int Price { get; set; }
}
```

7. ábra - Avatar modell

Avatars			
	Típus	Mezőnév	Megjegyzés
	INTEGER	Id	Auto Increment
	TEXT	Name	Az avatar megnevezése
	INTEGER	Price	Az avatarhoz tartozó ár

Bunny Run Status

Ebben a táblában a Bunny Run játékhoz tartozó felhasználói állapotok adatai kerülnek tárolásra. A modell egy gyermekosztály, amely a Status nevű ősosztályból származik, így az alapvető működés és felépítés onnan öröklődik. A tábla célja, hogy a játékosok egyéni teljesítményét rögzítse, és lehetővé tegye ezek későbbi visszakeresését.

Minden rekord rendelkezik egy egyedi azonosítóval, amely elsődleges kulcsként szolgál, valamint egy UserId mezővel, amely kapcsolatot teremt a felhasználóhoz tartozó objektummal. Emellett tartalmaz egy MaxDistance értéket, amely a játék során elért legnagyobb távolságot, vagyis a játékos rekordját tárolja. A tábla feladata ezen adatok egységes kezelése és tárolása, biztosítva, hogy a felhasználók teljesítménye bármikor pontosan lekérdezhető legyen.

```
public abstract class Status
{
    [Key]
    public int Id { get; set; }
    public int UserId { get; set; }
    [ForeignKey("UserId"), JsonIgnore]
    public User User { get; set; }
}
```

8. ábra - Status ősosztály

```
public class BunnyRunStatus : Status
{
    public int MaxDistance { get; set; } = 0;
}
```

9. ábra - Bunny Run Status modell

Bunny Run Status			
	Típus	Mezőnév	Megjegyzés
	INTEGER	Id	Auto Increment
	INTEGER	MaxDistance	A jelenlegi rekord
	INTEGER	UserId	A státuszhoz tartozó felhasználó azonosítója

Flappy Bird Status

Ebben a táblában a Flappy Bird játékhoz tartozó felhasználói állapotok adatai kerülnek tárolásra. A modell egy gyermekosztály, amely a [Status](#) nevű ősosztályból származik, így az alapvető működés és felépítés onnan öröklődik. A tábla célja, hogy a játékosok egyéni teljesítményét rögzítse, és lehetővé tegye ezek későbbi visszakeresését.

Minden rekord rendelkezik egy egyedi azonosítóval, amely elsődleges kulcsként szolgál, valamint egy UserId mezővel, amely kapcsolatot teremt a felhasználóhoz tartozó objektummal. Emellett tartalmaz egy MaxPassedPipes értéket, amely a játék során elért legtöbb elhagyott csövet, vagyis a játékos rekordját tárolja. A tábla feladata ezen adatok egységes kezelése és tárolása, biztosítva, hogy a felhasználók teljesítménye bármikor pontosan lekérdezhető legyen.

```
public class FlappyBirdStatus : Status
{
    public int MaxPassedPipes { get; set; } = 0;
}
```

8. ábra - Flappy Bird Status Modell

Flappy Bird Status			
	Típus	Mezőnév	Megjegyzés
	INTEGER	Id	Auto Increment
	INTEGER	MaxPassedPipes	A jelenlegi rekord
	INTEGER	UserId	A státuszhoz tartozó felhasználó azonosítója

Memory Game Status

Ebben a táblában a Memory Game játékhoz tartozó felhasználói állapotok adatai kerülnek tárolásra. A modell egy gyermekosztály, amely a [Status](#) nevű ősosztályból származik, így az alapvető működés és felépítés onnan öröklődik. A tábla célja, hogy a játékosok egyéni teljesítményét rögzítse, és lehetővé tegye ezek későbbi visszakeresését.

Minden rekord rendelkezik egy egyedi azonosítóval, amely elsődleges kulcsként szolgál, valamint egy UserId mezővel, amely kapcsolatot teremt a felhasználóhoz tartozó objektummal. Emellett tartalmaz egy MinTime értéket, amely a játék során legrövidebb megoldási időt tárolja, és tartalmaz egy MinFlipping értéket, ami a legkevesebb felfordítás rekordot tartalmazza. A tábla feladata ezen adatok egységes kezelése és tárolása, biztosítva, hogy a felhasználók teljesítménye bármikor pontosan lekérdezhető legyen.

```
public class MemoryGameStatus : Status
{
    public int[] MinTime { get; set; } = [999, 0, 0];
    public int MinFlipping { get; set; } = 0;
}
```

9. ábra - Memory Game Status Modell

Memory Game Status			
	Típus	Mezőnév	Megjegyzés
🔑	INTEGER	Id	Auto Increment
	TEXT	MinTime	A jelenlegi idő rekord
	INTEGER	MinFlipping	A jelenlegi felfordítás rekord
	INTEGER	UserId	A státuszhoz tartozó felhasználó azonosítója

Wordle Status

Ebben a táblában a Wordle játékhoz tartozó felhasználói állapotok adatai kerülnek tárolásra. A modell egy gyermekosztály, amely a [Status](#) nevű ősosztályból származik, így az alapvető működés és felépítés onnan öröklődik. A tábla célja, hogy a játékosok egyéni teljesítményét rögzítse, és lehetővé tegye ezek későbbi visszakeresését.

Minden rekord rendelkezik egy egyedi azonosítóval, amely elsődleges kulcsként szolgál, valamint egy UserId mezővel, amely kapcsolatot teremt a felhasználóhoz tartozó objektummal. Emellett tartalmaz egy CompletedWords értéket, amely a játékok során kitalált szavakat. A tábla feladata ezen adatok egységes kezelése és tárolása, biztosítva, hogy a felhasználók teljesítménye bármikor pontosan lekérdezhető legyen.

```
public class WordleStatus : Status
{
    public int CompletedWords { get; set; } = 0;
}
```

10. ábra - Wordle Status Modell

Wordle Status			
	Típus	Mezőnév	Megjegyzés
	INTEGER	Id	Auto Increment
	INTEGER	CompletedWords	A jelenlegi kitalált szavak száma
	INTEGER	UserId	A státuszhoz tartozó felhasználó azonosítója

Role / Roles

Ebben a táblában a felhasználókhöz tartozó jogosultsági körök adatai kerülnek tárolásra. A rendszer jelenlegi állapotában kétféle szerepkör van definiálva: User és Admin. A tábla célja, hogy meghatározza és kezelje a felhasználók különböző jogosultsági szintjeit, valamint biztosítsa ezek egyértelmű elkülönítését.

Minden rekord rendelkezik egy egyedi azonosítóval, amely elsődleges kulcsként szolgál, valamint tartalmazza a jogosultsági kör megnevezését. A tábla kapcsolatban áll a felhasználókat tároló táblával egy-a-többhöz kapcsolat formájában, amely lehetővé teszi, hogy egy adott szerepkörhöz több felhasználó is tartozzon. A tábla feladata a jogosultságok egységes kezelése és annak biztosítása, hogy a rendszerben minden felhasználó a számára megfelelő hozzáférési szinttel rendelkezzen.

```
public class Role
{
    [Key]
    public int Id { get; set; }
    public string Name { get; set; }
    [JsonIgnore]
    public List<User>? Users { get; set; }
}
```

11. ábra - Role Modell

Roles Status			
	Típus	Mezőnév	Megjegyzés
	INTEGER	Id	Auto Increment
	TEXT	Name	A jogosultságkör megnevezése

User / Users

Ebben a táblában a rendszer felhasználóinak adatai kerülnek tárolásra. A modell célja, hogy egységesen kezelje a felhasználók alapadatait, valamint a játékhoz kapcsolódó állapotokat és erőforrásokat. A tábla tartalmazza a felhasználónév és a jelszó hash formában történő tárolását, továbbá a játék során gyűjtött Coin egyenleget, amely alapértelmezetten 0 értékről indul.

Minden rekord rendelkezik egy egyedi azonosítóval, amely elsődleges kulcsként szolgál. A felhasználóhoz tartozik egy RoleId mező, amely a jogosultsági szintet határozza meg, valamint egy CurrentAvatarId, amely az aktuálisan használt avatárt jelöli. Emellett a modell listák formájában tárolja a felhasználó által birtokolt avatárok, illetve a teljesített kihívások azonosítóit. A felhasználóhoz különböző játékokhoz tartozó állapotok is kapcsolódhatnak (például Memory Game, Flappy Bird, Wordle vagy Bunny Run), amelyek külön táblákban kerülnek tárolásra, és ezekhez azonosítókön keresztül kapcsolódik.

A tábla feladata a felhasználók adatainak központi kezelése, valamint annak biztosítása, hogy a rendszerben minden, a játékoshoz tartozó információ strukturáltan és hatékonyan elérhető legyen.

```
public class User
{
    [Key]
    public int Id { get; set; }
    public string Username { get; set; }
    public string PasswordHash { get; set; }
    public int Coins { get; set; } = 0;
    public int? RoleId { get; set; }
    [ForeignKey("RoleId")]
    [JsonIgnore]
    public Role? Role { get; set; }
    public int CurrentAvatarId { get; set; } = 1;
    [ForeignKey("CurrentAvatarId")]
    [JsonIgnore]
    public Avatar CurrentAvatar { get; set; }

    public List<int> OwnedAvatarsId { get; set; } = new List<int>{ 1 };
    public List<int> CompletedChallengesId { get; set; } = new List<int>();
    [JsonIgnore]
    public int? MemoryCardStatusId { get; set; }
    [ForeignKey("MemoryCardStatusId")]
    [JsonIgnore]
    public MemoryGameStatus? MemoryGameStatus { get; set; }

    [JsonIgnore]
    public int? FlappyBirdStatusId { get; set; }
    [ForeignKey("FlappyBirdStatusId")]
    [JsonIgnore]
    public FlappyBirdStatus? FlappyBirdStatus { get; set; }

    [JsonIgnore]
    public int? WordleStatusId { get; set; }
    [ForeignKey("WordleStatusId")]
    [JsonIgnore]
    public WordleStatus? WordleStatus { get; set; }

    [JsonIgnore]
    public int? BunnyRunStatusId { get; set; }
    [ForeignKey("BunnyRunStatusId")]
    [JsonIgnore]
    public BunnyRunStatus? BunnyRunStatus { get; set; }
}
```

12. ábra - User Modell

Users			
	Típus	Mezőnév	Megjegyzés
	INTEGER	Id	Auto Increment
	TEXT	Username	A felhasználó neve
	TEXT	PasswordHash	A felhasználó jelszavának hashelt változata
	INTEGER	RoleId	A felhasználó jogosultság körének megnevezése
	INTEGER	Coins	A játékos online fizetőeszköze
	TEXT	CompletedChallengesId	A játékos által elért kihívások azonosítói
	INTEGER	CurrentAvatarId	A játékos jelenleg használt avatárja
	TEXT	OwnedAvatarsId	A játékos által birtokol avatárok azonosítói
	INTEGER	BunnyRunStatusId	A játékos Bunny Run státuszának azonosítója
	INTEGER	FlappyBirdStatusId	A játékos Flappy Bird státuszának azonosítója
	INTEGER	MemoryGameStatusId	A játékos Memory Game státuszának azonosítója
	INTEGER	WordleStatusId	A játékos Wordle státuszának azonosítója

RefreshToken / RefreshTokens

Ebben a táblában a felhasználókhhoz tartozó frissítő tokenek (refresh tokenek) adatai kerülnek tárolásra. Ezek a tokenek a hitelesítési folyamat részét képezik, és lehetővé teszik, hogy a felhasználó új hozzáférési tokent kapjon anélkül, hogy ismételten be kellene jelentkeznie. A tábla célja a tokenek biztonságos tárolása és a hozzájuk tartozó lejárat információk kezelése.

Minden rekord rendelkezik egy egyedi azonosítóval, amely elsődleges kulcsként szolgál, valamint tartalmazza a token értékét és annak lejárat idejét. Emellett a rekordhoz tartozik egy UserId mező is, amely az adott tokenhez tartozó felhasználó azonosítóját tárolja, azonban ez nem kerül explicit módon összekapcsolásra a felhasználó táblával.

A tábla feladata a frissítő tokenek nyilvántartása és kezelése, valamint annak biztosítása, hogy a rendszer a hitelesítési folyamat során csak érvényes és még le nem járt tokeneket fogadjon el.

```
public class RefreshToken
{
    [JsonIgnore, Key]
    public int Id { get; set; }
    public string Token { get; set; }
    public DateTime Expires { get; set; }
    [JsonIgnore]
    public int UserId { get; set; }
    [ForeignKey("UserId"), JsonIgnore]
    public User User { get; set; }
}
```

13. ábra - RefreshToken modell

RefreshTokens			
	Típus	Mezőnév	Megjegyzés
	INTEGER	Id	Auto Increment
	TEXT	Token	A frissítő token
	TEXT	Expires	A token lejárat dátuma
	INTEGER	UserId	A tokenhez tartozó felhasználó azonosítója

Log / Logs

Ebben a táblában a rendszer működése során keletkező naplóbejegyzések kerülnek tárolásra. A naplózás egy külön adatbázisban történik, így akkor is folyamatosan működik, ha a fő adatbázis valamilyen hiba miatt nem elérhető.

Minden rekord rendelkezik egy egyedi azonosítóval, valamint tartalmazza a napló típusát, egy opcionális hibaüzenetet, egy leírást és az esemény időpontját. Emellett egy opcionális UserId mező is tárolásra kerül, amely az adott eseményhez tartozó felhasználót azonosíthatja.

A tábla feladata a rendszer eseményeinek és hibáinak rögzítése, ezzel támogatva a működés nyomon követését és a hibák későbbi elemzését.

```
public class Log
{
    public int Id { get; set; }
    public string LogType { get; set; }
    public string? Error { get; set; }
    public string Description { get; set; }
    public DateTime DateTime { get; set; }
    public int? UserId { get; set; }
}
```

14. ábra - Log Modell

Logs			
	Típus	Mezőnév	Megjegyzés
	INTEGER	Id	Auto Increment
	TEXT	DateTime	A naplózás kelte
	TEXT	Description	A napló leírása
	TEXT	Error	A naplózott esetleges hiba
	TEXT	LogType	A napló típusa
	INTEGER	UserId	A naplóhoz tartozó felhasználó azonosító

3.4 API végpontok

Ebben a fejezetben a rendszer API végpontjai és azok működése kerül bemutatásra. A végpontok REST alapelvek szerint lettek kialakítva, vagyis a kliens és a szerver közötti kommunikáció HTTP metódusokon keresztül történik, és az egyes végpontok különböző erőforrások kezelésére szolgálnak. A működés a jól ismert CRUD műveletekre épül, amely magában foglalja az adatok létrehozását, lekérdezését, módosítását és törlését, ezek rendre a POST, GET, PUT és DELETE metódusokhoz kapcsolódnak.

A rendszerben hitelesítés és jogosultságkezelés is meg van valósítva, amely JWT alapú. Minden kéréshez tartozik egy token, amely alapján a szerver azonosítja a felhasználót, és ellenőrzi a jogosultságait. A végpontok különböző hozzáférési szintekkel rendelkeznek:

-  – nem igényel bejelentkezést
-  – bejelentkezett alap felhasználók számára érhető el
- ^A – csak admin jogosultsággal használható

Fontos megjegyezni, hogy nem minden végpont igényel hitelesítést. Bizonyos funkciók szándékosan elérhetők bejelentkezés nélkül is, például annak érdekében, hogy a játékosok azonnal kipróbálhassák a játékokat, vagy elérjenek alapvető oldalakat. Ez javítja a felhasználói élményt, és csökkenti a belépési akadályokat.

A backend felépítése során külön lett választva a kérések kezelése és az üzleti logika. A kontrollerek a beérkező kérések fogadásáért és a válaszok visszaadásáért felelnek, míg a tényleges feldolgozás a Service osztályokban történik. Ez a struktúra átláthatóbbá teszi a működést, és megkönnyíti a rendszer karbantartását és bővítését.

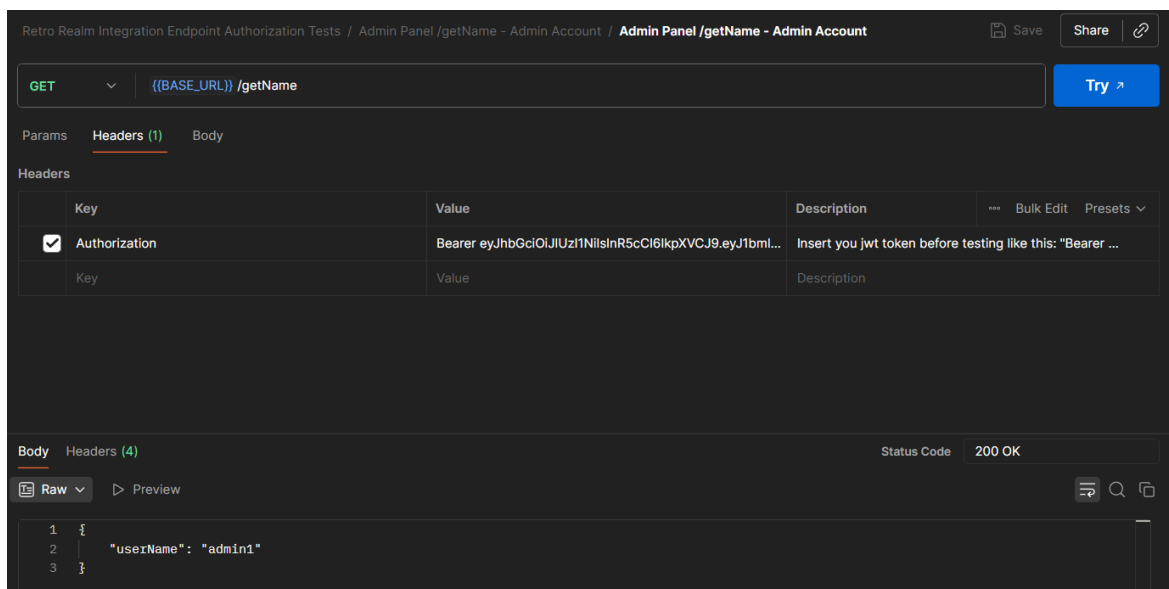
Admin felhasználónév lekérdezése végpont – /getName (🔒 A)

Ez a végpont az admin felületen kerül felhasználásra, és az aktuálisan bejelentkezett felhasználó nevének lekérdezésére szolgál. Segítségével az oldal ellenőrizni tudja, hogy a felhasználó rendelkezik-e admin jogosultsággal, valamint megjeleníti a nevét a felületen.

A kérés HTTP GET módszerrel történik a /getName útvonalon. A végpont használatához admin jogosultság szükséges, így csak megfelelő szerepkörrel rendelkező felhasználók érhetik el.

A működés során a rendszer a beérkező kéréshez tartozó JWT tokenből kiolvassa a felhasználó nevét. Ez az adat a tokenben található, így nincs szükség külön adatbázis lekérdezésre. Amennyiben a név nem található meg, a rendszer hibát ad vissza, ellenkező esetben visszaküldi azt.

A válasz egy egyszerű JSON objektum, amely a felhasználó nevét tartalmazza. Ez az adat az admin felületen kerül megjelenítésre.



15. ábra - /getName végpont válasza

Naplóbejegyzések lekérdezése végpont – /logs (🔒^A)

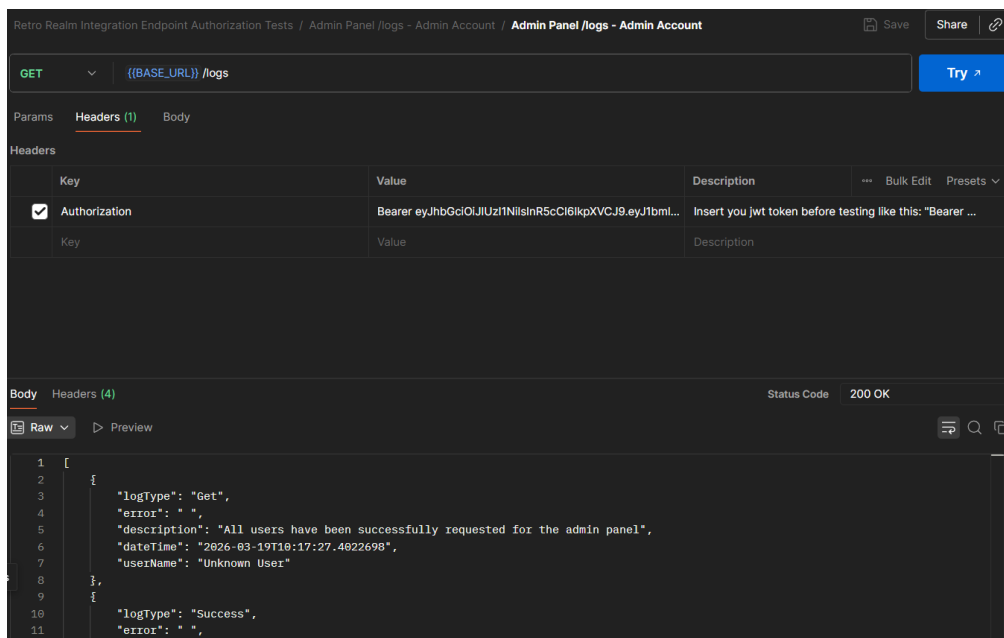
Ez a végpont az admin felületen a rendszer naplóbejegyzéseinek lekérdezésére szolgál. Segítségével az adminisztrátorok nyomon tudják követni a rendszer működését, valamint az esetleges hibákat és fontos eseményeket.

A kérés http GET módszerrel történik a /logs útvonalon. A végpont kizárólag admin jogosultsággal érhető el, így csak megfelelő hozzáféréssel rendelkező felhasználók használhatják.

A működés során a rendszer a naplóbejegyzéseket egy külön log adatbázisból kéri le. Ez biztosítja, hogy a naplózás akkor is működjön, ha a fő adatbázis valamilyen okból nem elérhető. A lekérdezett adatok tartalmazzák az esemény típusát, leírását, esetleges hibaüzenetét, valamint az esemény időpontját.

A válasz feldolgozása során a rendszer megpróbálja a naplóbejegyzésekhez tartozó felhasználóneveket is hozzárendelni a fő adatbázis alapján. Ha egy adott felhasználó nem található, akkor a rendszer „Unknown User” értéket ad vissza. A lista a végén megfordításra kerül, így a legfrissebb bejegyzések jelennek meg először.

A végpont meghívását követően a rendszer egy új naplóbejegyzést is rögzít, amely jelzi, hogy a logok lekérdezése megtörtént.



16. ábra - /logs végpont válasza

Felhasználók lekérdezése végpont – /users (🔒^A)

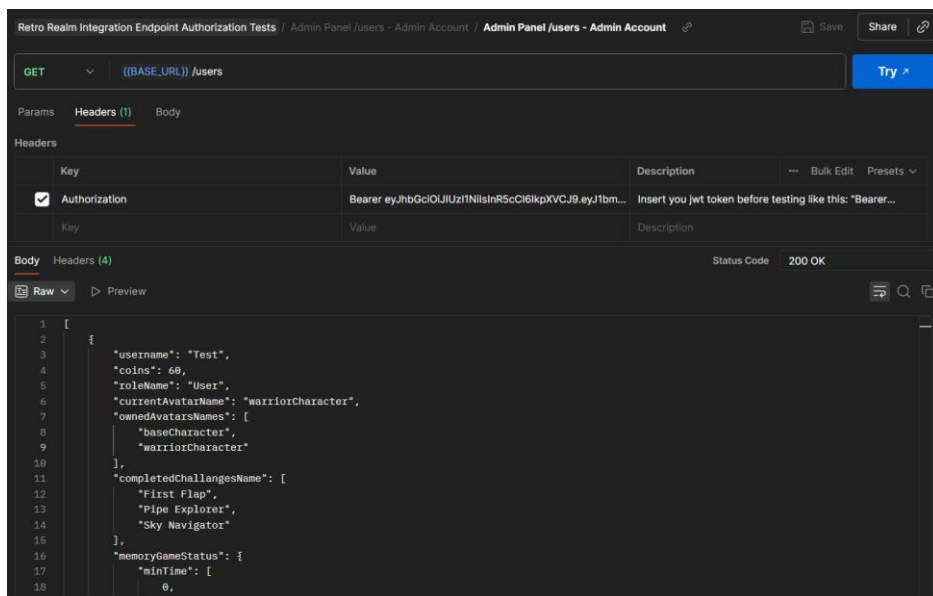
Ez a végpont az admin felületen a felhasználók adatainak lekérdezésére szolgál. Segítségével az adminisztrátor áttekintheti a rendszerben regisztrált felhasználókat, valamint azokhoz tartozó adatokat és játékstatisztikákat.

A kérés HTTP GET módszerrel történik a /users útvonalon. A végpont kizárólag admin jogosultsággal érhető el.

A működés során a rendszer lekéri az összes felhasználót a fő adatbázisból, majd betölti a hozzájuk tartozó adatokat is. Ide tartozik az aktuális avatar, a szerepkör, valamint a különböző játékokhoz tartozó állapotok. Emellett a felhasználó által birtokolt avatarok és a teljesített kihívások is feldolgozásra kerülnek, amelyek azonosítók alapján kerülnek összepárosításra a megfelelő nevekkal.

A kapott adatokat a rendszer egy egyszerűbb, megjelenítésre alkalmas formába alakítja át, amely már csak a szükséges információkat tartalmazza. A válaszban szerepel a felhasználónév, a Coin egyenleg, a szerepkör neve, az aktuális avatar, valamint a megszerzett elemek és játékstatisztikák.

A művelet végén a rendszer egy naplóbejegyzést is létrehoz, amely rögzíti a lekérdezés tényét.



17. ábra - /users végpont válasza

Avatárok lekérdezése végpont – /api/Avatars (♂)

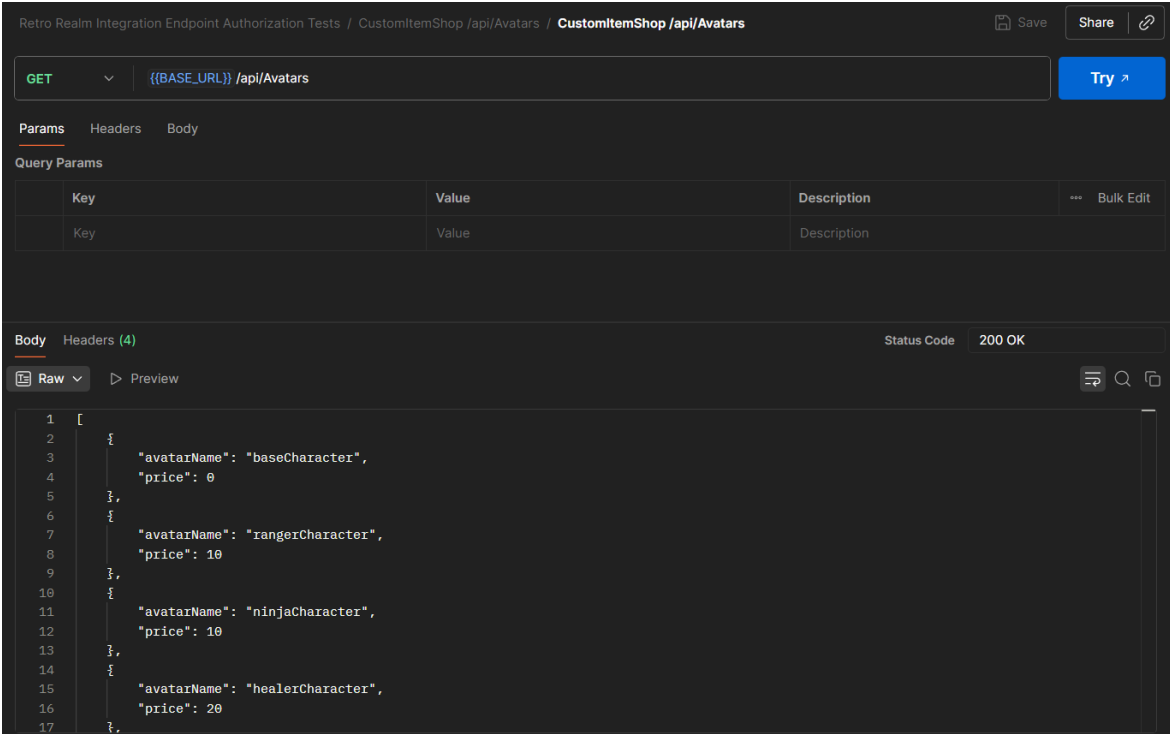
Ez a végpont az elérhető avatárok adatainak lekérdezésére szolgál. Az itt visszaadott adatok a CustomItemShop felületen kerülnek felhasználásra, ahol ezek az elemek a bolt kínálataként jelennek meg. A játékosok itt tudják megtekinteni az avatárokat, és a bejelentkezett felhasználók meg is vásárolhatják azokat, azonban az avatár kiválasztása nem ezen a végponton keresztül történik.

A kérés HTTP GET metódussal történik a /api/Avatars útvonalon. A végpont nem igényel hitelesítést, így bejelentkezés nélkül is elérhető. Ez lehetővé teszi, hogy a felhasználók már a játék megkezdése előtt is megtekinthessék a bolt kínálatát.

A működés során a controller meghívja a hozzá tartozó service réteget, amely lekéri az adatbázisból az összes avatárt. Az adatok ezt követően egy egyszerűbb formátumba kerülnek átalakításra, amely csak a megjelenítéshez szükséges információkat tartalmazza.

A válasz egy lista, amelyben minden avatárhoz tartozik egy név és egy ár. Ezek az adatok a kliens oldalon a bolt felületének feltöltésére szolgálnak.

A végpont meghívása után a rendszer egy naplóbemjegyzést is létrehoz, amely rögzíti a lekérdezés tényét.



The screenshot shows a REST client interface with the following details:

- Method: GET
- URL: `{{BASE_URL}} /api/Avatars`
- Params: Headers, Body (selected)
- Status Code: 200 OK
- Body (Raw):

```
1 [
2   {
3     "avatarName": "baseCharacter",
4     "price": 0
5   },
6   {
7     "avatarName": "rangerCharacter",
8     "price": 10
9   },
10  {
11    "avatarName": "ninjaCharacter",
12    "price": 10
13  },
14  {
15    "avatarName": "healerCharacter",
16    "price": 20
17  },
18 ]
```

18. ábra - /api/Avatars végpont válasza.

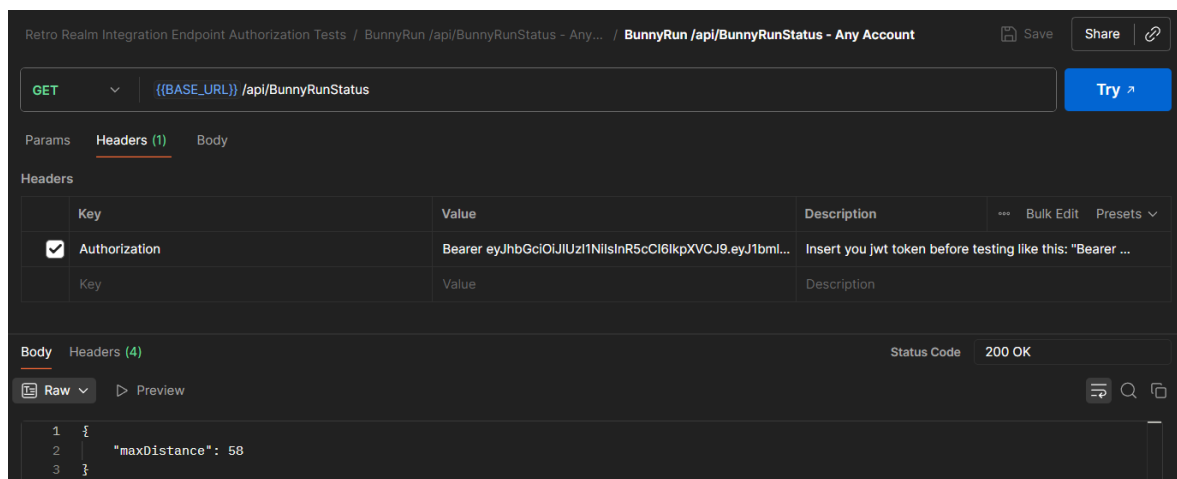
Bunny Run státusz lekérdezése végpont – /api/BunnyRunStatus (🔒)

Ez a végpont a bejelentkezett felhasználó Bunny Run játékhoz tartozó statisztikájának lekérdezésére szolgál. A kliens ezen keresztül tudja megkapni a játékos eddigi legjobb eredményét.

A kérés HTTP GET módszerrel történik a /api/BunnyRunStatus útvonalon. A végpont hitelesítést igényel, így csak bejelentkezett felhasználók érhetik el.

A működés során a rendszer a JWT tokenből kiolvassa a felhasználó nevét, majd ennek alapján lekéri az adott játékoshoz tartozó adatokat az adatbázisból. Ha a felhasználóhoz nem tartozik ilyen rekord, a rendszer hibát ad vissza, ellenkező esetben visszaküldi az adatokat.

A válasz egy objektum, amely a játékos legnagyobb elért távolságát tartalmazza. Ez az érték a kliens oldalon például statisztika vagy eredmény megjelenítésére használható.



19. ábra - /api/BunnyRunStatus lekérés végpont válasza

Bunny Run státusz frissítése végpont – /api/BunnyRunStatus (🔒)

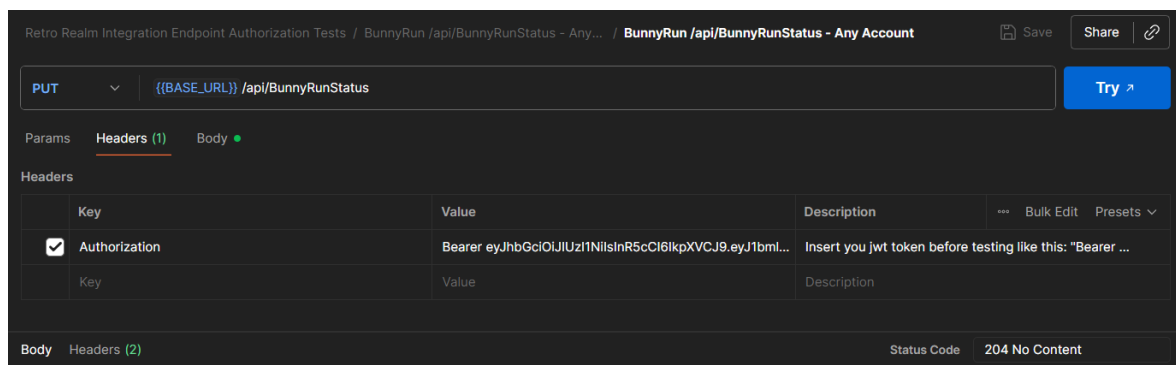
Ez a végpont a játék során elért eredmények frissítésére szolgál. A kliens csak abban az esetben küld kérést, ha az aktuális eredmény meghaladja a korábban elért rekordot.

A kérés HTTP PUT módszerrel történik a /api/BunnyRunStatus útvonalon. A végpont használatához bejelentkezés szükséges.

A működés során a rendszer a tokenből azonosítja a felhasználót, majd megkeresi a hozzá tartozó jelenlegi statisztikát. Bár a kliens már előzetesen ellenőrzi, hogy új rekord született-e, a backend ezt minden esetben újra validálja. Az új érték csak akkor kerül mentésre, ha valóban nagyobb, mint a korábban eltárolt eredmény, így biztosítva az adatok helyességét.

Sikeres frissítés esetén a rendszer nem ad vissza tartalmat, csak egy sikeres státuszkódot. Ha a felhasználóhoz nem tartozik státusz, akkor hibát jelez.

A művelet során a rendszer naplózza a frissítést, valamint ellenőrzi, hogy az új eredményhez tartozik-e teljesített kihívás.



20. ábra - /api/BunnyRunStatus frissítés végpont válasza

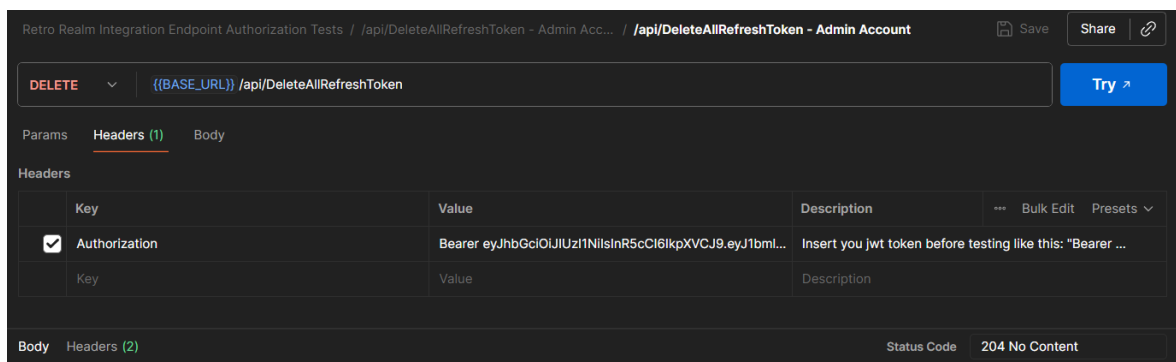
Összes refresh token törlése végpont – /api/DeleteAllRefreshToken (🔒^A)

Ez a végpont az összes eltárolt refresh token törlésére szolgál. Elsősorban adminisztrációs célokra használható, például biztonsági okokból, amikor szükség van minden aktív bejelentkezés megszüntetésére.

A kérés HTTP DELETE metódussal történik a /api/DeleteAllRefreshToken útvonalon. A végpont kizárólag admin jogosultsággal érhető el, így csak megfelelő szerepkörrel rendelkező felhasználók hajthatják végre.

A működés során a rendszer törli az összes refresh token az adatbázisból. Ennek hatására minden felhasználó elveszíti a lehetőséget arra, hogy új hozzáférési tokenet kérjen a meglévő refresh token segítségével, így újra be kell jelentkezniük.

Sikeres végrehajtás esetén a rendszer nem ad vissza tartalmat, csak egy sikeres státuszkódot. A művelet után egy naplóbejegyzés is készül, amely rögzíti a törlés tényét.



21. ábra - /api/DeleteAllRefreshToken végpont válasza

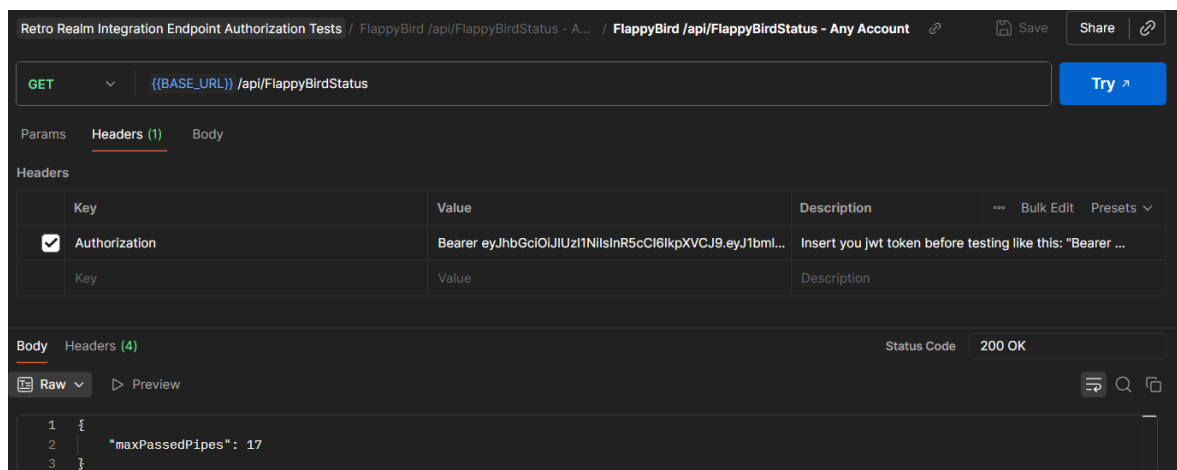
Flappy Bird státusz lekérdezése végpont – /api/FlappyBirdStatus (🔒)

Ez a végpont a bejelentkezett felhasználó Flappy Bird játékhoz tartozó statisztikájának lekérdezésére szolgál. A kliens ezen keresztül tudja megkapni a játékos eddigi legjobb eredményét.

A kérés HTTP GET módszerrel történik a /api/FlappyBirdStatus útvonalon. A végpont hitelesítést igényel, így csak bejelentkezett felhasználók érhetik el.

A működés során a rendszer a JWT tokenből kiolvassa a felhasználó nevét, majd ennek alapján lekéri az adott játékoshoz tartozó adatokat az adatbázisból. Ha a felhasználóhoz nem tartozik ilyen rekord, a rendszer hibát ad vissza, ellenkező esetben visszaküldi az adatokat.

A válasz egy objektum, amely a játékos által elért legnagyobb átjutott csövek számát tartalmazza. Ez az adat a kliens oldalon statisztika vagy eredmény megjelenítésére használható.



22. ábra - /api/FlappyBirdStatus lekérés végpont válasza

Flappy Bird státusz frissítése végpont – /api/FlappyBirdStatus (🔒)

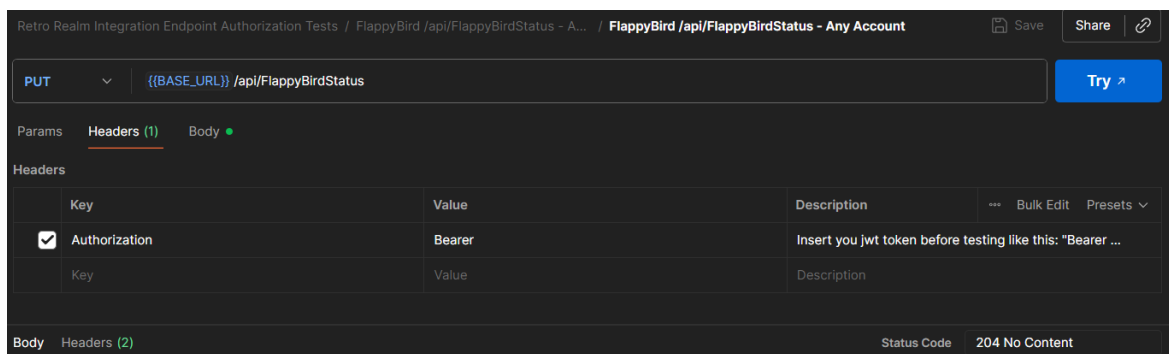
Ez a végpont a játék során elért eredmények frissítésére szolgál. A kliens csak abban az esetben küld kérést, ha az aktuális eredmény meghaladja a korábban elért rekordot.

A kérés HTTP PUT módszerrel történik a /api/FlappyBirdStatus útvonalon. A végpont használatához bejelentkezés szükséges.

A működés során a rendszer a tokenből azonosítja a felhasználót, majd megkeresi a hozzá tartozó jelenlegi statisztikát. Bár a kliens előzetesen ellenőrzi az új rekordot, a backend ezt minden esetben újra validálja. Az új érték csak akkor kerül mentésre, ha valóban nagyobb, mint a korábban eltárolt eredmény.

Sikeres frissítés esetén a rendszer nem ad vissza tartalmat, csak egy sikeres státuszkódot. Ha a felhasználóhoz nem tartozik státusz, akkor hibát jelez.

A művelet során a rendszer naplózza a frissítést, valamint ellenőrzi a teljesített kéréseket is.



23. ábra - /api/FlappyBirdStatus frissítés végpont válasza

Kihívások lekérdezése végpont – /api/GetAchievements (🔒)

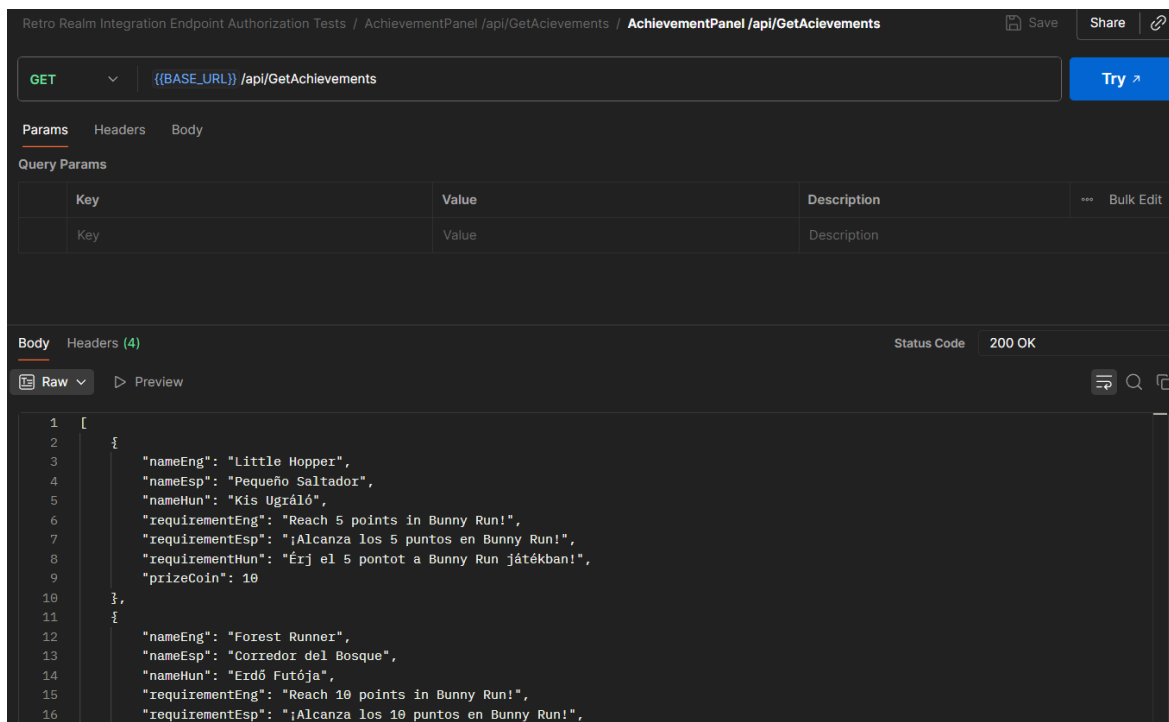
Ez a végpont a játékokhoz tartozó kihívások adatainak lekérdezésére szolgál. Az itt visszaadott információk az Achievements oldalon kerülnek felhasználásra, ahol a rendszer felsorolja az összes elérhető kihívást a felhasználó számára.

A kérés HTTP GET módszerrel történik a /api/GetAchievements útvonalon. A végpont nem igényel hitelesítést, így bejelentkezés nélkül is elérhető. Ez lehetővé teszi, hogy a felhasználók szabadon megtekinthessék a kihívásokat.

A működés során a rendszer lekéri az összes kihívást az adatbázisból, majd az adatokat feldolgozza. A különböző játékokhoz tartozó követelmények szöveges formában kerülnek összeállításra, több nyelven is, így a kliens közvetlenül felhasználható adatokat kap.

A válasz egy lista, amely minden kihíváshoz tartalmazza annak nevét három különböző nyelven, a megszerezhető Coin jutalmat, valamint a teljesítési feltételt szintén több nyelven.

A végpont meghívása után a rendszer egy naplóbejegyzést is létrehoz, amely rögzíti a lekérdezés tényét.



The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** `{{BASE_URL}} /api/GetAchievements`
- Status Code:** 200 OK
- Body (Raw):**

```
[
  {
    "nameEng": "Little Hopper",
    "nameEsp": "Pequeño Saltador",
    "nameHun": "Kis Ugráló",
    "requirementEng": "Reach 5 points in Bunny Run!",
    "requirementEsp": "¡Alcanza los 5 puntos en Bunny Run!",
    "requirementHun": "Érj el 5 pontot a Bunny Run játékban!",
    "prizeCoin": 10
  },
  {
    "nameEng": "Forest Runner",
    "nameEsp": "Corredor del Bosque",
    "nameHun": "Erdő Futója",
    "requirementEng": "Reach 10 points in Bunny Run!",
    "requirementEsp": "¡Alcanza los 10 puntos en Bunny Run!",
    "prizeCoin": 10
  }
]
```

24. ábra - /api/GetAchievements végpont válasza

Felhasználói adatok lekérdezése végpont – /api/GetUserData (🔒)

Ez a végpont egy adott felhasználóhoz tartozó összes fontos adat lekérdezésére szolgál. Az itt visszaadott információk a profil oldalon kerülnek felhasználásra, ahol a rendszer megjeleníti a felhasználó adatait, statisztikáit és elért eredményeit.

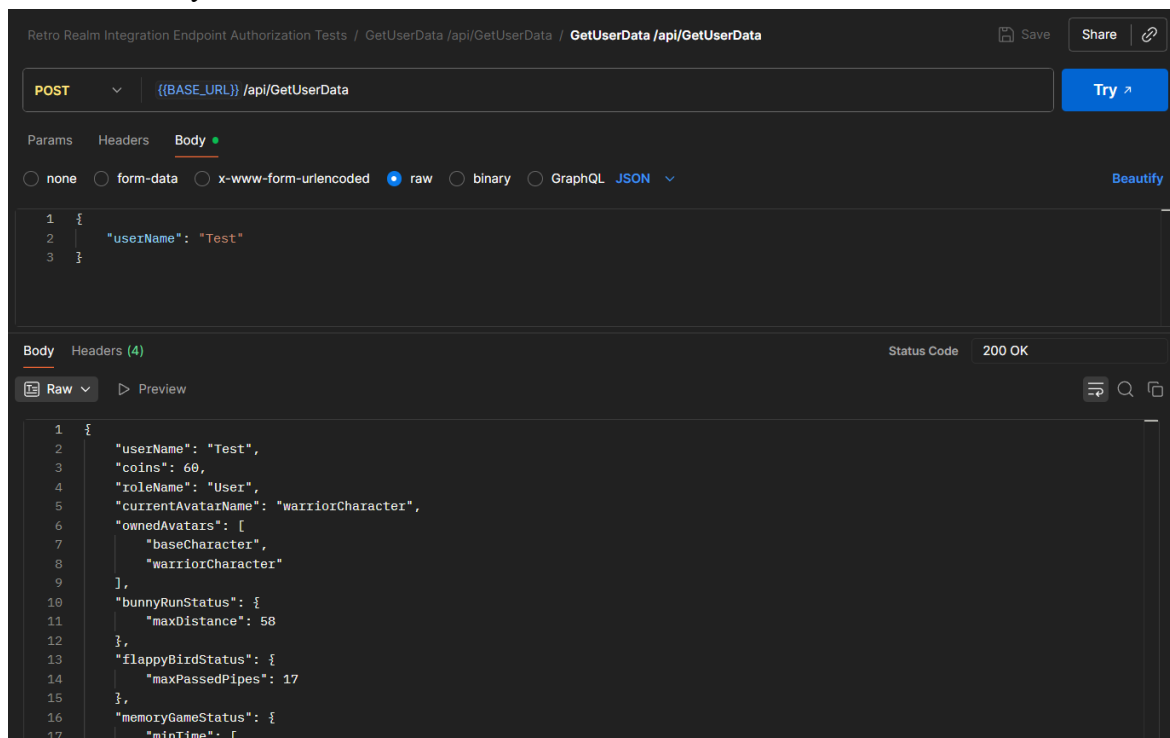
A kérés HTTP POST módszerrel történik a /api/GetUserData útvonalon. A végpont nem igényel hitelesítést, a felhasználó azonosítása a kérés törzsében megadott felhasználónév alapján történik.

A működés során a rendszer megkeresi a megadott felhasználót az adatbázisban, majd betölti a hozzá tartozó kapcsolódó adatokat is. Ide tartozik a szerepkör, az aktuális avatar, valamint az egyes játékokhoz tartozó állapotok. Emellett feldolgozásra kerülnek a felhasználó által birtokolt avatarok és a teljesített kihívások is.

A válasz egy összetett objektum, amely tartalmazza a felhasználónév, Coin egyenleg, szerepkör nevét, az aktuális avatar nevét, a birtokolt avatarok listáját, valamint az egyes játékok statisztikáit. Ezen felül a teljesített kihívások is megjelennek több nyelven.

Ha a megadott felhasználó nem található, a rendszer hibát ad vissza. Sikeres lekérdezés esetén az adatok közvetlenül felhasználhatók a profil oldal megjelenítéséhez.

A végpont meghívása után a rendszer egy naplóbemjegyzést is létrehoz, amely rögzíti a lekérdezés tényét.



25. ábra - /api/GetUserData végpont válasza

Felhasználónév lekérdezése végpont – /api/GetUserName (🔒)

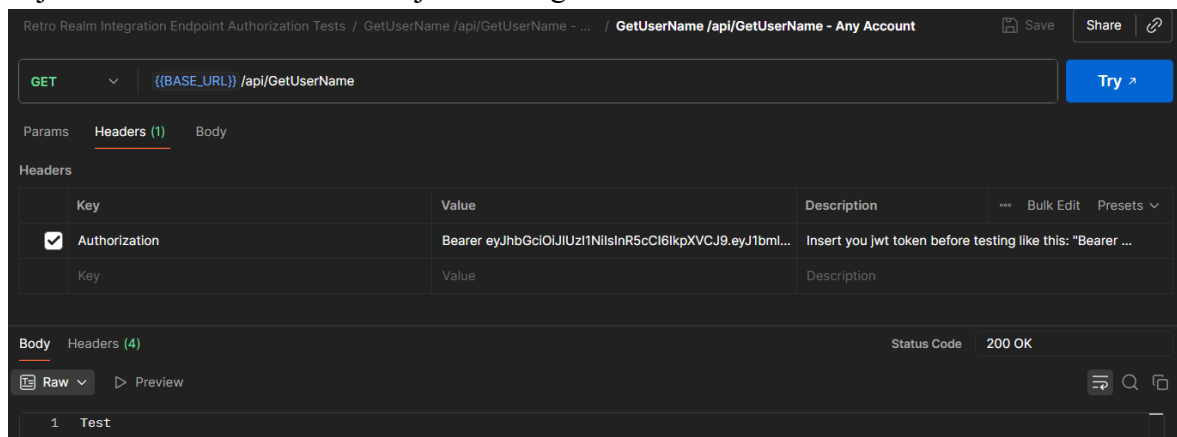
Ez a végpont a bejelentkezett felhasználó nevének lekérdezésére szolgál. Elsősorban olyan esetekben használjuk, amikor a kliensnek szüksége van a felhasználó nevére, de azt nem tárolja külön.

A kérés HTTP GET módszerrel történik a /api/GetUserName útvonalon. A végpont hitelesítést igényel, így csak bejelentkezett felhasználók érhetik el.

A működés során a rendszer a JWT tokenben található adatokból olvassa ki a felhasználó nevét. Ehhez a tokenben tárolt claim-ek kerülnek feldolgozásra, amelyek közül a megfelelő mező tartalmazza a felhasználó azonosításához szükséges nevet.

Sikeres lekérdezés esetén a rendszer visszaadja a felhasználó nevét egyszerű szöveggént. Ha a token nem tartalmazza a szükséges adatot, a rendszer hibát jelez.

Ez a végpont gyakran használható például a felhasználói felület fejlécében, ahol a bejelentkezett felhasználó neve jelenik meg.



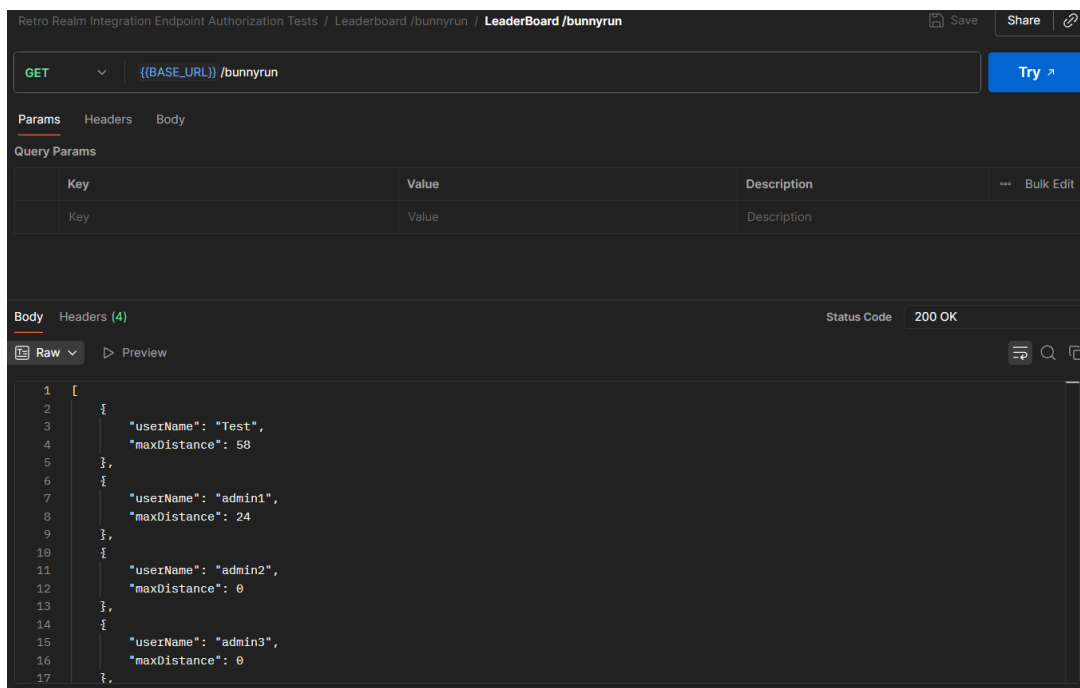
26. ábra - /api/GetUserName végpont válasza

Bunny Run ranglista lekérdezése végpont – /bunnyrun (🐇)

Ez a végpont a Bunny Run játék ranglistájának lekérdezésére szolgál. A Leaderboards oldalon kerül felhasználásra, ahol a játékosok megtekinthetik egymás teljesítményét.

A kérés HTTP GET módszerrel történik a /bunnyrun útvonalon. A végpont nem igényel hitelesítést, így bárki számára szabadon elérhető.

A működés során a rendszer lekérdezi a játékosokhoz tartozó Bunny Run státuszokat, majd ezeket a maximálisan elért távolság alapján rendezi csökkenő sorrendbe. Sikeres lekérdezés esetén a rendszer egy listát ad vissza, amely tartalmazza a felhasználó nevét és az általa elért legnagyobb távolságot. Ez lehetővé teszi a játékosok közötti egyszerű összehasonlítást.



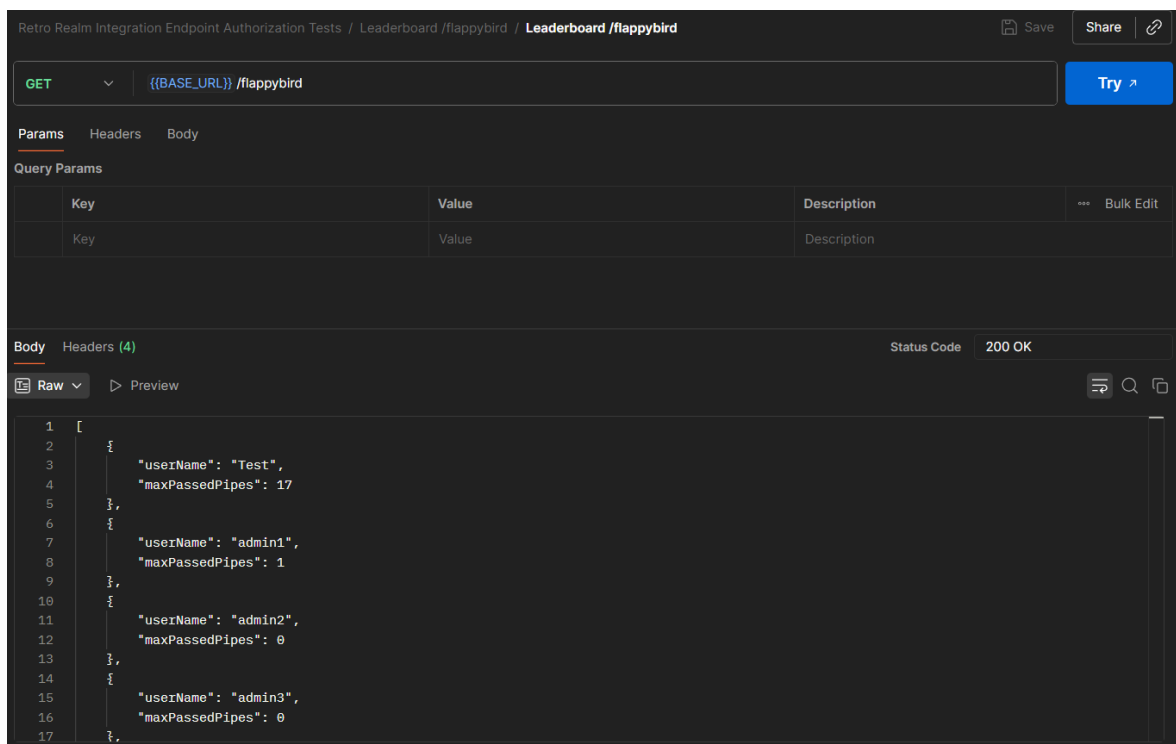
27. ábra - /bunnyrun végpont válasza

Flappy Bird ranglista lekérdezése végpont – /flappybird (🐦)

Ez a végpont a Flappy Bird játék ranglistáját szolgáltatja. A Leaderboards oldalon jelenik meg, ahol a játékosok láthatják a legjobb eredményeket.

A kérés HTTP GET módszerrel történik a /flappybird útvonalon. A végpont nem igényel hitelesítést.

A rendszer a játékosokhoz tartozó Flappy Bird státuszokat dolgozza fel, majd a maximálisan áthaladt csövek száma alapján rendezi őket csökkenő sorrendbe. Sikeres válasz esetén a kliens egy listát kap vissza, amely tartalmazza a felhasználó nevét és az elért eredményt.



28. ábra - /flappybird végpont válasza

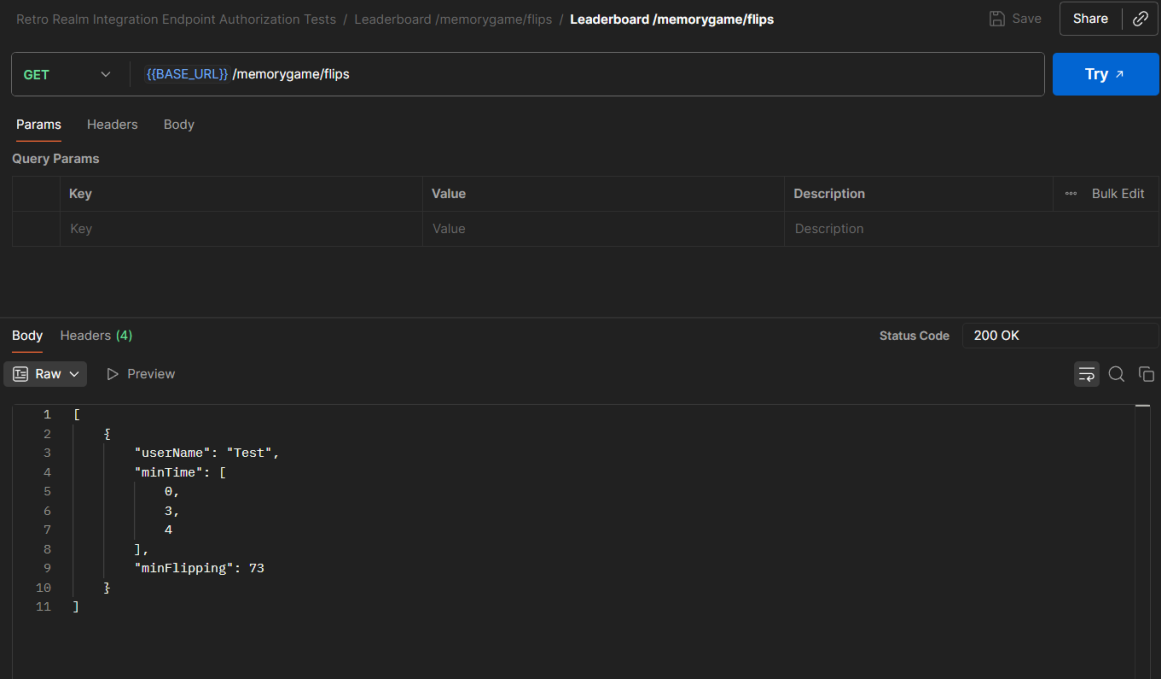
Memória játék ranglista (felfordítás) végpont – /memorygame/flips (🔒)

Ez a végpont a memóriajáték ranglistáját adja vissza a lépésszám alapján. A Leaderboards oldalon kerül megjelenítésre.

A kérés HTTP GET módszerrel történik a /memorygame/flips útvonalon, és nem igényel hitelesítést.

A működés során a rendszer azokat az eredményeket veszi figyelembe, ahol a játékos már teljesítette a játékot, majd ezeket a legkevesebb felfordítás alapján növekvő sorrendbe rendezi.

A válasz egy lista, amely tartalmazza a felhasználó nevét és a szükséges lépések számát.



Retro Realm Integration Endpoint Authorization Tests / Leaderboard /memorygame/flips / Leaderboard /memorygame/flips

GET {{BASE_URL}} /memorygame/flips Try

Params Headers Body

Query Params

Key	Value	Description	...	Bulk Edit
Key	Value	Description		

Body Headers (4) Status Code 200 OK

Raw Preview

```
1 [
2   {
3     "userName": "Test",
4     "minTime": [
5       0,
6       3,
7       4
8     ],
9     "minFlipping": 73
10  }
11 ]
```

29. ábra - /memorygame/flips végpont válasza

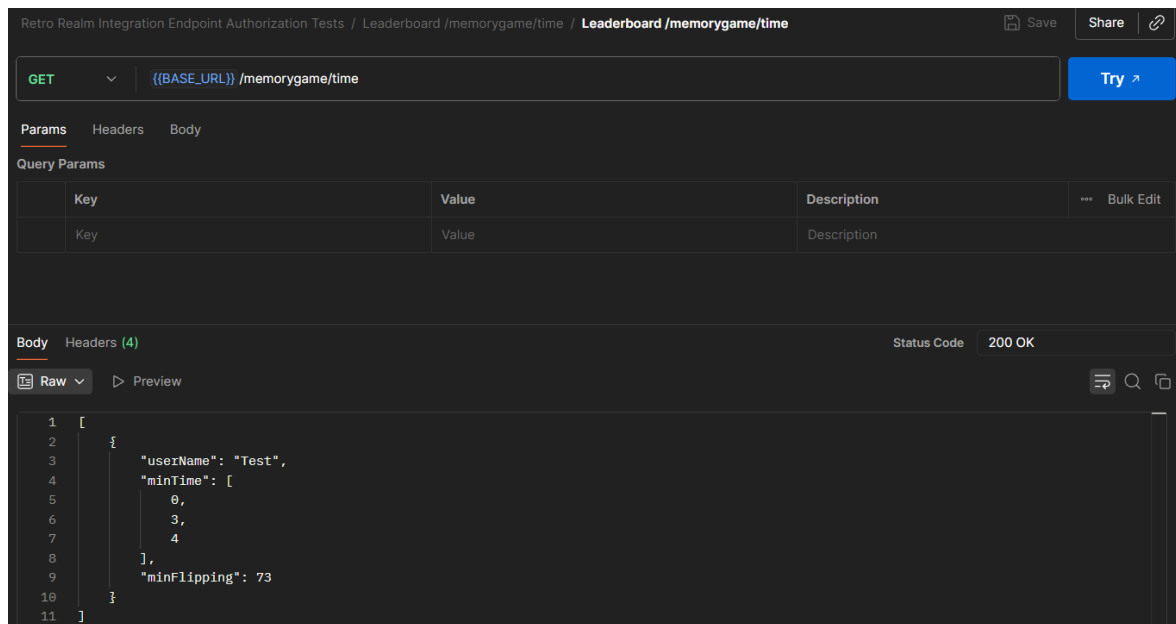
Memória játék ranglista (idő) végpont – /memorygame/time (🔒)

Ez a végpont a memóriajáték ranglistáját szolgáltatja idő alapján. Szintén a Leaderboards oldalon használjuk.

A kérés HTTP GET módszerrel történik a /memorygame/time útvonalon. A végpont bárki számára elérhető.

A rendszer csak az érvényes, befejezett játékokat veszi figyelembe, majd ezeket a teljesítési idő alapján növekvő sorrendbe rendezi.

Sikeres lekérdezés esetén a válasz tartalmazza a játékos nevét és a legjobb idejét.



30. ábra - /memorygame/time végpont válasza

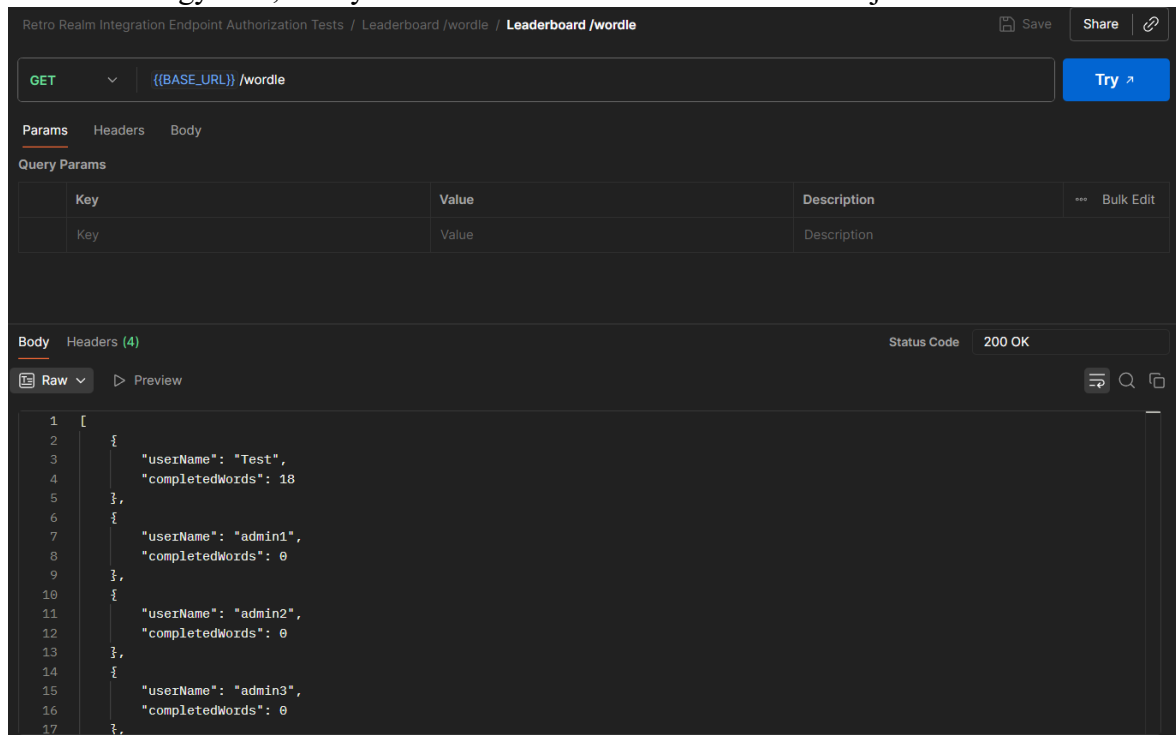
Wordle ranglista lekérdezése végpont – /wordle (🔗)

Ez a végpont a Wordle játék ranglistáját biztosítja. A Leaderboards oldalon jelenik meg.

A kérés HTTP GET módszerrel történik a /wordle útvonalon, és nem igényel hitelesítést.

A rendszer a játékosok teljesített szavainak számát veszi alapul, majd ezeket csökkenő sorrendbe rendezi.

A válasz egy lista, amely tartalmazza a felhasználó nevét és a teljesített szavak számát.



31. ábra - /wordle végpont válasza

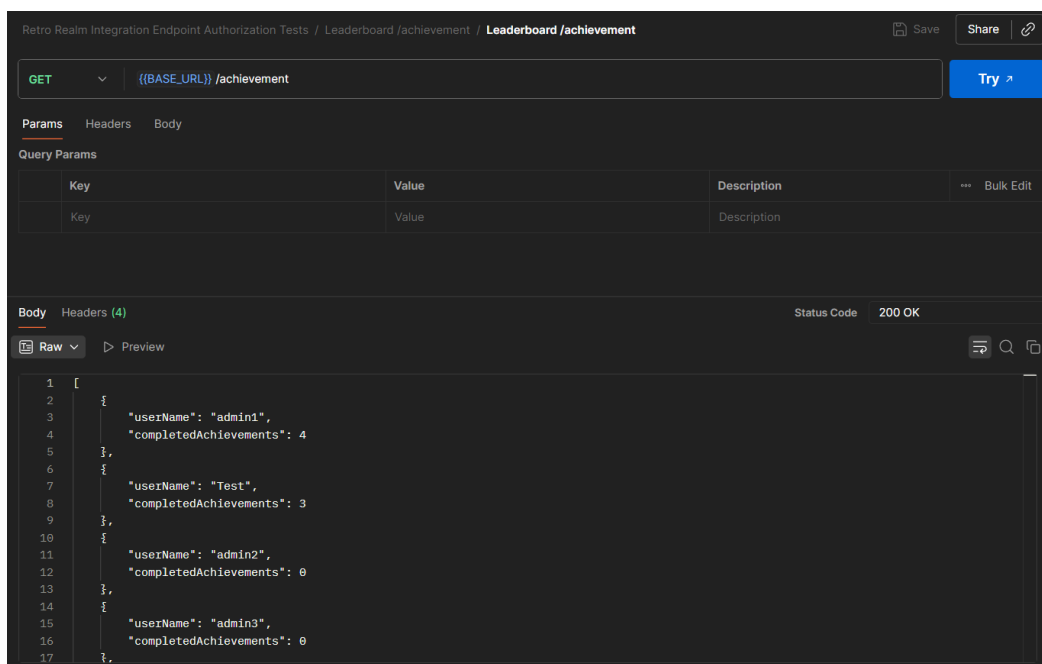
Achievement ranglista lekérdezése végpont – /achievement (🔒)

Ez a végpont az achievement alapú ranglistát adja vissza. A Leaderboards oldalon kerül felhasználásra.

A kérés HTTP GET metódussal történik a /achievement útvonalon. A végpont nem igényel hitelesítést.

A működés során a rendszer a felhasználók által megszerzett kihívások számát számolja össze, majd ez alapján csökkenő sorrendbe rendezi őket.

A válasz tartalmazza a felhasználó nevét és az összes teljesített achievement számát, így átfogó képet ad a játékos aktivitásáról.



32. ábra - /achievement végpont válasza

Bejelentkezés végpont – /api/Login (🔒)

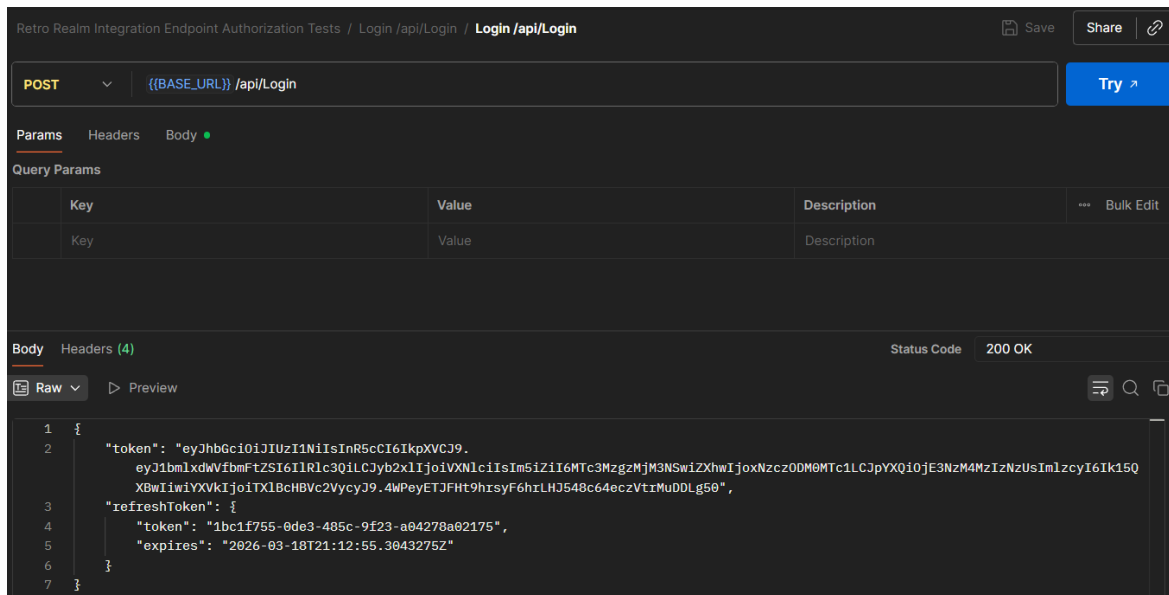
Ez a végpont a felhasználók bejelentkezésére szolgál. A rendszer ezen keresztül hitelesíti a felhasználót, és sikeres bejelentkezés esetén visszaadja a további kommunikációhoz szükséges tokeneket.

A kérés HTTP POST módszerrel történik a /api/Login útvonalon. A végpont nem igényel előzetes hitelesítést, így anonim módon is elérhető.

A kérés törzsében a felhasználónév és a jelszó kerül elküldésre. A működés során a rendszer megkeresi a felhasználót az adatbázisban, majd ellenőrzi a megadott jelszót a tárolt hash alapján.

Sikeres hitelesítés esetén a rendszer egy JWT token-t és egy refresh token-t generál. A JWT token tartalmazza a felhasználó azonosításához szükséges adatokat, míg a refresh token lehetővé teszi egy új token igénylését a későbbiekben. A refresh token eltárolásra kerül az adatbázisban.

A válasz tartalmazza mindkét token, amelyeket a kliens a további kérések során használ fel. Sikertelen bejelentkezés esetén a rendszer hibát ad vissza, ha a megadott adatok nem megfelelőek.



33. ábra - /api/Login végpont válasza

Kijelentkezés végpont – /api/Logout (🔒)

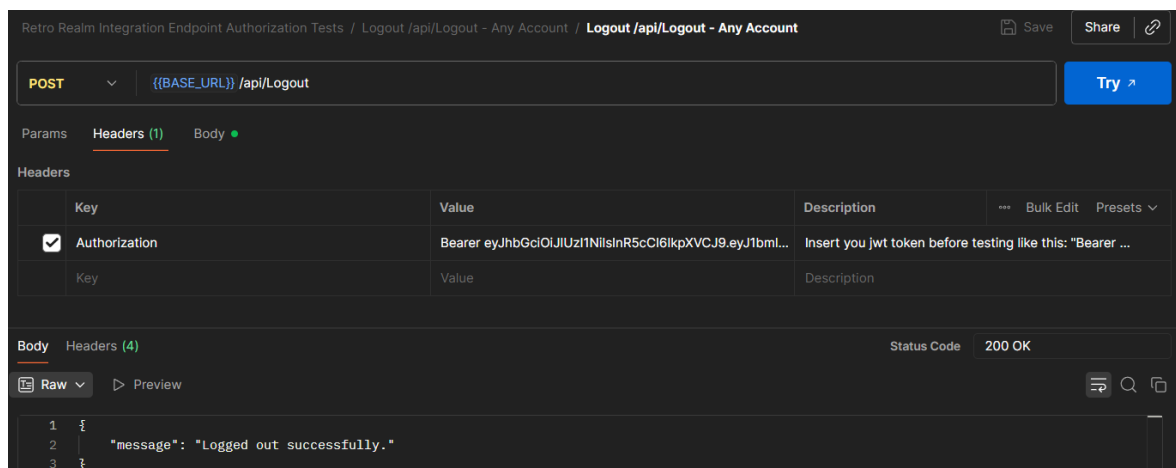
Ez a végpont a felhasználók kijelentkeztetésére szolgál. A rendszer ezen keresztül érvényteleníti a felhasználóhoz tartozó refresh token, így megszüntetve a további automatikus hitelesítés lehetőségét.

A kérés HTTP POST módszerrel történik a /api/Logout útvonalon. A végpont nem igényel hitelesítést.

A kérés törzsében a refresh token kerül elküldésre. A működés során a rendszer megkeresi ezt a token az adatbázisban, majd ha létezik, eltávolítja azt.

Sikeres végrehajtás esetén a rendszer egy egyszerű visszajelzést ad, amely jelzi, hogy a kijelentkezés megtörtént. Ha a token nem található, a rendszer akkor is sikeres választ ad vissza, azonban tényleges törlés nem történik.

Ez a végpont biztosítja, hogy a felhasználó kijelentkezés után ne tudjon új token igényelni a korábbi refresh token segítségével.



34. ábra - /api/Logout végpont válasza

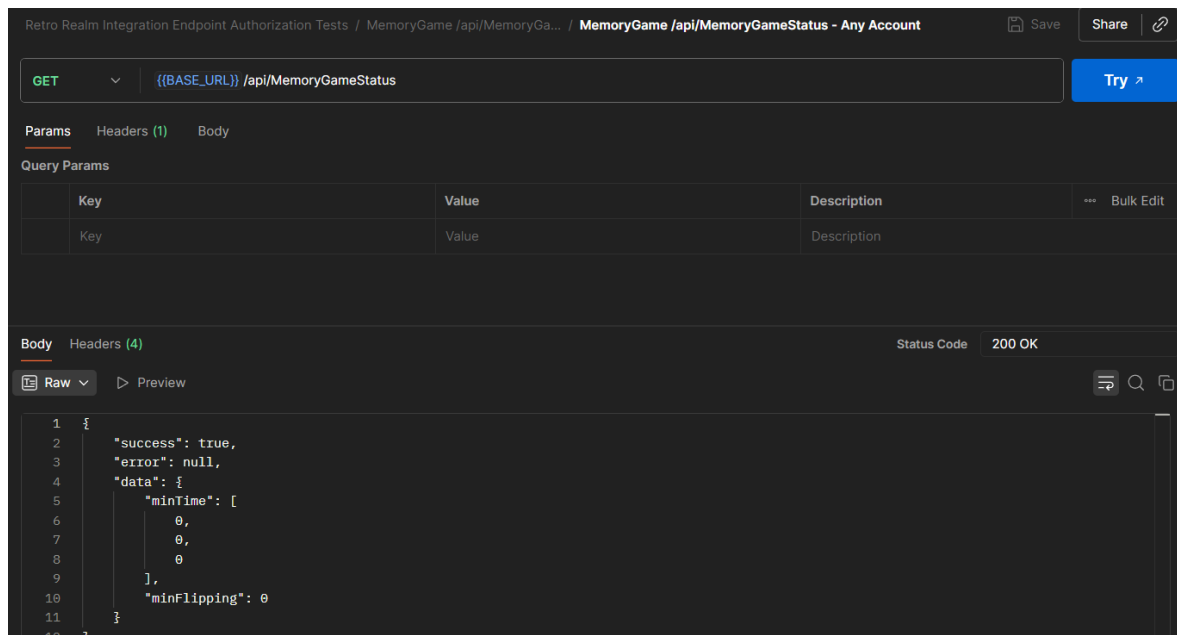
Memória játék státusz lekérdezése végpont – /api/MemoryGameStatus (🔒)

Ez a végpont a bejelentkezett felhasználó memóriajátékhoz tartozó statisztikáinak lekérdezésére szolgál. A játék során elért legjobb eredmények megjelenítéséhez használjuk.

A kérés HTTP GET módszerrel történik a /api/MemoryGameStatus útvonalon. A végpont hitelesítést igényel, így csak bejelentkezett felhasználók érhetik el.

A működés során a rendszer a JWT tokenben található adatok alapján azonosítja a felhasználót, majd lekérdezi a hozzá tartozó memóriajáték státuszt az adatbázisból.

Sikeres lekérdezés esetén a válasz tartalmazza a legjobb lépésszámot, valamint a leggyorsabb teljesítési időt. Ha a felhasználóhoz nem tartozik ilyen adat, a rendszer hibát jelez.



35. ábra - /api/MemoryGameStatus lekérés végpont válasza

Memória játék státusz frissítése végpont – /api/MemoryGameStatus (🔒)

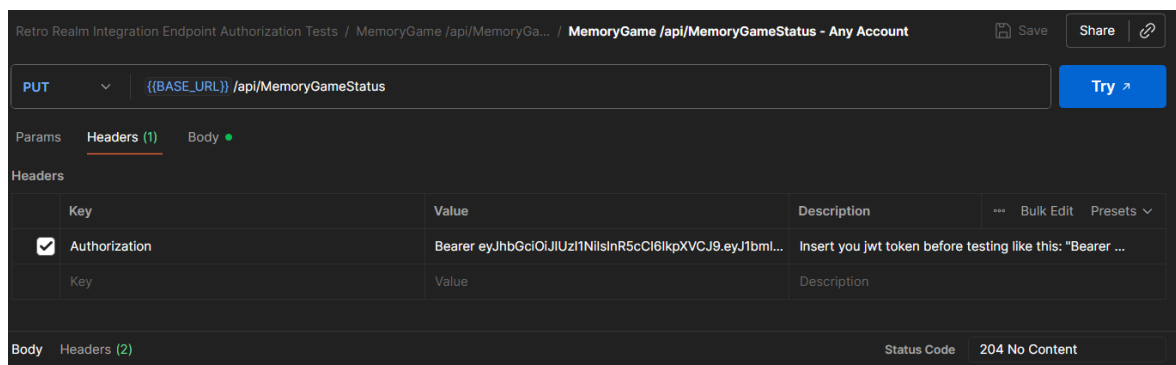
Ez a végpont a memóriajáték során elért eredmények frissítésére szolgál. A kliens akkor hívja meg, amikor a játékos új eredményt ér el.

A kérés HTTP PUT módszerrel történik a /api/MemoryGameStatus útvonalon. A végpont hitelesítést igényel.

A kérés törzsében a frissített adatok kerülnek elküldésre, amely tartalmazza a lépésszámot és a teljesítési időt. A működés során a rendszer összehasonlítja az új értékeket a korábban eltárolt adatokkal.

Frissítés csak abban az esetben történik, ha az új eredmény jobb a korábbinál, vagyis kevesebb lépésből vagy rövidebb idő alatt sikerült teljesíteni a játékot. Emellett a rendszer ellenőrzi az achievementek teljesülését is.

Sikeres frissítés esetén a végpont nem ad vissza tartalmat, csak egy sikeres státuszkódot. Ha a státusz nem található, a rendszer hibát jelez.



36. ábra - /api/MemoryGameStatus frissítés végpont válasza

Avatár vásárlás végpont – /api/PurchaseAvatar (🔒)

Ez a végpont lehetővé teszi a felhasználók számára, hogy avatárt vásároljanak a játékban megszerzett coinok felhasználásával.

A kérés HTTP POST módszerrel történik a /api/PurchaseAvatar útvonalon. A végpont hitelesítést igényel, így csak bejelentkezett felhasználók érhetik el.

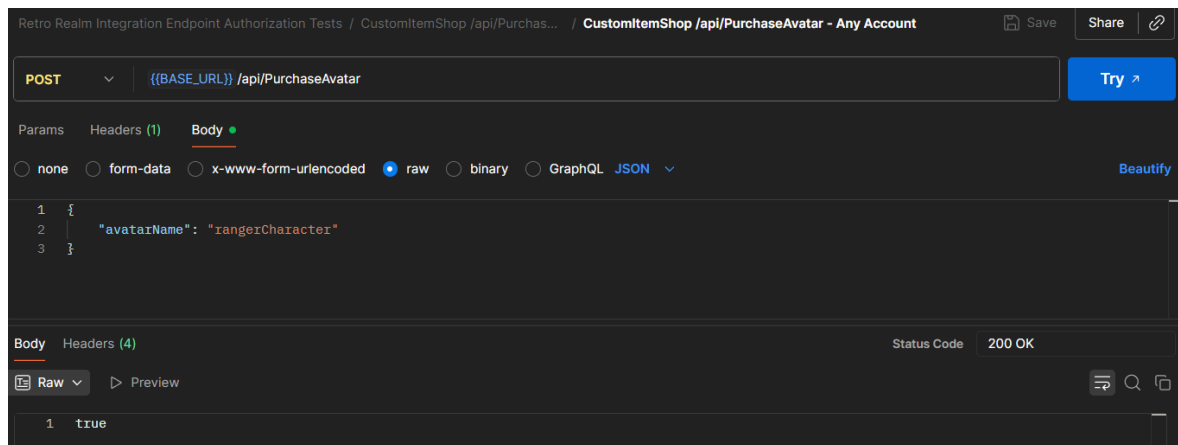
A működés során a rendszer a JWT tokenből kinyeri a felhasználó nevét, majd megkeresi az adatbázisban a felhasználót és a megvásárolni kívánt avatárt annak neve alapján. Ezt követően több ellenőrzés történik:

- létezik-e a felhasználó,
- létezik-e az avatár,
- rendelkezik-e a felhasználó elegendő coin-nal,
- nem birtokolja-e már az adott avatárt.

Amennyiben minden feltétel teljesül, a rendszer hozzáadja az avatárt a felhasználó tulajdonaihoz, majd levonja a vásárláshoz szükséges coin mennyiséget.

Sikeres vásárlás esetén a rendszer visszaigazolást ad. Hibás esetekben a megfelelő hibaüzenet kerül visszaadásra, például ha az avatár vagy a felhasználó nem található, nincs elegendő coin, vagy a felhasználó már rendelkezik az adott avatárral.

Ez a végpont a CustomItemShop-ban kerül használatra, ahol a játékosok megvásárolhatják az avatárokat.



37. ábra - /api/PurchaseAvatar végpont válasza

Token frissítés végpont – /api/RefreshToken (🔒)

Ez a végpont a lejáró vagy lejárt JWT token megújítására szolgál refresh token segítségével. Segítségével a felhasználónak nem kell újra bejelentkeznie, ha a hozzáférési token lejár.

A kérés HTTP POST metódussal történik a /api/RefreshToken útvonalon. A végpont nem igényel hitelesítést.

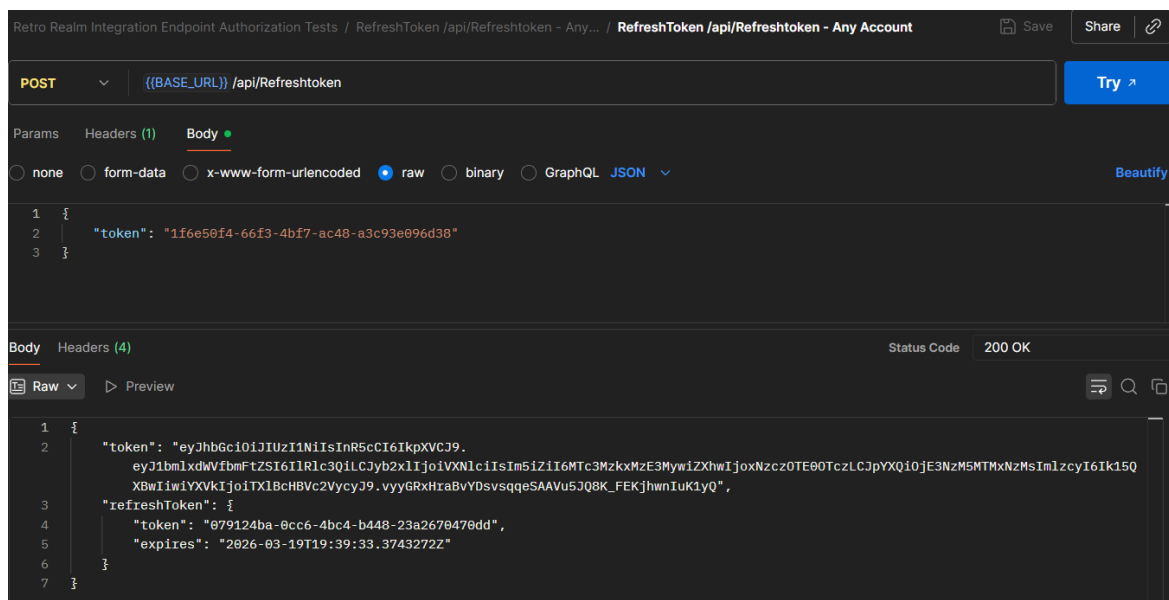
A működés során a rendszer megkeresi az adatbázisban a megadott refresh token. Ezután két fontos ellenőrzést végez:

- létezik-e a refresh token az adatbázisban,
- nem járt-e le a refresh token érvényessége.

Amennyiben a token érvényes, a rendszer új JWT tokenet és új refresh tokenet generál. A régi refresh token törlésre kerül, és az új kerül eltárolásra az adatbázisban.

Sikeres művelet esetén a válasz tartalmazza az új JWT token és az új refresh token. Hibás esetekben a rendszer hibát ad vissza, például ha a refresh token érvénytelen vagy lejárt.

Ez a végpont automatikusan kerül használatra a kliens oldalon, amikor a felhasználó tokenje lejár, és azt frissíteni kell.



38. ábra - /api/RefreshToken végpont válasza

Regisztráció végpont – /api/Register (🔒)

Ez a végpont új felhasználók regisztrációjára szolgál a rendszerben. A felhasználó ezen keresztül hozhat létre új fiókot, amely később bejelentkezésre és a rendszer funkcióinak használatára jogosít.

A kérés HTTP POST módszerrel történik a /api/Register útvonalon. A végpont nem igényel hitelesítést, így bárki elérheti, aki még nem rendelkezik fiókkal.

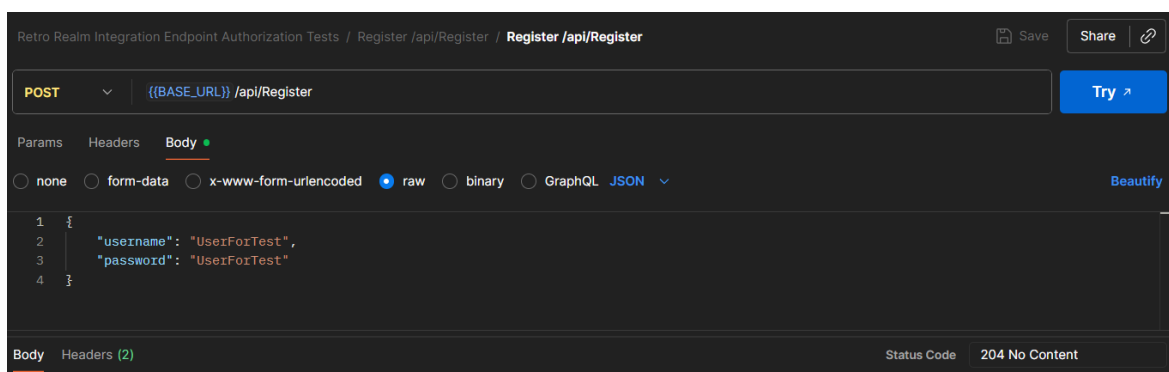
A működés során a kliens egy regisztrációs adatokat tartalmazó objektumot küld (például felhasználónév és jelszó). A rendszer először ellenőrzi, hogy létezik-e már ilyen felhasználónév az adatbázisban. Ha igen, a regisztráció sikertelen lesz.

Sikeres ellenőrzés esetén a rendszer létrehozza az új felhasználót. A jelszó nem kerül nyers formában tárolásra, hanem titkosítva (hash-elve) mentésre kerül az adatbázisba a biztonság érdekében.

A felhasználó létrehozása után a rendszer automatikusan inicializálja a szükséges játékstatisztikákat (például BunnyRun, FlappyBird, MemoryGame és Wordle státuszok), hogy a felhasználó azonnal használni tudja a játékokat.

Sikeres regisztráció esetén a rendszer létrehozza a felhasználót és visszajelzést ad a sikeres műveletről. Ha a felhasználónév már foglalt, vagy adatbázis hiba történik, a rendszer hibát jelez.

Ez a végpont a Regisztráció felületen kerül felhasználásra.



39. ábra - /api/Register végpont válasza

Karakter beállítása végpont – /api/setCharacter (🔒)

Ez a végpont lehetővé teszi a felhasználó számára, hogy kiválassza és beállítsa az aktuálisan használt karakterét (avatarját). A kiválasztott karakter jelenik meg például a profil oldalon vagy a játék során.

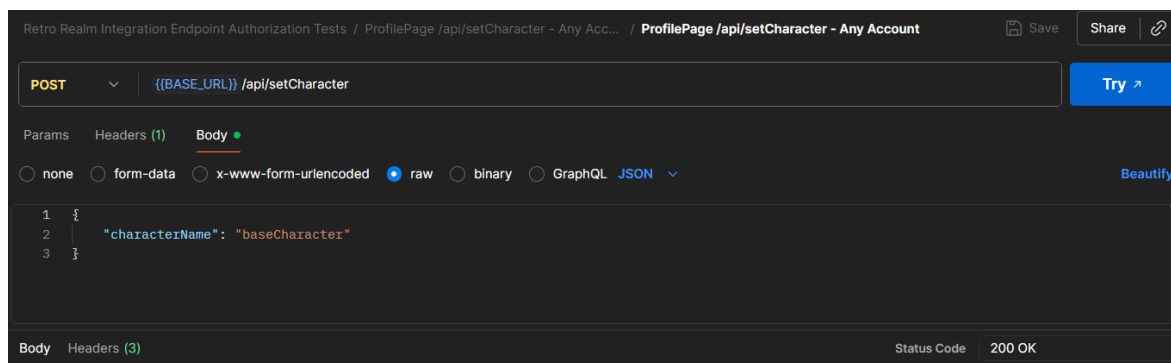
A kérés HTTP POST módszerrel történik a /api/setCharacter útvonalon. A végpont hitelesítést igényel, így csak bejelentkezett felhasználók érhetik el.

A működés során a rendszer a JWT tokenből kinyeri a felhasználó nevét, majd a kérésben kapott karakter (avatar) nevét használja a művelet végrehajtásához. Először ellenőrzi, hogy a felhasználó létezik-e, majd ezután, hogy a megadott karakter létezik-e az adatbázisban.

Ezt követően a rendszer azt is ellenőrzi, hogy a felhasználó valóban birtokolja-e az adott karaktert. Amennyiben nem, a művelet sikertelen lesz. Ha minden feltétel teljesül, a rendszer beállítja a kiválasztott karaktert a felhasználó aktuális karaktereként, és elmenti a módosítást az adatbázisban.

Sikeres művelet esetén a végpont visszajelzést ad a sikeres beállításról. Hiba esetén – például ha a felhasználó vagy a karakter nem található, vagy a felhasználó nem rendelkezik az adott karakterrel – a rendszer megfelelő hibát ad vissza.

Ez a végpont a profil oldalon kerül felhasználásra.



40. ábra - /api/setCharacter végpont válasza

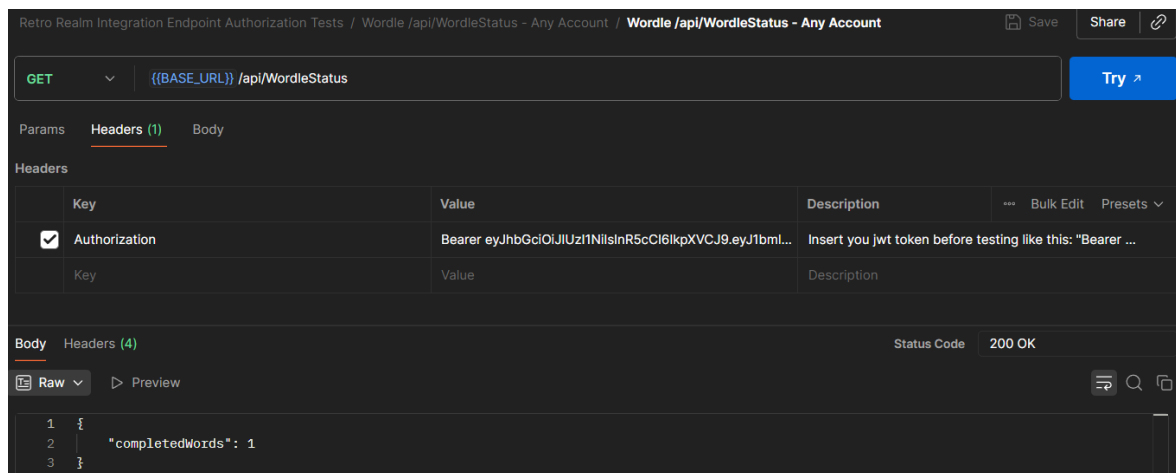
Wordle státusz lekérése végpont – /api/WordleStatus (🔒)

Ez a végpont a Wordle játékhoz tartozó felhasználói statisztikák lekérdezésére szolgál. Segítségével a rendszer nyomon követi, hogy a felhasználó hány szót teljesített sikeresen.

A végpont HTTP GET metódust támogat a /api/WordleStatus útvonalon. A végpont hitelesítést igényel, így csak bejelentkezett felhasználók érhetik el.

GET kérés esetén a rendszer a JWT tokenben található felhasználónév alapján azonosítja a felhasználót, majd lekérdezi az adatbázisból a hozzá tartozó Wordle státuszt. Sikeres lekérdezés esetén visszaadja a felhasználó által teljesített szavak számát. Ha a státusz nem található, a rendszer hibát jelez.

Ez a végpont a Wordle játék során kerül felhasználásra.



41. ábra - /api/WordleStatus lekérés végpont válasza

Wordle státusz frissítése végpont – /api/WordleStatus (🐞)

Ez a végpont a Wordle játékhoz tartozó felhasználói statisztikák frissítésére szolgál. Segítségével a rendszer frissíti a felhasználó teljesített szavainak számát.

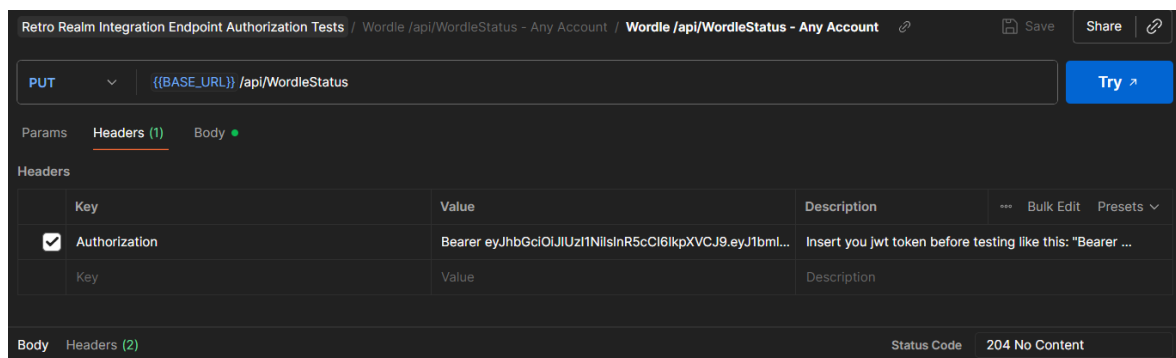
A végpont HTTP PUT módszert támogat a /api/WordleStatus útvonalon. A végpont hitelesítést igényel, így csak bejelentkezett felhasználók érhetik el.

A kérés esetén a rendszer a JWT tokenben található felhasználónév alapján azonosítja a felhasználót, majd lekérdezi az adatbázisból a hozzá tartozó Wordle státuszt. A rendszer automatikusan növeli a teljesített szavak számát minden sikeres játék után.

A frissítés során a rendszer ellenőrzi az achievement-eket is, és szükség esetén új teljesítményeket old fel a felhasználó számára.

Sikeres frissítés esetén a végpont nem ad vissza tartalmat. Ha a felhasználóhoz tartozó státusz nem található, a rendszer hibát jelez.

Ez a végpont a Wordle játék során kerül felhasználásra.



42. ábra - /api/WordleStatus frissítés végpont válasza

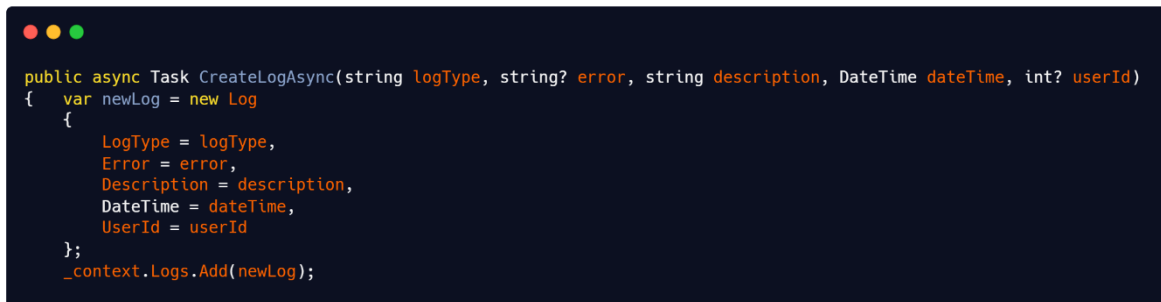
3.5 Egyéb Backend Algoritmusok

Logolási rendszer

A logolási rendszer a backend egyik központi szolgáltatása, amely a rendszer működése során keletkező események, műveletek és hibák rögzítésére szolgál. Segítségével biztosítható a rendszer működésének nyomon követése, a hibák visszakövethetősége, valamint a felhasználói aktivitások monitorozása.

A logolás egy dedikált szolgáltatáson keresztül történik, amely minden fontos esemény esetén meghívásra kerül. A szolgáltatás paraméterként fogadja a log típusát, a leírást, az esetleges hibát, az időbélyeget, valamint opcionálisan a felhasználó azonosítóját. Ez a megközelítés lehetővé teszi, hogy a rendszer egységes formátumban kezelje a különböző típusú eseményeket.

A log létrehozása során a rendszer először egy új log objektumot hoz létre a megadott adatok alapján, amely tartalmazza az esemény típusát, leírását, az esetleges hibát, az esemény időpontját, valamint a felhasználó azonosítóját. Ezt követően az objektum bekerül az adatbázis kontextusba, ahol várakozik a mentésre.

A screenshot of a code editor showing a C# method named 'CreateLogAsync'. The method takes five parameters: 'logType' (string), 'error' (string?), 'description' (string), 'dateTime' (DateTime), and 'userId' (int?). Inside the method, a new 'Log' object is created and its properties are set: 'LogType', 'Error', 'Description', 'DateTime', and 'UserId'. Finally, the object is added to a collection named '_context.Logs' using the 'Add' method.

```
public async Task CreateLogAsync(string logType, string? error, string description, DateTime dateTime, int? userId)
{
    var newLog = new Log
    {
        LogType = logType,
        Error = error,
        Description = description,
        DateTime = dateTime,
        UserId = userId
    };
    _context.Logs.Add(newLog);
}
```

43. ábra - Log objektum létrehozása

A feldolgozás következő lépésében a rendszer a log típusától függően meghívja a megfelelő naplózási szintet a beépített logger segítségével. Amennyiben a log típusa hiba, a rendszer hibaként rögzíti az eseményt, egyéb esetekben információs szintű naplózás történik. Ez lehetővé teszi, hogy a futó alkalmazás során valós időben is követhetőek legyenek az események.

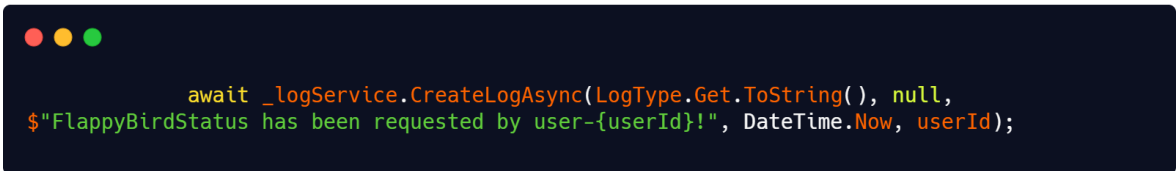
A folyamat végén az adatbázis mentési művelete lefut, amely fizikailag is eltárolja a logot. Ennek köszönhetően minden esemény tartósan megőrzésre kerül, és később visszakereshetővé válik. Ez különösen fontos hibakeresés, auditálás és rendszerfelügyelet szempontjából.

A logolási rendszer támogatja a logok lekérdezését is. Lehetőség van az összes log beolvasására, amely során a rendszer visszaadja az adatbázisban tárolt összes bejegyzést. Emellett biztosított egy adott log lekérdezése is azonosító alapján. Amennyiben a keresett log nem található, a rendszer hibát jelez, így a hívó fél megfelelően kezelheti ezt az esetet.

A logolási mechanizmus szorosan integrálva van a backend többi szolgáltatásával. Például a játékokhoz tartozó szolgáltatások – mint a Flappy Bird állapotkezelés – minden fontos művelet során naplózást végeznek. Ide tartozik az adatok lekérdezése, létrehozása, frissítése, valamint a hibakezelési ágak is. Ennek eredményeként a rendszer működése teljes mértékben visszakövethetővé válik.

A hibakezelés során a logolási rendszer kiemelt szerepet kap. Amikor egy kivétel történik, a rendszer nemcsak hibát ad vissza, hanem részletes információt is rögzít a logban. Ez magában foglalja a hiba üzenetét, az esemény kontextusát, valamint a felhasználóhoz kapcsolódó adatokat. Ez jelentősen megkönnyíti a hibák azonosítását és javítását.

A logolási folyamat több egymásra épülő lépésből áll. Először a backend szolgáltatás meghívja a logolási metódust, majd a rendszer létrehozza a log objektumot. Ezt követően a megfelelő naplózási szint kiválasztása történik meg, majd végül az adatbázisba mentés zárja a folyamatot. Ez az egységes működés biztosítja a rendszer stabil és átlátható működését.



```
await _logService.CreateLogAsync(LogType.Get.ToString(), null,
    $"FlappyBirdStatus has been requested by user-{userId}!", DateTime.Now, userId);
```

44. ábra - Log szolgáltatás meghívása

A rendszer architektúrális szempontból is jól elkülönített. A különböző üzleti logikát megvalósító szolgáltatások nem közvetlenül kezelik a logolást, hanem egy külön logolási szolgáltatást hívnak meg. Ez biztosítja az egységes működést és a könnyebb karbantarthatóságot.

JWT alapú hitelesítési rendszer

A rendszer hitelesítése JSON Web Token (JWT) alapokon működik, amely lehetővé teszi a felhasználók biztonságos azonosítását és jogosultságaik kezelését. A JWT használatával a backend stateless módon működik, így nincs szükség szerveroldali session tárolásra.

A token generálása a bejelentkezést követően történik egy dedikált szolgáltatáson keresztül. A létrehozott token tartalmazza a felhasználó nevét és szerepkörét, amelyek később a jogosultságkezeléshez szükségesek. A token lejáratí ideje konfiguráció alapján kerül beállításra, és szimmetrikus kulccsal kerül aláírásra a biztonság érdekében.

A rendszer a hozzáférési token mellett refresh token is generál, amely lehetővé teszi új token igénylését újrabemjelentkezés nélkül. Ez javítja a felhasználói élményt és növeli a biztonságot.

A backend minden védett végpont esetén ellenőrzi a token érvényességét, beleértve az aláírást, a lejáratí időt, valamint a kibocsátót és a célközönséget. A tokenből kinyert adatok alapján a rendszer azonosítja a felhasználót.

A rendszer szerepkör alapú jogosultságkezelést is alkalmaz, amely meghatározza, hogy egy adott végpont milyen felhasználók számára érhető el. Ez biztosítja, hogy a különböző jogosultsági szintek megfelelően elkülönüljenek.

```
public string GenerateJwtToken(User user)
{
    var tokenHandler = new JwtSecurityTokenHandler();
    var key = Encoding.UTF8.GetBytes(_configuration["Jwt:Key"]);
    var tokenDescriptor = new SecurityTokenDescriptor
    {
        // Setting the claims
        Subject = new ClaimsIdentity(new[]
        {
            new Claim(ClaimTypes.Name, user.Username),
            new Claim("role", user.Role.Name)
        }),
        // Setting the expireMinutes
        Expires = DateTime.UtcNow.AddMinutes(_configuration.GetValue<int>("Jwt:ExpireMinutes")),
        // Setting the Credentials
        SigningCredentials = new SigningCredentials(new SymmetricSecurityKey(key), SecurityAlgorithms.HmacSha256Signature),
        // Setting the issuer and the audience
        Issuer = _configuration["Jwt:Issuer"],
        Audience = _configuration["Jwt:Audience"]
    };
    var token = tokenHandler.CreateToken(tokenDescriptor);
    return tokenHandler.WriteToken(token);
}
```

45. ábra - JWT token generálás

Data Transfer Objectek használata

A rendszerben a DTO-k (Data Transfer Object) az adatok átvitelére szolgálnak a különböző rétegek között, például a kliens és a backend, illetve a backend belső komponensei között. A DTO-k célja, hogy csak a szükséges adatokat tartalmazzák, és elrejtsek az adatbázis modellek belső felépítését.

A DTO-k használata lehetővé teszi, hogy a rendszer ne közvetlenül az adatbázis entitásokat adja vissza a kliensnek, hanem egy szűrt és biztonságos adatstruktúrát. Ez különösen fontos biztonsági szempontból, mivel így elkerülhető, hogy érzékeny adatok – például jelszavak vagy belső azonosítók – kiszivároghassanak.

A DTO-k emellett segítik a rendszer rugalmasságát is. Ha az adatbázis szerkezete megváltozik, a DTO-k segítségével a külső API változatlan maradhat. Ez csökkenti a kliens oldali módosítások szükségességét.

A backend különböző típusú DTO-t használ különböző célokra. Például léteznek olvasási DTO-k, amelyek adatlekérdezéshez használatosak, valamint létrehozási és frissítési DTO-k, amelyek a kliens által küldött adatok feldolgozására szolgálnak. Ez a szétválasztás biztosítja, hogy minden művelet csak a számára szükséges adatokat kezelje.

A DTO-k használata javítja a kód átláthatóságát és karbantarthatóságát is. A különálló adatstruktúrák egyértelművé teszik, hogy egy adott végpont milyen adatokat vár és milyen adatokat ad vissza.

Összességében a DTO-k fontos szerepet játszanak a rendszer biztonságos, átlátható és jól karbantartható működésében, mivel elválasztják az adatbázis réteget az API rétegtől, és biztosítják a kontrollált adatáramlást.

```
public class LoginDTO
{
    [Required]
    public string Username { get; set; }

    [Required]
    public string Password { get; set; }
}
```

46. ábra - Login DTO

WebSocket alapú valós idejű kommunikáció – backend algoritmus

A rendszer WebSocket alapú kommunikációt használ az online RetroRealm játék valós idejű működésének biztosítására. Ennek segítségével a játékosok folyamatosan látják egymás mozgását és állapotát anélkül, hogy ismételt HTTP kéréseket kellene küldeniük.

A kapcsolat létrejötte után a szerver egy folyamatos ciklusban figyeli a beérkező üzeneteket. A kliens JSON formátumban küldi az adatokat, amelyeket a szerver feldolgoz, majd válaszként visszaküldi az aktuális játékállapotot.

A rendszer minden játékos állapotát egy közös listában tárolja, amely tartalmazza például a pozíciót, mozgást, karaktert és az esetleges szöveges üzeneteket. Amikor egy játékos adatot küld, a szerver frissíti a hozzá tartozó állapotot, vagy új játékosként hozzáadja a listához.

Amennyiben a felhasználó nincs bejelentkezve, a rendszer ideiglenes, anonim játékost hoz létre. Ha később azonosítás történik, az anonim állapot lecserélésre kerül a valódi felhasználói adatokra.

A szerver minden üzenet feldolgozása után visszaküldi az összes aktív játékos aktuális állapotát a kliens számára. Ez biztosítja, hogy minden résztvevő naprakész információval rendelkezzen a többi játékosról.

A rendszer automatikusan kezeli az inaktív adatokat is. A régi vagy nem frissített játékos adatok eltávolításra kerülnek, valamint a szöveges üzenetek egy bizonyos idő után törlődnek. Ez biztosítja, hogy a megjelenített információk mindig aktuálisak legyenek.

Hiba vagy kapcsolat megszakadása esetén a szerver eltávolítja a játékost az aktív listából, így elkerülhető a hibás vagy “bent ragadt” állapotok megjelenése.

```
private async Task<List<WebSocketOutputDTO>> SendOutput(WebSocketInputDTO input)
{
    var output = await CreateOutputFromServerLogic(input);

    _currentUsername = output.UserName;

    var existing = InGameData.FirstOrDefault(x => x.UserName == output.UserName);

    if (existing != null)
    {
        existing.Role = _playerState!.Role;
        existing.CharacterName = _playerState.CharacterName;
        existing.moveX = (int)_playerState.X;
        existing.moveY = (int)_playerState.Y;
        existing.posX = _playerState.posX;
        existing.posY = _playerState.posY;
        existing.Text = _playerState.Text;
        existing.LastTextDate = _playerState.LastTextDate;
        existing.LastSend = _playerState.LastSend;
    }
    else
    {
        InGameData.Add(output);
    }

    return InGameData;
}
```

47. ábra - Játékos állapot frissítése és tárolása a WebSocket rendszerben

3.6 Frontend játékok

Bunny Run

A Bunny Run egy 2D endless runner típusú játék, amely JavaScript nyelven készült, és a Kaplay játékmotorra épül. A játék egy egyszerű mechanikájú, mégis technikailag jól strukturált szoftver.

Alkalmazás architektúra

A játék moduláris architektúrára épül, ahol minden fő funkció külön fájlban és komponensben kerül megvalósításra. A rendszer központi eleme a Kaplay engine példány, amely a renderelésért, az eseménykezelésért és az időalapú frissítésekért felel.

A belépési pont az index.js fájl, amely inicializálja a környezetet, beállítja a kurzort, ellenőrzi a képernyőméretet, majd elindítja a loadingScene jelenetet. A kaplayCtx.js fájl konfigurálja a játékmotort, beleértve a felbontást, skálázást, debug módot és a renderelési beállításokat.



```
import kaplay from "https://unpkg.com/kaplay@3001.0.19/dist/kaplay.mjs";

var k = kaplay({
  width: 1536,
  height: 1024,
  letterbox: true,
  scale: 3,
  filter: "nearest",
  background: "#000000",
  debug: false,
});

export default k;
```

48. ábra - Játékmotor inicializálása és beállítása

Jelenetkezelés

A játék működése scene alapú struktúrára épül, ahol minden logikai egység külön jelenetként van definiálva. Ez lehetővé teszi az egyes részek izolált kezelését és egyszerű váltást közöttük.

A loadingScene egy bevezető animációt jelenít meg, amely vizuális átmenetet biztosít a játék indulásakor. A menuScene a felhasználói interakció elsődleges pontja, ahol a játék indítható. A gameScene tartalmazza a teljes játékmeneti logikát, beleértve a fizikai rendszert, az objektumkezelést és az eseményeket. Az endScene felel a játék lezárásáért, a pontszám megjelenítéséért és a további navigációért.

```
import k from "../kaplayCtx.js";
import menuScene from "../menuScene.js";
import { kaplayDino } from "../sprites.js";

// Egyszerű loading / intro scene (logo animációval)
export default scene("loadingScene", () => {
  // Skálázási érték az animációhoz
  let scaling = 0;

  // Alap kurzor visszaállítása
  k.setCursor("../assets/cursor.png");

  // Kaplay logó kirajzolása középre
  const kaplayLabel = k.add([
    sprite("kaplay_dino-o"),
    anchor("center"),
    pos(k.center()),
    scale(scaling),
  ]);

  // Frame-alapú animáció
  k.onUpdate(() => {
    // Fokozatos nagyítás
    if (scaling < 1.5) {
      scaling += 0.04;
    } else {
      // Kis késleltetés után továbblépés a menübe
      k.wait(1, () => {
        k.go("menuScene");
      });
    }

    // Aktuális skála alkalmazása a sprite-ra
    kaplayLabel.scale = vec2(scaling, scaling);
  });
});
```

49. ábra - Loading Scene implementációja

Játékos karakter implementációja

A játékos karaktert a Bunny osztály reprezentálja, amely egy Kaplay game object köré épül. Az objektum tartalmazza a sprite megjelenítést, az ütközési területet, valamint a fizikai komponenst, amely lehetővé teszi a gravitáció és az ugrás kezelését.

A karakter működése eseményvezérelt. A játékos input (kattintás) hatására a rendszer ellenőrzi, hogy a karakter a talajon van-e, majd ugrást hajt végre. Az animációk állapotgép-szerűen működnek, ahol az ugrás, esés és futás animációk időzítve váltják egymást.

```
import k from "../kaplayCtx.js";

// A játékos karakter (nyúl) kezelése
export class Bunny {
  constructor() {
    // A nyúl game object létrehozása és eltárolása
    this.object = k.add([
      sprite("bunny"),

      // Alap pozíció: bal oldalról indul, talaj közelében
      pos(k.width() / 5, k.height() - 150),

      // Ütközési terület – kisebb, mint a sprite,
      // hogy ne legyen "túl érzékeny" a collision
      area({
        shape: new k.Rect(k.vec2(0, 3), 30, 30),
      }),

      // Fizika (gravitáció, ugrás, esés stb.)
      body(),

      // Sprite felnagyítása
      scale(3),

      // Rétegmélység – háttér elé kerül
      z(4),

      // Tag, hogy könnyű legyen hivatkozni rá máshol
      "player",
    ]);
  }

  // Animáció lejátszása kívülről (pl. idle, run, jump)
  PlayAnim(anim) {
    this.object.play(anim);
  }
}
```

50. ábra - Játékos karakterének osztálya

Háttérrendszer

A háttér megvalósítása egy folyamatosan scrollozó rendszerrel történik, amely két sprite váltogatására épül. A Background osztály két külön háttérelemet kezel, amelyek egymást követve mozognak balra. Amikor az egyik elem elhagyja a képernyőt, azonnal a másik mögé kerül visszahelyezésre.

Ez a megközelítés biztosítja a végtelen pálya érzetét anélkül, hogy folyamatosan új objektumokat kellene létrehozni, így teljesítmény szempontból is hatékony.

```
import k from "../kaplayCtx.js";

// Két sprite-ból álló, végtelenül scrollozó háttér
export class Background {
  constructor() {
    // Alap scroll sebesség (px / mp)
    this.speed = 100;

    // Első háttér – ez látszik induláskor
    this.backgroundElement1 = k.add({
      sprite("background"),
      pos(0, 0),
      z(0),
      scale(0.8),
    });

    // Második háttér – közvetlenül az első mögé rakva
    // Így tudjuk folyamatosan körbeforgatni őket
    this.backgroundElement2 = k.add({
      sprite("background"),
      pos(1840, 0),
      z(0),
      scale(0.8),
    });

    // Az X pozíciókat külön tároljuk,
    // hogy ne közvetlenül a sprite-ot piszkáljuk logikailag
    this.background1X = 0;
    this.background2X = 1840;
  }

  // Külső vezérléshez (pl. gyorsítás, lassítás játék közben)
  SetBackgroundSpeed(speed) {
    this.speed = speed;
  }

  // Ezt frame-enként kell hívni (pl. update loopból)
  DisplayBackground() {
    // Delta time miatt framerate-független lesz a mozgás
    const dx = this.speed * k.dt();

    // Balra toljuk mindkét háttérrel
    this.background1X -= dx;
    this.background2X -= dx;

    // Ha az egyik teljesen kiment a képernyőről,
    // átdobjuk a másik mögé
    if (this.background1X <= -1843) {
      this.background1X = 1840;
    }

    if (this.background2X <= -1843) {
      this.background2X = 1840;
    }

    // Végül frissítjük a sprite-ok pozícióját
    this.backgroundElement1.pos.x = this.background1X;
    this.backgroundElement2.pos.x = this.background2X;
  }
}
```

51. ábra - Háttér osztály implementációja

Ellenségek kezelése

Az ellenségek kezelése külön osztályokban történik, amelyek felelősek az objektumok létrehozásáért, mozgatásáért és törléséért. A Fox osztály a földi ellenségeket reprezentálja, míg a Vultures osztály a légi akadályokat kezeli.

Az ellenségek generálása időzített eseményekkel történik a gameScene-en belül. Minden ellenség a képernyőn kívül jelenik meg, majd folyamatos balra mozgással kerül a játékos látóterébe. Az objektumok életciklusa szigorúan kontrollált, így a képernyőről kikerült elemek törlésre kerülnek, elkerülve a memória túlterhelését.

```
import k from "../kaplayCtx.js";
import { foxSheet } from "../sprites.js";

// Ellenség (róka) kezelés – spawn, mozgás, takarítás
export class Fox {
  constructor() {
    // Aktív rókák listája
    this.foxes = [];

    // Alap mozgási sebesség (px / mp)
    this.speed = 400;
  }

  // Új róka létrehozása a képernyő jobb szélén kívül
  GenerateFox() {
    let fox = k.add({
      sprite("fox"),

      // Jobbról fut be, talaj közelében
      pos(k.width() + 100, k.height() - 200),

      // Fizika komponens (gravity + collision)
      body(),

      // Szűkebb hitbox a pontosabb ütközéshez
      area({
        shape: new k.Rect(k.vec2(5, 6), 40, 20),
      }),

      // Sprite felnagyítása
      scale(4),

      // A játékos és a háttér fölé kerül
      z(5),

      // Tag: könnyű ütközés- és csoportkezeléshez
      "enemy",
    });

    // Alap animáció indítása spawn után
    fox.play("run");

    // Elmentjük, hogy később tudjuk mozgatni / törölni
    this.foxes.push(fox);
  }

  // Összes aktív róka balra mozgatása
  // (játék loopból hívva)
  MoveFoxes() {
    this.foxes.forEach((fox) => {
      fox.move(-this.speed, 0);
    });
  }

  // Képernyőről kifutott rókák eltávolítása
  DeleteFoxes() {
    for (let i = 0; i < this.foxes.length; i++) {
      const fox = this.foxes[i];

      // Ha már nem látható, nem tartjuk tovább memóriában
      if (fox.pos.x < -100) {
        fox.destroy();
        this.foxes.splice(i, 1);
        i--; // splice miatt visszalépünk egyet
      }
    }
  }
}
```

52. ábra - Fox osztály implementációja

Fizika és ütközés

A játék fizikai rendszere a Kaplay beépített body komponensére épül. A gravitáció globálisan kerül beállításra a gameScene-ben, amely minden fizikai objektumra hatással van.

Az ütközések kezelése tag alapú rendszerrel történik. A játékos "player", míg az ellenségek "enemy" taggel rendelkeznek. A rendszer figyeli ezek ütközését, és esemény hatására azonnal megszakítja a játékmenetet, majd átvált az endScene-re.

Pontszám kezelés

A pontszám kezelését a ScoreBoard osztály végzi, amely nem hagyományos szöveg megjelenítést használ, hanem sprite alapú számkijelzést. Minden egyes számjegy külön sprite-ként kerül kirajzolásra, ami vizuálisan jobban illeszkedik a játék stílusához.

A pontszám időalapon növekszik, amely egy egyszerű, de hatékony módja a játék nehézségének és jutalmazási rendszerének.

A globális scoreState objektum biztosítja, hogy a pontszám elérhető legyen más jelenetek számára is, például az endScene-ben történő megjelenítéshez.

```
import { spriteSheet } from "../sprites.js";
import k from "../kaplayCtx.js";
import scoreState from "../scoreNum.js";

// Pontszám megjelenítése sprite-okból (nem sima text)
export class ScoreBoard {
  constructor() {
    this.k = k;

    // Aktuális pontszám
    this.score = 0;

    // Kirajzott számjegyek referenciái
    // (minden frissítésnél újraépítjük)
    this.scoreSprites = [];

    // Egy számjegy alap szélessége
    this.DIGIT_WIDTH = 60;
  }

  // Pontszám növelése (pl. akadály átvágásánál)
  increaseScore() {
    this.score += 1;
  }

  // Előző frame-ben kirajzott számjegyek törlése
  clearScoreSprites() {
    this.scoreSprites.forEach((s) => s.destroy());
    this.scoreSprites = [];
  }

  // Aktuális pontszám kirajzolása a képernyőre
  displayScore() {
    // Előbb letakarítjuk a régi számjegyeket
    this.clearScoreSprites();

    // A pontszámot stringként kezeljük,
    // így könnyű számjegenként kirajzolni
    const scoreString = this.score.toString();

    // Bal felső saroktól indulunk (HUD elem)
    const startX = 20;

    for (let i = 0; i < scoreString.length; i++) {
      const number = scoreString[i];

      // Egy számjegy kirajzolása
      const digitSprite = k.add([
        sprite( `digit${number}` ),
        pos(startX + i * (this.DIGIT_WIDTH + 1), 20),
        fixed(), // kamera mozgástól független (HUD)
        z(10), // mindig minden fölött
        scale(4),
      ]);

      this.scoreSprites.push(digitSprite);
    }
  }

  // Pontszám átadása globális state-nek
  // (pl. game over képernyőnél)
  setStateScore() {
    scoreState.score = this.score;
  }
}
```

53. ábra - ScoreBoard osztály implementációja

Grafika kezelés

A grafikai elemek betöltése központilag a sprites.js fájlban történik. Ez tartalmazza a sprite atlasokat, karakter animációkat és UI elemeket.

Az atlas alapú megközelítés csökkenti a betöltési időt és optimalizálja a renderelést, mivel több grafikai elem egyetlen textúrából kerül feldolgozásra

```
// UI + statikus elemek sprite atlasza
export var spriteSheet = k.loadSpriteAtlas(
  "./assets/spriteSheet.png",
  {
    // Talaj sprite
    "ground": {
      x: 0,
      y: 858,
      width: 1536,
      height: 166,
    },

    // UI gombok
    "restartButton": { x: 0, y: 16, width: 37, height: 12 },
    "menuButton": { x: 0, y: 29, width: 37, height: 13 },
    "playButton": { x: 40, y: 16, width: 52, height: 29 },
    "leaderBoardButton": { x: 93, y: 16, width: 52, height: 29 },

    // Számjegyek (score kijelzéshez)
    "digit1": { x: 0, y: 0, width: 13, height: 14 },
    "digit2": { x: 14, y: 0, width: 13, height: 14 },
    "digit3": { x: 28, y: 0, width: 13, height: 14 },
    "digit4": { x: 42, y: 0, width: 13, height: 14 },
    "digit5": { x: 56, y: 0, width: 13, height: 14 },
    "digit6": { x: 70, y: 0, width: 13, height: 14 },
    "digit7": { x: 84, y: 0, width: 13, height: 14 },
    "digit8": { x: 98, y: 0, width: 13, height: 14 },
    "digit9": { x: 112, y: 0, width: 13, height: 14 },
    "digit0": { x: 126, y: 0, width: 13, height: 14 },

    // Feliratok / panelek
    "getReadyLabel": {
      x: 0,
      y: 46,
      width: 92,
      height: 26,
    },
    "gameOverLabel": {
      x: 0,
      y: 72,
      width: 96,
      height: 21,
    },
    "endScreenPanel": {
      x: 0,
      y: 94,
      width: 113,
      height: 57,
    },
    "newRecordLabel": {
      x: 0,
      y: 152,
      width: 49,
      height: 8,
    },
    "bunnyRunLabel": {
      x: 0,
      y: 161,
      width: 88,
      height: 24,
    },
  }
);

// Nyúl sprite + animációk
export var bunnySpriteSheet = k.loadSprite(
  "bunny",
  "./assets/bunnySheet.png",
  {
    sliceX: 8,
    sliceY: 2,
    anims: {
      run: { from: 8, to: 10, loop: true, speed: 7.5 },
      jump: { from: 1, to: 1, loop: true },
      fall: { from: 3, to: 3, loop: true },
    },
  }
);
```

54. ábra – UI- és Statikus elemek és Bunny sprite importálása

Backend kommunikáció

A játék képes kommunikálni egy backend API-val, amely a játékos rekordjának tárolására szolgál. A kommunikáció REST alapú, és tokenes hitelesítést használ.

A GetRecord függvény lekéri az aktuális rekordot, míg az UploadRecord felel az új rekord feltöltéséért. A CheckRecord logikai réteggént működik, amely eldönti, hogy szükséges-e a frissítés.

A rendszer automatikusan kezeli a token lejáratát, és szükség esetén frissíti azt, majd újrapróbálja a kérést.

```
import getAuthData from "../../scripts/getAuthData.js";
import refreshToken from "../../scripts/tokenRefresher.js";

export default async function UploadRecord(currentRecord) {
  const endPoint = "https://localhost:7234/api/BunnyRunStatus";
  let isTried = false;

  // Aktuális auth adatok
  let authData = await getAuthData();

  while (true) {
    const response = await fetch(endPoint, {
      method: "PUT",
      headers: {
        "Content-Type": "application/json",
        Authorization: `Bearer ${authData.token}`,
      },
      body: JSON.stringify(currentRecord),
    });

    // Sikeres feltöltés
    if (response.ok) {
      return true;
    }

    // Token lejárt => egyszeri frissítés és retry
    if (!isTried && response.status === 401) {
      await refreshToken(authData.refreshToken);
      authData = await getAuthData();
      isTried = true;
      continue;
    }

    // Egyéb hiba
    throw new Error("HTTP error: " + response.status);
  }
}
```

55. ábra - Felhasználónak rekordjának frissítésének kérése

Játékmenet folyamata

A játék folyamata jól definiált ciklusra épül. A játék elindítása után a gameScene inicializálja az összes szükséges objektumot, beleértve a játékost, hátteret és ellenségkezelő rendszereket. A fő update loop folyamatosan frissíti az objektumok állapotát, mozgatja az ellenségeket és kirajzolja a pontszámot.

A játék addig fut, amíg egy ütközés nem történik. Ekkor a rendszer leállítja az aktív logikát, eltárolja a pontszámot, majd átvált az endScene-re.

Flappy Bird

A Flappy Bird játék implementációja moduláris felépítésben készült, ahol az egyes funkcionális egységek külön JavaScript fájlokban kerülnek megvalósításra. A rendszer alapját a Kaplay játékmotor adja, amely biztosítja a renderelést, az eseménykezelést és a fizikai szimulációt. Az alkalmazás belépési pontja az index.js fájl, amely inicializálja a motort, ellenőrzi a képernyő méretét, majd elindítja a betöltési jelenetet.

A motor konfigurációja a kaplayCtx.js fájlban történik, ahol meghatározásra kerül a játék felbontása, skálázása és megjelenítési paraméterei. Ez a központi objektum kerül felhasználásra az összes további modulban, így biztosítva az egységes működést.

A játék több jelenetre van bontva, amelyek külön fájlokban találhatók. Ezek közé tartozik a betöltési képernyő, a főmenü, a játékmenet és a játék végi képernyő. A jelenetek közötti váltás a Kaplay motor beépített scene kezelő rendszerén keresztül történik.

A screenshot of a code editor with a dark background and light-colored text. The code is in JavaScript and initializes the Kaplay engine. It includes an import statement for Kaplay from a specific URL, followed by a comment and a const declaration for 'k' with various configuration options like width, height, letterbox, scale, filter, and background. The code ends with an export default statement.

```
import kaplay from "https://unpkg.com/kaplay@3001.0.19/dist/kaplay.mjs";

// Kaplay engine init
const k = kaplay({
  width: 143,
  height: 256,
  letterbox: true,
  scale: 8,
  filter: "nearest",
  background: "#000000",
});

export default k;
```

56. ábra - Játékmotor inicializálása és beállítása

Játékmenet logika

A játékmenet központi eleme a gameScene, amely inicializálja a játék objektumait és kezeli a teljes futási logikát. A jelenet indulásakor létrejönnek a fő objektumok: háttér, talaj, madár, csövek és a pontszámláló.

A játék indulása előtt egy visszaszámlálási mechanizmus fut le, amely során a játékos vizuális visszajelzést kap. A visszaszámlálás egy időzített ciklus segítségével valósul meg, amely másodpercenként frissíti az állapotot és jeleníti meg a megfelelő számot.

A tényleges játék a felhasználói kattintás hatására indul el, amely aktiválja a gravitációt és a madár mozgását. Ezzel párhuzamosan elindul a csövek generálása és mozgatása, valamint a pontszám folyamatos frissítése.

```
function Game() {
  k.setGravity(300);

  gameClick = k.onClick(() => {
    bird.BirdJump(pipes, background, ground, scoreBoard);
  });

  pipes.DisplayPipes();

  k.onUpdate(() => {
    pipes.MovePipes();
    scoreBoard.DisplayScore();
  });

  k.onCollide("bird", "scoreZone", (zone) => {
    if (!zone.passed) scoreBoard.IncreaseScore();
  });

  k.onCollide("bird", "pipe", () => {
    bird.DisableJump();
    pipes.SetPipeSpeed(0);
    background.SetBackgroundSpeed(0);
    ground.SetGroundSpeed(0);
    scoreBoard.SetStateScore();
    k.wait(2, () => k.go("endScreen"));
  });
}
```

57. ábra - Játékot futtató metódus

Madár működése

A madár objektum a játék egyik központi eleme, amely fizikai testtel és ütközés figyeléssel rendelkezik. Az objektum inicializálásakor beállításra kerül a pozíciója, valamint egy egyedi ütközési test, amely pontosabb ütközésvizsgálatot tesz lehetővé.

A madár mozgása ugrásokon alapul, amelyeket a játékos input vált ki. Az ugrás csak addig engedélyezett, amíg nem történik ütközés. Az állapotkezelés egy logikai változó segítségével történik, amely megakadályozza a további input feldolgozását ütközés után.

A rendszer folyamatosan ellenőrzi a madár pozícióját, és ha az kilép a képernyőből vagy érintkezik a talajjal, akkor a játék véget ér. Ilyenkor minden mozgó elem leáll, a pontszám mentésre kerül, majd egy késleltetés után betöltődik az end screen jelenet.

```
BirdJump(pipes, background, ground, scoreBoard) {  
  // Ugrás csak aktív állapotban  
  if (this.isNotCollided) {  
    this.object.jump(100);  
  }  
  
  this.object.onUpdate(() => {  
    // Halál feltételek: kirepül vagy földre esik  
    if (this.object.pos.y < -20 || this.object.isGrounded()) {  
      this.DisableJump();  
  
      // Játék megállítása  
      pipes.SetPipeSpeed(0);  
      background.SetBackgroundSpeed(0);  
      ground.SetGroundSpeed(0);  
  
      // Score rögzítése és end screen  
      scoreBoard.SetStateScore();  
      k.wait(2, () => this.k.go("endScreen"));  
    }  
  });  
}
```

58. ábra - Madár ugrásáért felelős metódus

Csőrendszer

A csövek dinamikusan generált akadályok, amelyek a játékmenet nehézségét adják. A PipeGroup osztály felelős a csövek létrehozásáért, mozgatásáért és törléséért.

A csövek generálása véletlenszerű pozícióval és réssel történik, amely biztosítja a változatos játékmenetet. Minden cső tartalmaz egy láthatatlan ütközési zónát, amely a pontszám növelésére szolgál, amikor a játékos sikeresen áthalad rajta.

A csövek folyamatosan balra mozognak frame-alapú számítással, amely a delta idő figyelembevételével történik. Azok az elemek, amelyek elhagyják a képernyőt, törlésre kerülnek a memóriából a teljesítmény optimalizálása érdekében.

```
GeneratePipe() {
  const y = rand(this.minY, this.maxY);
  const gap = rand(this.minGap, this.maxGap);

  // Felső cső (parent)
  const pipe = this.k.add([
    sprite("pipe_top"),
    anchor("botleft"),
    pos(this.k.width(), y),
    body({ isStatic: true }),
    area(),
    z(2),
    "pipe",
  ]);

  // Alsó cső
  pipe.add([
    sprite("pipe_bottom"),
    pos(0, gap),
    body({ isStatic: true }),
    area(),
    z(2),
    "pipe",
  ]);

  // Láthatatlan score trigger
  pipe.add([
    rect(10, gap),
    pos(30, 0),
    area(),
    z(2),
    opacity(0),
    "scoreZone",
    { passed: false },
  ]);

  this.pipes.push(pipe);
}
```

59. ábra - Csövek generálásáért felelős metódus

Háttér és talaj animáció

A háttér és a talaj folyamatos mozgását két-két sprite segítségével valósítja meg a rendszer. Ezek egymás után helyezkednek el, és amikor az egyik kilép a képernyőről, a másik mögé kerül vissza.

Ez a megoldás végtelen scroll hatást biztosít, amely alapvető eleme a játék vizuális működésének. A mozgás sebessége dinamikusan változtatható, például játék végekor nullára állítható.

```
DisplayBackground() {
  // Frame-alapú elmozdulás
  const dx = this.backgroundElementSpeed * this.k.dt();

  this.bg1X -= dx;
  this.bg2X -= dx;

  // Ha kiment balra, visszaugrik jobbra
  if (this.bg1X <= -this.k.width()) {
    this.bg1X = this.k.width() - 1;
  }

  if (this.bg2X <= -this.k.width()) {
    this.bg2X = this.k.width() - 1;
  }

  // Sprite pozíciók frissítése
  this.backgroundElement1.pos.x = this.bg1X;
  this.backgroundElement2.pos.x = this.bg2X;
}
```

60. ábra - Háttérelemek mozgatásáért felelős metódus

Pontszám kezelés

A pontszám kezelése egy külön ScoreBoard osztályban történik, amely felelős a pontok növeléséért, megjelenítéséért és tárolásáért. A pontszám növelése akkor történik meg, amikor a madár áthalad egy csőhöz tartozó score zónán.

A megjelenítés sprite alapú, ahol minden számjegy külön képként kerül kirajzolásra. A rendszer minden frissítéskor újrarajzolja a számjegyeket, biztosítva az aktuális érték helyes megjelenítését.

A játék végén a pontszám egy globális állapotba kerül mentésre, amely lehetővé teszi az adatok továbbítását más jelenetek felé.

```
DisplayScore() {  
  // Score újrarajzolása  
  this.ClearScoreSprites();  
  
  const scoreString = this.score.toString();  
  const totalWidth =  
    scoreString.length * this.DIGIT_WIDTH + (scoreString.length - 1);  
  
  const startX = width() / 2 - totalWidth / 2;  
  
  for (let i = 0; i < scoreString.length; i++) {  
    const number = scoreString[i];  
  
    const digitSprite = add([  
      sprite(`digit${number}`),  
      pos(startX + i * (this.DIGIT_WIDTH + 1), this.k.height() - 26),  
      fixed(),  
      z(10),  
    ]);  
  
    this.scoreSprites.push(digitSprite);  
  }  
}
```

61. ábra - A pontszám megjelenítéséért felelős metódus

Rekord kezelés és API kommunikáció

A rendszer támogatja a játékos rekordjának szerveroldali tárolását és frissítését. A rekord lekérése és feltöltése külön modulokban történik, amelyek HTTP kéréseken keresztül kommunikálnak egy backend API-val.

A GetRecord függvény felelős az aktuális rekord lekéréséért, amely hitelesített kérés formájában történik. Amennyiben a token lejárt, a rendszer automatikusan frissíti azt, majd újrapróbálja a lekérést.

A CheckRecord függvény összehasonlítja az aktuális játék eredményét a szerveren tárolt rekorddal, és szükség esetén meghívja az UploadRecord függvényt, amely frissíti az adatokat.

Ez a mechanizmus biztosítja a játékos előrehaladásának tartós tárolását és a biztonságos kommunikációt.

```
export default async function CheckRecord() {
  // Aktuális rekord lekérése
  const recordResponse = await GetRecord();
  const status = scoreState.score;

  // Ha nem sikerült lekérni a rekordot
  if (!recordResponse.success) {
    return false;
  }

  const record = recordResponse.data.maxPassedPipes;

  // Új rekord esetén frissítés
  if (record < status) {
    await UploadRecord({ maxPassedPipes: status });
    return true;
  }

  // Nincs új rekord
  return false;
}
```

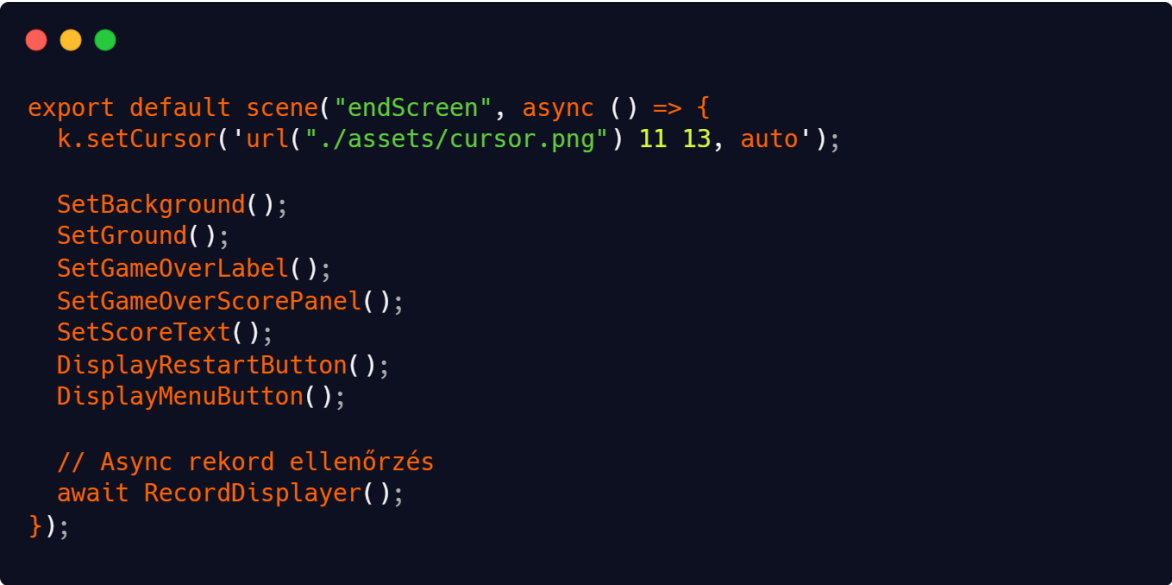
62. ábra - Pontszám vizsgálatáért felelős metódus

Jelenetek kezelése

A játék több különálló jelenetre van bontva, amelyek egymástól függetlenül kezelik a megjelenítést és a logikát. A fő jelenetek közé tartozik a menü, a játék, a betöltési képernyő és a játék végi képernyő.

A jelenetek közötti váltás a Kaplay motor beépített funkcióival történik, amely lehetővé teszi az állapotok elkülönítését és a kód tiszta szervezését.

A játék végi képernyő felelős a pontszám megjelenítéséért, valamint a rekord ellenőrzéséért. Amennyiben új rekord születik, a rendszer ezt vizuálisan is jelzi a felhasználó számára.

A screenshot of a code editor with a dark background and light-colored text. The code is written in JavaScript and defines a default scene for the end screen. It includes comments in Hungarian and sets various UI elements like cursor, background, ground, game over labels, score panel, score text, restart button, and menu button. It also includes an async function call to check for a new record.

```
export default scene("endScreen", async () => {  
  k.setCursor('url("./assets/cursor.png") 11 13, auto');  
  
  SetBackground();  
  SetGround();  
  SetGameOverLabel();  
  SetGameOverScorePanel();  
  SetScoreText();  
  DisplayRestartButton();  
  DisplayMenuButton();  
  
  // Async rekord ellenőrzés  
  await RecordDisplayer();  
});
```

63. ábra - Játék vége scen

Memory Game

A Memory Game egy böngészőben futó frontend alkalmazás, amely egy klasszikus párosító játék modern webes megvalósítása. A játék célja, hogy a felhasználó megtalálja az egymáshoz tartozó kártyapárokat, miközben fejleszti memóriáját és koncentrációs képességét.

A játék különlegessége, hogy nemcsak szórakoztató célt szolgál, hanem oktatási funkcióval is rendelkezik. A játék során a felhasználók gyümölcsök és zöldségek neveit tanulhatják meg három különböző nyelven: angolul, magyarul és spanyolul. Ezáltal a játék egyszerre fejleszti a memóriát és az idegen nyelvi készségeket.

A játék teljes mértékben kliens oldalon működik, azonban bizonyos funkciók, például a felhasználói adatok kezelése és statisztikák mentése backend kommunikációval valósul meg.

A játék működése

A játék indításakor a rendszer létrehozza a játéktáblát a kiválasztott nehézségi szint alapján. Ehhez először véletlenszerűen kiválaszt egy adott számú gyümölcs- és zöldségtípust, majd ezekből párokat hoz létre. A párok egy tömbbe kerülnek, amelyet a program egy keverésalgoritmussal véletlenszerű sorrendbe rendez. Ez biztosítja, hogy minden játék eltérő legyen.

```
function MakeGame() {  
  // Stop the existing game updater loop if it exists  
  if (timerId !== null) {  
    clearTimeout(timerId);  
  }  
  
  // Reset animations to prevent conflicts  
  gameContainer.style.animation = "none";  
  infoPanel.style.animation = "none";  
  
  // Generate the game structure  
  GetCardTypes(); // Select random card types  
  FillGameArray(); // Create card pairs  
  ShuffleArray(); // Shuffle cards  
  DisplayBoard(); // Render cards on screen  
  Reset(); // Reset runtime values  
  UpdateInfo(); // Update UI  
  
  // Start the game loop updater  
  timerId = setTimeout(UpdateGame, 1000);  
}
```

64. ábra - Játék inicializálása

A játékos kattintással választ ki kártyákat. A rendszer egyszerre legfeljebb két kártyát enged kiválasztani. A második kártya kiválasztása után a program automatikusan összehasonlítja a két elemet. Ha egyeznek, a kártyák rögzítésre kerülnek, ellenkező esetben egy rövid animáció után visszafordulnak.

```
// Compares the two selected cards
function CheckCards() {
  if (selectedCards[0][0].eng == selectedCards[1][0].eng) {
    GoodPairing();
  } else {
    BadPairing();
  }
}
```

65. ábra - Kártyák összehasonlítása

A játék addig tart, amíg az összes pár megtalálásra nem kerül. A befejezés után a rendszer megjeleníti az eredményeket, beleértve az eltelt időt és a lépések számát. Amennyiben a játékos új rekordot ér el, azt a rendszer külön jelzi.

Felhasználói felület

A játéktábla dinamikusan jön létre JavaScript segítségével. Minden kártya egy külön DOM elemként kerül létrehozásra, amelyhez eseménykezelő tartozik. Ez biztosítja, hogy a felhasználó interakcióba léphessen a játékkal.

```
newCard.addEventListener("click", () => {
  function ChoosingCard() {
    // Only allow valid selections
    if (
      selectedCards.length < 2 &&
      gameCards.includes(element) &&
      !selectedCards.some([el, idx] => el === element && idx === index)
    ) {
      selectedCards.push([element, index]);
      newCard.style.animation = "rotation 0.5s forwards";
      newCard.style.background =
        "linear-gradient(to right, darkgray, lightslategray)";

      setTimeout(() => {
        newCard.style.backgroundImage = `url(./assets/fruit-images/${element.img})`;
        newName.style.display = "block";
      }, 100);

      // When two cards are selected, check for match
      if (selectedCards.length == 2) setTimeout(CheckCards(), 3000);
    }
  }
  ChoosingCard();
});
```

66. ábra - Kártya eseménykezelés

A játék különböző animációkat használ a vizuális visszajelzés érdekében. A kártyák felfordulnak, hibás választás esetén rezegnek, míg helyes párosítás esetén színnel kiemelésre kerülnek. Ezek az effektek javítják a felhasználói élményt.

Nyelvkezelés

A játék támogatja a többnyelvű működést. A kiválasztott nyelvet a rendszer eltárolja a böngészőben, és ennek megfelelően frissíti a felhasználói felület szövegeit. Ez lehetővé teszi, hogy a felhasználó bármikor nyelvet váltson.

```
// Switches language and updates static menu texts
function SwitchLang(newLang) {
  lang = newLang;
  localStorage.setItem("lang", newLang);
  console.log(lang);
  document.getElementById("h5").textContent = Texts[0][lang];
  document.getElementById("play-button").textContent = Texts[1][lang];
}
```

67. ábra - Nyelvkezelés egy szelete

Backend kommunikáció

A játék backend szerverrel is kommunikál, amely a felhasználói adatokat és statisztikákat kezeli. A kommunikáció HTTP kérések segítségével történik, és a rendszer token alapú hitelesítést alkalmaz.

A backend lehetővé teszi a felhasználói adatok lekérését és az eredmények mentését.

```
var response = fetch("https://localhost:7234/api/MemoryGameStatus", {
  method: "PUT",
  headers: {
    "Content-Type": "application/json",
    Authorization: `Bearer ${token}`,
  },
  body: JSON.stringify(newStatus),
});
```

68. ábra - Rekord frissítés végpont meghívása

Wordle Game

A Wordle Game egy böngészőben futó frontend alkalmazás, amely egy klasszikus szókitaláló játék modern webes megvalósítása. A játék célja, hogy a felhasználó egy véletlenszerűen kiválasztott, öt betűből álló angol szót kitaláljon legfeljebb hat próbálkozásból. A játék elsősorban logikai gondolkodást és szókincset fejleszt, miközben egyszerű és intuitív felhasználói élményt biztosít.

A játék teljes mértékben kliens oldalon működik, azonban bizonyos funkciók – például a felhasználói statisztikák kezelése – backend kommunikációval valósulnak meg. A rendszer token alapú hitelesítést alkalmaz, amely biztosítja a felhasználói adatok védelmét.

A rendszer működése

A játék indításakor a rendszer egy előre definiált szólistából választ ki egy véletlenszerű elemet, amely az aktuális játék célja lesz. A szólista körülbelül 1300 darab, kizárólag öt betűs angol szót tartalmaz. A véletlenszerű kiválasztás biztosítja, hogy minden játék eltérő legyen, ezáltal növelve az újrajátszhatóságot.

A játékos betűnként írja be a tippjét, amely maximum öt karakter hosszú lehet. A bevétel kétféle módon történhet: fizikai billentyűzet használatával, illetve egy képernyőn megjelenő virtuális billentyűzet segítségével. A rendszer mindkét esetben ugyanazt a logikát alkalmazza, így biztosítva a konzisztens működést.

Amikor a játékos egy teljes szót megad, a rendszer ellenőrzi annak érvényességét. A szó csak akkor kerül elfogadásra, ha pontosan öt betűből áll, és szerepel a szólistában. Ez megakadályozza az értelmetlen vagy nem létező szavak bevitelét.

Szó kiértékelési mechanizmus

A játék legfontosabb része a beírt szó kiértékelése, amely a Wordle szabályrendszerén alapul. A kiértékelés két lépésben történik annak érdekében, hogy a betűk többszöri előfordulása is helyesen kezelhető legyen.

Az első lépés során a rendszer megvizsgálja, hogy a játékos által beírt betűk közül melyek találhatók meg a megfelelő pozícióban. Az ilyen betűk zöld színt kapnak, és a rendszer csökkenti az adott betű előfordulási számát a cél szóban. Ez biztosítja, hogy ugyanaz a betű ne legyen többször helyesen értékelve.

A második lépés során a rendszer a még nem értékelt betűket vizsgálja. Amennyiben egy betű szerepel a cél szóban, de nem a megfelelő helyen, sárga színt kap. Ha a betű egyáltalán nem szerepel a szóban, szürke színt kap. Ez a kétlépcsős algoritmus garantálja a pontos és korrekt visszajelzést.

```
// Evaluates a guess and applies Wordle rules
async function CheckRow(row, inputword, word) {
  if (isAnimating) return true;
  isAnimating = true;

  const rowElement = GetRowElement(row);
  if (!rowElement) {
    isAnimating = false;
    return true;
  }

  const targetChars = word.split("");
  const guessChars = inputword.split("");
  let ids = Array(5).fill("");

  // Count target letter occurrences
  const letterCount = {};
  for (let i = 0; i < 5; i++) {
    letterCount[targetChars[i]] = (letterCount[targetChars[i]] || 0) + 1;
  }

  // Correct position (green)
  for (let i = 0; i < 5; i++) {
    if (guessChars[i] === targetChars[i]) {
      ids[i] = "green";
      letterCount[guessChars[i]]--;
    }
  }

  // Wrong position (yellow) or not present (grey)
  for (let i = 0; i < 5; i++) {
    if (ids[i]) continue;

    if (letterCount[guessChars[i]] > 0) {
      ids[i] = "yellow";
      letterCount[guessChars[i]]--;
    } else {
      ids[i] = "grey";
    }
  }

  // Update keyboard colors
  for (let i = 0; i < 5; i++) {
    SetKeyColor(inputword[i], ids[i]);
  }

  await AnimateRow(rowElement, guessChars, ids);
  LoadKeys();
  isAnimating = false;

  const isWin = ids.every((id) => id === "green");
  if (isWin || row === 6) {
    EndGame(isWin);
    return true;
  }

  return false;
}
```

69. ábra - Szó kiértékelési algoritmus implementációja

Felhasználói felület

A játék felhasználói felülete dinamikusan generált DOM elemekből áll. A játéktábla hat sorból és soronként öt mezőből épül fel, amelyek a beírt karaktereket jelenítik meg. Minden egyes mező külön DOM elemként létezik, amely lehetővé teszi az animációk és stílusok egyedi kezelését.

A felület része egy virtuális billentyűzet is, amely valós időben frissül a játék során. A billentyűk színezése a korábbi próbálkozások alapján történik, és prioritási rendszer szerint működik, ahol a zöld szín felülírja a sárgát, a sárga pedig a szürkét. Ez segíti a játékost a következő próbálkozások során.

A játék több különböző képernyőt tartalmaz, beleértve a főmenüt, a játéktérrel, valamint a játék végén megjelenő eredményképernyőt. Emellett egy súgó panel is rendelkezésre áll, amely információt nyújt a játékmenetről és a nyelvi beállításokról.

```
// Updates the current row display
function UpdateNextRow(row, inputword) {
  const rowElement = GetRowElement(row);
  if (!rowElement) return;

  const chars = inputword.split("").concat(Array(5).fill("")).slice(0, 5);

  rowElement.innerHTML = chars
    .map((char) => `<div class="card"><p>${char}</p></div>`)
    .join("");
}
```

70. ábra - Játékrács dinamikus renderelése

Animációk és felhasználói élmény

A játék vizuális visszajelzéseit animációk biztosítják. A beírt szó kiértékelése során a kártyák egy flip animáció segítségével fordulnak meg, és egymás után fedik fel a színüket. Ez az animáció aszinkron módon, időzítéssel kerül végrehajtásra, amely gördülékeny és jól követhető élményt biztosít.

Az animációk során a felhasználói bevitel ideiglenesen letiltásra kerül egy állapotváltozó segítségével. Ez megakadályozza a hibás vagy ütköző interakciókat, és biztosítja a játék stabil működését.

```
/* =====  
  ROW ANIMATION  
===== */  
  
// Animates the card flip and color reveal  
async function AnimateRow(rowElement, chars, ids, delay = 500) {  
  for (let i = 0; i < 5; i++) {  
    rowElement.innerHTML = chars  
      .map((char, j) => {  
        if (j < i) {  
          return `<div class="card" id="${ids[j]}"><p>${char}</p></div>`;  
        } else if (j === i) {  
          return `<div class="card flipping"><p>${char}</p></div>`;  
        } else {  
          return `<div class="card"><p>${char}</p></div>`;  
        }  
      })  
      .join("");  
  
    await new Promise((r) => setTimeout(r, delay / 2));  
  
    const flippingCard = rowElement.querySelector(".card.flipping");  
    if (flippingCard) flippingCard.id = ids[i];  
  
    await new Promise((r) => setTimeout(r, delay / 2));  
  
    if (flippingCard) flippingCard.classList.remove("flipping");  
  }  
}
```

71. ábra - Animáció időzítési folyamat

Állapotkezelés

A játék állapotát globális változók kezelik, amelyek nyomon követik az aktuális szót, a játékos bevitelét, az aktuális sort, valamint a játék állapotát. Az állapotkezelés egyszerű, de hatékony megoldást biztosít egy ilyen méretű alkalmazás esetében.

Az animációk kezelésére egy külön változó szolgál, amely megakadályozza az új bevitel feldolgozását, amíg az aktuális animáció le nem fut. Ez a megközelítés biztosítja az események helyes sorrendiségét.

Backend kommunikáció

A játék backend szerverrel is kommunikál a felhasználói statisztikák frissítése érdekében. A kommunikáció HTTP kérések segítségével történik, és JWT alapú hitelesítést használ. A szükséges tokenek a böngésző session tárolójában kerülnek eltárolásra.

Amennyiben a játékos sikeresen kitalálja a szót, a rendszer egy API hívást indít, amely frissíti a felhasználó statisztikáit. Ha a hozzáférési token érvénytelen, a rendszer automatikusan megpróbálja frissíteni azt egy refresh token segítségével, majd újraküldi a kérést.

```
// Updates the player's statistics on the backend
// Uses stored JWT tokens and attempts token refresh on failure
function ChangeStat() {
  const sessionRefreshToken = sessionStorage.getItem("RefreshToken");
  const token = sessionStorage.getItem("Token");

  // Abort if authentication data is missing
  if (sessionRefreshToken !== undefined && token !== undefined) {
    // Send statistics update request
    let response = fetch("https://localhost:7234/api/WordleStatus", {
      method: "PUT",
      headers: {
        "Content-Type": "application/json",
        Authorization: `Bearer ${token}`,
      },
      body: JSON.stringify({
        CompletedWords: 0,
      }),
    });

    // If request fails, try refreshing the token and retry
    if (!response.ok) {
      refreshToken(sessionRefreshToken);

      fetch("https://localhost:7234/api/WordleStatus", {
        method: "PUT",
        headers: {
          "Content-Type": "application/json",
          Authorization: `Bearer ${token}`,
        },
        body: JSON.stringify({
          CompletedWords: 0,
        }),
      });
    }
  }

  return true;
}

return false;
}
```

72. ábra - Felhasználó Státuszának frissítése a Wordle játékban

Nyelvkezelés

A játék többnyelvű felhasználói felületet támogat, amely lehetővé teszi a felhasználók számára, hogy különböző nyelveken használják az alkalmazást. A választható nyelvek közé tartozik az angol, a magyar és a spanyol.

A kiválasztott nyelv a böngésző localStorage-ában kerül tárolásra, így az oldal újratöltése után is megmarad. A felhasználói felület szövegei egy külön konfigurációs fájlban kerülnek definiálásra, amely megkönnyíti a karbantartást és a bővítést.

Fontos megjegyezni, hogy a játékmenet kizárólag angol nyelvű szavakkal működik, függetlenül a felhasználói felület nyelvétől.

Játék befejezése

A játék akkor ér véget, amikor a játékos sikeresen kitalálja a szót, vagy amikor elhasználja az összes rendelkezésre álló próbálkozást. A játék befejezésekor a rendszer megjelenít egy végképernyőt, amely tartalmazza az eredményt, valamint a helyes megoldást.

A végképernyő lehetőséget biztosít új játék indítására, illetve a főmenübe való visszatérésre. Győzelem esetén a rendszer megpróbálja frissíteni a felhasználói statisztikákat a backend szerveren.

RetroRealm

A Retro Realm egy többjátékos, valós idejű kommunikációra épülő platform, amelyben a felhasználók egy közös térben mozoghatnak és szöveges üzeneteket jeleníthetnek meg egymás számára. Az alkalmazás a Kaplay játékmotorra épül, amely biztosítja a renderelést, az inputkezelést és az objektumkezelést.

Az alkalmazás belépési pontja az index.js fájl, amely ellenőrzi a kliens képernyőméretét, majd elindítja a játékmenetet tartalmazó jelenetet. A motor inicializálása a kaplayCtx.js fájlban történik, ahol meghatározásra kerül a felbontás, a maximális FPS és a debug mód.

A rendszer moduláris felépítésű, ahol külön fájlok kezelik a karaktereket, a térképet, az inputot, valamint a hálózati kommunikációt.

A screenshot of a code editor with a dark background and light-colored text. The code is in JavaScript and defines a function to check the screen size. At the top left of the editor window, there are three colored circles (red, yellow, green) representing window control buttons. The code includes imports for 'k' and 'gameScene', a 'checkScreenSize' function that checks if the window width is less than 1024 and redirects to a landing page if so, and then calls 'checkScreenSize' and adds a 'resize' event listener. Finally, it calls 'k.go' with 'gameScene' as an argument.

```
import k from "./scripts/kaplayCtx.js";
import { gameScene } from "./scripts/gameScene.js";

function checkScreenSize() {
  if (window.innerWidth < 1024) {
    window.location.href = "/landingpage";
  }
}

checkScreenSize();

window.addEventListener("resize", checkScreenSize);

k.go("gameScene");
```

73. ábra - Alkalmazás indító scriptje

Térkép generálás és rétegek kezelése

A játék világának felépítése több rétegben történik, amelyeket a CreateMap modul hoz létre. A térkép három fő rétegből áll: alap (föld), felső (fű), valamint objektum réteg.

A rétegek sprite alapú mátrixként vannak definiálva, ahol minden sprite egy adott tile-nak felel meg. A rendszer a Kaplay addLevel függvényével dolgozza fel ezeket a rétegeket, és generálja le a megfelelő sprite-okat.

A különböző tile-ok esetén a rendszer nemcsak vizuális elemeket hoz létre, hanem ütközési objektumokat is, amelyek megakadályozzák a játékos mozgását bizonyos területeken. A véletlenszerűség érdekében egyes tile-ok több variációval rendelkeznek, amelyek közül a rendszer generáláskor választ.

```
export function CreateMap() {
  const dirtmap = k.addLevel(dirtLayer,
  {
    tileWidth: 32,
    tileHeight: 32,
    tiles: {
      "a": () => [
        sprite(getRandomDirt()),
        z(1),
      ],
      "b": () => [
        sprite("waterLayerOuterEmptyTopLeft"),
        z(1),
        "object",
        area(),
        body({ isStatic: true }),
      ],
      "c": () => [
        sprite("waterLayerOuterEmptyTopCenter"),
        z(1),
        "object",
        area(),
        body({ isStatic: true }),
      ],
      "d": () => [
        sprite("waterLayerOuterEmptyTopRight"),
        z(1),
      ],
      "e": () => [
        sprite("waterLayerOuterEmptyCenterLeft"),
        z(1),
        "object",
        area(),
        body({ isStatic: true }),
      ],
      "f": () => [
        sprite("waterLayerOuterEmptyCenterRight"),
        z(1),
        "object",
        area(),
        body({ isStatic: true }),
      ],
    },
  },
  1,
```

74. ábra - Játéktér generáló metódus szelete

Karakter rendszer működése

A karakterek kezelése a Character osztályon keresztül történik, amely felelős a játékosok megjelenítéséért, mozgásáért és állapotáért. Minden karakter rendelkezik egy sprite-tal, pozícióval, valamint egyedi azonosítóval (felhasználónév).

A karakter fölött megjelenítésre kerül a felhasználónév, amely két rétegben van kirajzolva: egy fekete árnyék és egy színes szöveg. A szín a felhasználó szerepkörétől függ, például admin esetén piros, normál felhasználó esetén zöld.

A mozgás irányát és animációját a rendszer dinamikusan kezeli.

```
update(dx, dy) {
  const moving = dx !== 0 || dy !== 0;

  if (moving) {
    this.object.moveBy(dx * this.speed * k.dt(), dy * this.speed * k.dt());
  }

  if (!moving) {
    this.play(
      this.facing === "up"
        ? "idleUp"
        : this.facing === "down"
        ? "idleDown"
        : this.facing === "left"
        ? "idleLeft"
        : "idleRight",
    );
    return;
  }

  if (dx > 0) {
    this.facing = "right";
    this.play("walkRight");
  } else if (dx < 0) {
    this.facing = "left";
    this.play("walkLeft");
  } else if (dy < 0) {
    this.facing = "up";
    this.play("walkUp");
  } else if (dy > 0) {
    this.facing = "down";
    this.play("walkDown");
  }
}
```

75. ábra - Karakterek frissítéséért felelős metódus

Mozgás és input kezelés

A játékos mozgása billentyűzet input alapján történik, ahol a rendszer folyamatosan figyeli a lenyomott gombokat. A mozgás irányát egy vektor reprezentálja, amely az aktuális input alapján frissül.

A mozgás csak akkor engedélyezett, ha a játékos nem használja az input mezőt. Ez egy logikai változó segítségével van kezelve, amely letiltja a mozgást szövegbevitel közben.

A karakter mozgása frame-alapú számítással történik, amely biztosítja az egyenletes mozgást különböző teljesítményű rendszereken is.

```
k.onKeyDown(["d", "right"], () => {
    if (!movementEnabled) return;
    dir.x = 1;
});

k.onKeyDown(["a", "left"], () => {
    if (!movementEnabled) return;
    dir.x = -1;
});

k.onKeyDown(["w", "up"], () => {
    if (!movementEnabled) return;
    dir.y = -1;
});

k.onKeyDown(["s", "down"], () => {
    if (!movementEnabled) return;
    dir.y = 1;
});

k.onKeyRelease(["d", "right", "a", "left"], () => dir.x = 0);
k.onKeyRelease(["w", "up", "s", "down"], () => dir.y = 0);
```

76. ábra - Gomb lenyomásokat érzékelő kódrészlet

Szövegbevitel és chat rendszer

A kommunikáció az InputBox osztály segítségével valósul meg, amely egy dinamikusan megjelenő szövegbeviteli felületet biztosít. A felhasználó egy gomb lenyomásával aktiválhatja az input mezőt, ahol maximum karakterszám mellett írhat üzenetet.

A rendszer kezeli a karakterbevitel eseményeit, beleértve a törlést, a kilépést és az üzenet elküldését. Az elküldött szöveg a karakter fölött jelenik meg egy külön komponens segítségével.

A megjelenítés a TextRect osztályon keresztül történik, amely egy háttérrel ellátott szövegdobozt hoz létre a karakter felett. Az üzenetek időkorlátosak, és egy meghatározott idő után automatikusan eltűnnek.

```
onCharInput((char) => {
  if (!this.visible) return
  if (this.text.length >= this.MAX_CHARS) return

  if (isKeyDown("shift")) {
    this.text += char.toUpperCase()
  } else {
    this.text += char
  }
});

onKeyPress("backspace", () => {
  if (!this.visible) return
  this.text = this.text.slice(0, -1)
});

onKeyPress("escape", () => {
  if (!this.visible) return
  this.hide()
});

onKeyPress("enter", () => {
  if (!this.visible) return
  if (this.text.length > 0) {
    this.onSend(this.text);
  }
  this.hide()
});
```

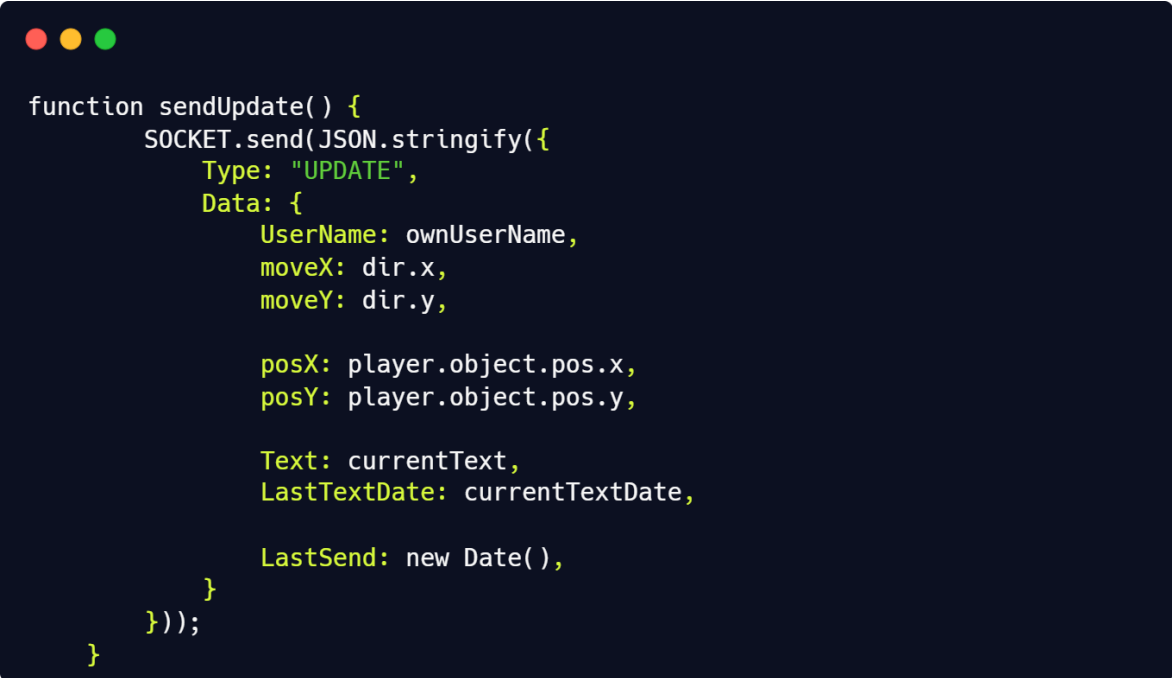
77. ábra - Szövegbeviteli mező lehetséges inputjai

Valós idejű kommunikáció (WebSocket)

A többjátékos működés alapját egy WebSocket kapcsolat biztosítja, amely folyamatos adatcserét tesz lehetővé a kliens és a szerver között. A kapcsolat létrejötte után a kliens rendszeres időközönként elküldi az aktuális állapotát.

Az elküldött adatok tartalmazzák a játékos pozícióját, mozgásirányát, aktuális szövegét és annak időbélyegét. A szerver válasza tartalmazza az összes aktív játékos állapotát, amely alapján a kliens frissíti a megjelenített karaktereket.

A rendszer külön kezeli a saját játékost és a többi játékost, biztosítva, hogy az állapotok ne keveredjenek. Az inaktív vagy kilépett játékosok automatikusan eltávolításra kerülnek a kliensből.



```
function sendUpdate() {
    SOCKET.send(JSON.stringify({
        Type: "UPDATE",
        Data: {
            UserName: ownUserName,
            moveX: dir.x,
            moveY: dir.y,

            posX: player.object.pos.x,
            posY: player.object.pos.y,

            Text: currentText,
            LastTextDate: currentTextDate,

            LastSend: new Date(),
        }
    }));
}
```

78. ábra - Kliens állapotküldése a szerverre

Felhasználónév lekérése és autentikáció

A rendszer a felhasználó azonosítását token alapú autentikációval végzi. A `GetUserName` modul felelős a felhasználónév lekéréséért, amely egy védett API végponton keresztül történik.

Amennyiben a token lejárt, a rendszer automatikusan megpróbálja frissíteni azt, majd újraküldi a kérést. Ez biztosítja a folyamatos működést anélkül, hogy a felhasználónak újra be kellene jelentkeznie.

Hibás vagy sikertelen kérés esetén a rendszer üres értéket ad vissza, így elkerülhető a kliens oldali hibás működés.

```
import refreshToken from "../../scripts/tokenRefresher.js";

export default async function GetUserName() {
  const token = sessionStorage.getItem("Token");
  if (!token) return "";

  async function request(tokenToUse) {
    return fetch("https://localhost:7234/api/GetUserName", {
      method: "GET",
      headers: {
        "Authorization": `Bearer ${tokenToUse}`,
      },
    });
  }

  try {
    let response = await request(token);

    if (response.status === 401) {
      const refreshed = await refreshToken();
      if (!refreshed) return "";

      const newToken = sessionStorage.getItem("Token");
      if (!newToken) return "";

      response = await request(newToken);
    }

    if (!response.ok) return "";

    const userName = await response.text();
    return userName;
  } catch (err) {
    // console.error("GetUserName error:", err);
    return "";
  }
}
```

79. ábra - `GetUserName` metódus implementációja

Játék ciklus és állapotfrissítés

A játék folyamatos frissítése a Kaplay motor `onUpdate` eseményén keresztül történik. Ebben a ciklusban kerül frissítésre a játékos mozgása, valamint a többi játékos állapota is.

A kliens emellett időzített ciklusban küldi el a saját állapotát a szerver felé. Ez a mechanizmus biztosítja a valós idejű szinkronizációt a játékosok között.

A rendszer úgy van kialakítva, hogy minimalizálja a késleltetést, miközben folyamatos és konzisztens játékélményt biztosít.

```
k.onUpdate(() => {  
    if (player && movementEnabled) {  
        player.update(dir.x, dir.y);  
    }  
  
    otherPlayers.forEach(o => {  
        o.char.update(o.dir.x, o.dir.y);  
    });  
  
    // npc.update(  
    //     npcRoute[npcRouteId].x,  
    //     npcRoute[npcRouteId].y  
    // );  
});
```

80. ábra - Játékállapot frissítése

3.7 Frontend oldalak

Admin Panel

Az admin felület célja a rendszer működésének átfogó felügyelete és az alkalmazás adatainak kezelése. A felület segítségével az adminisztrátor képes megtekinteni a regisztrált felhasználókat, azok adatait, valamint a rendszer működése során keletkező naplóbejegyzéseket. Ez lehetővé teszi a rendszer állapotának folyamatos ellenőrzését, valamint az esetleges hibák gyors azonosítását és kezelését.

A bejelentkezési folyamat működése

Az admin felület eléréséhez a felhasználónak először hitelesítenie kell magát egy bejelentkezési űrlapon keresztül. A felhasználónév és jelszó megadását követően a rendszer egy HTTP POST kérés segítségével elküldi az adatokat a backend felé, ahol megtörténik az ellenőrzés.

Sikeres hitelesítés esetén a szerver egy token-t és egy frissítő token-t ad vissza, amelyek a böngésző munkamenetében kerülnek eltárolásra. Ezek a tokenek biztosítják a további kommunikáció hitelességét. A bejelentkezést követően a felhasználó automatikusan átirányításra kerül az admin felület főoldalára. A működés részletei a bejelentkezési logikát tartalmazó kódrészletben figyelhetők meg.

```
const response = await fetch("https://localhost:7234/api/Login", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify(
    {
      Username: username,
      Password: password
    }
  )
});

if (!response.ok) {
  console.log("error");
  alert("Something went of during the login!");
}
else {
  const data = await response.json();
  sessionStorage.setItem("Token", data.token);
  sessionStorage.setItem("RefreshToken", data.refreshToken.token);
  window.location.href = "../AdminPanel/index.html";
}
```

81. ábra - Belépés kérés elküldése

Az admin felület felépítése

Az admin panel több logikai egységre van bontva, amelyek különböző funkciókat látnak el, azonban egységes felületen belül működnek. A különböző nézetek közötti váltás dinamikusan történik, az oldal újratöltése nélkül.

A kezdőoldal egy áttekintő felületként szolgál, ahol az adminisztrátor gyors információkat kap a rendszer aktuális állapotáról. Itt megjelenik például a felhasználók száma, valamint a rendszerben tárolt logok mennyisége. Ez a nézet egyfajta irányítópultként funkcionál, ahonnan a további funkciók könnyen elérhetők.

A felhasználók kezelése külön nézetben valósul meg, ahol listázásra kerül minden regisztrált felhasználó. Az adatok között szerepel a felhasználónév, a szerepkör, az aktuálisan használt karakter, valamint a teljesített hívások száma. A lista dinamikusan generálódik, amelynek működése a felhasználók megjelenítését végző kódrészletben figyelhető meg.

A logok kezelése szintén külön felületen történik, ahol a rendszer működése során keletkező események jelennek meg. A logok tartalmazzák az esemény típusát, leírását és időpontját. A hibák külön kiemelésre kerülnek, ezáltal az adminisztrátor gyorsan azonosíthatja a problémás működéseket. A logok listázásának működése a megfelelő kódrészletben látható.

```
logs.forEach((log) => {
  let logRecord = document.createElement("div");
  logRecord.className = "log-record";

  let logTime = log.dateTime.slice(0, 23).replace("T", " ");

  logRecord.innerHTML = `
    <div class="data">
      <div style="border-left: none; width: 10%">${log.logType}</div>
      <div style="width: 65%;"><p style="color: ${log.logType == "Error" ? "red" : "black"}">Description: ${log.description}</p></div>
      <div style="width: 25%;">DateTime: ${logTime}</div>
    </div>
  `;

  logRecord.addEventListener("click", () => DisplayDetailedLogPanel(log));
  logRecordsArea.appendChild(logRecord);
});
```

82. ábra - Napló esemény táblázatsorának létrehozása

Részletes nézetek működése

Az admin felület egyik fontos eleme a részletes nézetek használata, amelyek lehetővé teszik az adatok mélyebb megtekintését anélkül, hogy a felhasználó elhagyná az aktuális oldalt. Ezek a nézetek egy felugró, úgynevezett overlay panel segítségével jelennek meg.

A felhasználók részletes nézete tartalmazza az adott felhasználóhoz kapcsolódó összes fontos információt, például a karakterét, statisztikáit, coin egyenlegét, valamint az elért eredményeit és megszerzett elemeit. Ez a nézet átfogó képet ad az adott felhasználó aktivitásáról.

A logok részletes nézete további információkat biztosít az adott eseményről, beleértve a pontos időpontot, az érintett felhasználót, valamint a részletes leírást és az esetleges hibaüzenetet. A működésük a megfelelő megjelenítő függvényekben követhető nyomon.

```
export default function DisplayDetailedLogPanel(log) {
  const blackOverlay = document.getElementById("black-overlay");
  blackOverlay.style.display = "flex";

  const closeDetailedLogPanelBtn = document.getElementById(
    "close-detailed-log-panel-btn",
  );
  closeDetailedLogPanelBtn.addEventListener(
    "click",
    () => (blackOverlay.style.display = "none"),
  );

  const logTypeArea = document.getElementById("log-type");
  logTypeArea.textContent = log.logType;

  const logTimeArea = document.getElementById("log-time");
  const logTime = log.dateTime.toLocaleString("hu-HU", {
    year: "numeric",
    month: "2-digit",
    day: "2-digit",
    hour: "2-digit",
    minute: "2-digit",
  });
  logTimeArea.textContent = logTime;

  const logUserNameArea = document.getElementById("user-name");
  logUserNameArea.textContent = log.userName;

  const logDescriptionArea = document.getElementById("log-description");
  logDescriptionArea.textContent = log.description;

  const logErrorArea = document.getElementById("log-error");
  logErrorArea.textContent = log.error;
}
```

83. ábra - Napló részletes adatlapjának felépítése

Navigáció és felhasználói élmény

A felhasználói élmény javítása érdekében az admin felület egy egyszerű, de hatékony navigációs rendszert használ. A különböző panelek közötti váltást egy hamburger menü segíti, amely animált módon jelenik meg és tűnik el.

A navigáció eseménykezelők segítségével valósul meg, így a felhasználó gyorsan és gördülékenyen tud váltani az egyes nézetek között. Az adatok frissítési ideje minden nézetben megjelenítésre kerül, amely segíti az adminisztrátort abban, hogy mindig tisztában legyen az adatok aktualitásával.

Adatkezelés és API kommunikáció

Az admin felület működésének alapját az API kommunikáció biztosítja. Az adatok lekérése különböző végpontokon keresztül történik, például külön kérés szolgál a felhasználók és külön a logok lekérésére.

Minden API kérés hitelesítést igényel, amely a bejelentkezés során kapott token segítségével történik. Ez biztosítja, hogy az adatokhoz csak jogosult felhasználók férhessenek hozzá. Az adatlekérési folyamat működése a megfelelő API hívásokat tartalmazó kódrészletekben figyelhető meg.

Biztonság

A rendszer biztonságát token alapú hitelesítési mechanizmus biztosítja. A felhasználó csak sikeres bejelentkezést követően kap hozzáférést az admin felülethez, és minden további kérés során kötelező a token használata.

Amennyiben a token hiányzik vagy érvénytelen, a rendszer automatikusan megszakítja a hozzáférést, és visszairányítja a felhasználót a fő oldalra. Ez a megoldás biztosítja, hogy illetéktelen személyek ne férhessenek hozzá az adminisztrációs funkciókhoz.

Custom item shop

A Custom Item Shop felület célja, hogy a felhasználók számára lehetőséget biztosítson különböző avatárok megtekintésére és megvásárlására. Az oldal egy dinamikusan betöltődő webshopként működik, amely közvetlen kapcsolatban áll a backend API-val, így a megjelenített adatok mindig aktuálisak. A felület egyszerű és átlátható kialakítású, amely biztosítja a gyors és intuitív használatot.

Az oldal működése

Az oldal betöltésekor a frontend automatikusan lekéri az elérhető avatárokat a backend rendszerből egy API hívás segítségével. A lekért adatok tartalmazzák az avatárok nevét és árát, amelyek alapján a rendszer dinamikusan generálja a megjelenítendő elemeket.

A termékek listája egy központi konténerben jelenik meg, ahol minden avatar külön blokkban látható. Ezek a blokkok tartalmazzák az adott avatar képét, nevét, árát, valamint egy vásárlás gombot. A lista megjelenítése teljes mértékben JavaScript segítségével történik.

```
function DisplayItems() {
  const itemList = document.getElementById("item-list");
  itemList.innerHTML = ``;

  list.forEach((avatar) => {
    itemList.innerHTML += `<div class="item">
      
      <h2 class="item-name" id='${avatar.avatarName}-id'>${avatar.avatarName}</h2>
      <p class="item-price">${avatar.price} coins</p>
      <button class="buy" onclick="Purchase('${avatar.avatarName}')>Buy</button>
    </div>`;
  });
}
```

84. ábra - Az avatárok dinamikus megjelenítését végző DisplayItems függvény

Nyelvválasztás működése

A felület többnyelvű támogatással rendelkezik, amely lehetővé teszi a felhasználók számára, hogy különböző nyelvek között váltsanak. A jelenleg támogatott nyelvek az angol, a magyar és a spanyol.

A nyelvválasztás egy külön menü segítségével történik, amely a nyelv gombra kattintva jelenik meg. A kiválasztott nyelv eltárolásra kerül a böngésző LocalStorage-jében, így az oldal újratöltése után is megmarad. A nyelv módosítása során a rendszer dinamikusan frissíti: az avatárok neveit, a vásárlás gomb feliratát, valamint a nyelvválasztó gomb szövegét.

```
function setLanguage(lang) {
  localStorage.setItem("lang", lang);

  document.getElementById("lang-selection").style.display = "none";
  langButton.textContent = texts.langButton[lang];
  list.forEach((avatar) => {
    document.getElementById(`${avatar.avatarName}-id`).textContent =
      texts[avatar.avatarName][lang];
  });

  document
    .querySelectorAll(".buy")
    .forEach((button) => (button.textContent = texts.buyButtonText[lang]));
}
```

85. ábra - A felület szövegeinek frissítését végző `setLanguage` függvény

Vásárlási folyamat működése

A vásárlás a felhasználó által kiválasztott avatarhoz tartozó gomb megnyomásával indítható el. A gomb egy eseménykezelőn keresztül meghívja a vásárlási logikát, amely egy külön modulban került megvalósításra.

A vásárlási folyamat során a rendszer először ellenőrzi, hogy a felhasználó rendelkezik-e érvényes hitelesítési tokennel. Amennyiben a felhasználó nincs bejelentkezve, a rendszer egy hibaüzenetet jelenít meg.

Sikeres hitelesítés esetén a rendszer egy HTTP POST kérést küld a backend felé, amely tartalmazza a kiválasztott avatar nevét. A kérés során a token az Authorization fejlécben kerül elküldésre, így biztosított a biztonságos kommunikáció.

A backend válaszára a rendszer visszajelzést ad a felhasználónak. Sikeres vásárlás esetén megerősítő üzenet jelenik meg, hiba esetén a backend által küldött üzenet kerül megjelenítésre.

```
const response = await fetch(`${baseUrl}/api/PurchaseAvatar`, {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    Authorization: `Bearer ${sessionStorage.getItem("Token")}`,
  },
  body: JSON.stringify({
    avatarName: avatarName,
  }),
});
```

86. ábra - Avatar vásárlásának megvalósítása HTTP POST kérés segítségével

```
export default async function PurchaseAvatar(avatarName) {
  if (!sessionStorage.getItem("Token")) {
    return "User not logged in!";
  }

  console.log(sessionStorage.getItem("RefreshToken"));
  await refreshToken(sessionStorage.getItem("RefreshToken"));
```

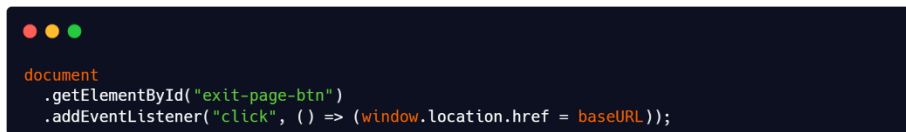
87. ábra - Felhasználói hitelesítés ellenőrzése token segítségével

Navigáció és felhasználói élmény

A felület egyszerű navigációs elemekkel rendelkezik. A felső sávban található egy kilépés gomb, amely lehetővé teszi a felhasználó számára, hogy visszatérjen a főoldalra.

A nyelvválasztó menü dinamikusan jelenik meg és tűnik el, amely javítja a felhasználói élményt. Az eseménykezelők segítségével a rendszer azonnal reagál a felhasználói interakciókra, így az oldal használata gördülékeny és gyors.

A teljes felület újratöltés nélkül működik, amely modern, reszponzív élményt biztosít.



```
document
.getElementById("exit-page-btn")
.addEventListener("click", () => (window.location.href = baseUrl));
```

88. ábra - Navigáció megvalósítása a főoldalra történő visszalépéssel

Adatkezelés és API kommunikáció

A Custom Item Shop működésének alapját az API kommunikáció biztosítja. Az avatárok lekérése egy GET kérés segítségével történik, míg a vásárlás egy POST kérésen keresztül valósul meg.

```
export default async function GetAvatars() {
  const response = await fetch(`${baseUrl}/api/Avatars`);
  if (response.ok) {
    return await response.json();
  }
  return [];
}
```

89. ábra - Avatárok lekérése backend API végpontról

Az adatok kezelése kétféle böngésző tárolási megoldással történik:

- LocalStorage: a kiválasztott nyelv tárolása
- SessionStorage: a hitelesítési token és a frissítő token tárolása

A token minden API kérés során felhasználásra kerül, amely biztosítja a jogosultság ellenőrzését.

```
Authorization: `Bearer ${sessionStorage.getItem("Token")}`,
```

90. ábra - Token alapú hitelesítés használata az API kérés során

Biztonság

A rendszer biztonságát token alapú hitelesítési mechanizmus biztosítja. A vásárlási folyamat csak akkor hajtható végre, ha a felhasználó rendelkezik érvényes tokennel. A rendszer minden vásárlás előtt frissíti a token, így biztosítva annak érvényességét. Amennyiben a token hiányzik vagy érvénytelen, a rendszer nem engedélyezi a művelet végrehajtását. Ez a megoldás garantálja, hogy kizárólag hitelesített felhasználók hajthassanak végre vásárlási műveleteket.

Leaderboard felület

A Leaderboard felület célja, hogy a felhasználók számára áttekinthető módon jelenítse meg a különböző játékokhoz tartozó ranglistákat. Az oldal dinamikusan tölti be az adatokat a backend rendszerből, így mindig az aktuális eredmények jelennek meg. A felület támogatja több különböző játék statisztikáinak megjelenítését, egységes struktúrában.

Az oldal működése

Az oldal betöltésekor a rendszer automatikusan lekéri az összes elérhető ranglistát egy központi API hívás segítségével. Az adatok különböző játékokhoz tartozó eredményeket tartalmaznak, amelyek egy objektumban kerülnek tárolásra.

A lekért adatok alapján a rendszer dinamikusan generálja a megjelenítendő tartalmat. A ranglista elemei egy központi konténerben jelennek meg, ahol minden rekord tartalmazza a felhasználó nevét és az adott játékhoz tartozó eredményt.

```
export default async function GetLeaderboards() {
  const leaderboards = {
    BunnyRunLeaderboard: await GetBunnyRunLeaderboard(),
    FlappyBirdLeaderboard: await GetFlappyBirdLeaderboard(),
    MemoryGameFlipsLeaderboard: await GetMemoryGameFlipsLeaderboard(),
    MemoryGameTimeLeaderboard: await GetMemoryGameTimeLeaderboard(),
    WordleLeaderboard: await GetWordleLeaderboard(),
    AchievementLeaderboard: await GetAchievementLeaderboard(),
  };

  return leaderboards;
}
```

91. ábra - Ranglisták lekérése és összegyűjtése egy közös objektumba

Ranglisták kezelése

A rendszer több különböző ranglistát kezel, amelyek külön API végpontokon keresztül érhetők el. A ranglisták a következők:

- Bunny Run-ban elért távolság
- Flappy Bird-ben elhagyott csövek száma
- Memory Game (idő és fordítás szám külön)
- Wordle-ben teljesített szavak száma
- Teljesített mérföldkövek száma

Minden ranglista külön függvény segítségével kerül lekérésre, majd egy közös objektumban kerülnek összegyűjtésre. Ez a megoldás lehetővé teszi az adatok strukturált és könnyen kezelhető feldolgozását.

```
async function GetBunnyRunLeaderboard() {  
  const response = await fetch(`${baseUrl}/bunnyrun`, {  
    method: "GET",  
    headers: {  
      "Content-Type": "application/json",  
    },  
  });  
  
  if (response.ok) {  
    return response.json();  
  }  
  
  return [];  
}
```

92. ábra - Ranglista lekérése HTTP GET kérés segítségével

Ranglista megjelenítése

A kiválasztott ranglista megjelenítése dinamikusan történik. A felhasználó egy legördülő menü segítségével választhatja ki, hogy melyik játék ranglistáját szeretné megtekinteni.

A kiválasztás után a rendszer törli az előző tartalmat, majd új elemeket generál a megfelelő adatok alapján. A lista fejléct és az adatsorokat JavaScript segítségével hozza létre.

```
function DisplayBunnyRunLeaderBoard() {  
  const bunnyRunLeaderboard = leaderboards.BunnyRunLeaderboard;  
  currentLeaderboard = "bunnyrun";  
  
  dropdownBtn.textContent =  
    texts.bunnyRunLeaderboard[localStorage.getItem("lang")];  
  
  contentArea.innerHTML = "";  
  
  contentArea.innerHTML +=  
    <div class="item">  
      <div class="username"><u>${texts.username[localStorage.getItem("lang")]}</u></div>  
      <div class="record"><u>${texts.maxDistance[localStorage.getItem("lang")]}</u></div>  
    </div>;  
  
  bunnyRunLeaderboard.forEach((element) => {  
    contentArea.innerHTML +=  
      <div class="item">  
        <div class="username">${element.userName}</div>  
        <div class="record">${element.maxDistance}</div>  
      </div>;  
  });  
}
```

93. ábra - Ranglista fejléc és adatok dinamikus megjelenítése

URL paraméter alapú betöltés

A rendszer támogatja a ranglisták közvetlen elérését URL paraméterek segítségével. Amennyiben a felhasználó egy adott ranglistára mutató linket nyit meg, az oldal automatikusan betölti a megfelelő nézetet.

Ez a működés a lekérdezett URL paraméter alapján történik, amely meghatározza, hogy melyik ranglista jelenjen meg alapértelmezetten.

```
const params = new URLSearchParams(window.location.search);
const inputLeaderboardName = params.get("leaderboard") ?? undefined;

switch (inputLeaderboardName) {
  case "bunnyrun":
    DisplayBunnyRunLeaderBoard();
    break;
  case "flappybird":
    DisplayFlappyBirdLeaderBoard();
    break;
  case "memorygametime":
    DisplayMemoryGameTimeLeaderBoard();
    break;
  case "memorygameflips":
    DisplayMemoryGameFlipsLeaderBoard();
    break;
  case "wordle":
    DisplayWordleLeaderBoard();
    break;
  case "achievement":
    DisplayAchievementLeaderBoard();
    break;
  default:
    break;
}
```

94. ábra - Ranglista kiválasztása URL paraméter alapján

Nyelvválasztás működése

A felület többnyelvű támogatással rendelkezik, amely lehetővé teszi a felhasználók számára a különböző nyelvek közötti váltást.

A nyelvválasztás egy lenyíló menü segítségével történik, amely a nyelv gombra kattintva jelenik meg.

Navigáció és felhasználói élmény

A felület egyszerű és intuitív navigációval rendelkezik. A felhasználó egy legördülő menü segítségével válthat a különböző ranglisták között. A navigáció eseménykezelők segítségével valósul meg, amely lehetővé teszi a gyors és gördülékeny váltást az egyes nézetek között.

A felső sávban található egy kilépés gomb, amely visszairányítja a felhasználót a főoldalra.

```
document
.getElementById("bunny-run-btn")
.addEventListener("click", DisplayBunnyRunLeaderBoard);

document
.getElementById("flappy-bird-btn")
.addEventListener("click", DisplayFlappyBirdLeaderBoard);

document
.getElementById("memory-game-time-btn")
.addEventListener("click", DisplayMemoryGameTimeLeaderBoard);

document
.getElementById("memory-game-flips-btn")
.addEventListener("click", DisplayMemoryGameFlipsLeaderBoard);

document
.getElementById("wordle-btn")
.addEventListener("click", DisplayWordleLeaderBoard);

document
.getElementById("achievements-btn")
.addEventListener("click", DisplayAchievementLeaderBoard);

document
.getElementById("exit-page-btn")
.addEventListener("click", () => (window.location.href = baseUrl));
```

95. ábra - Navigáció megvalósítása eseménykezelők segítségével

Profile felület

A Profile oldal célja, hogy a felhasználó számára átfogó képet adjon saját játékbeli adatairól, statisztikáiról, valamint az elért eredményeiről. A felület dinamikusan tölti be az adatokat a backend rendszerből, és azokat strukturált módon jeleníti meg. Az oldal reszponzív kialakítású, így különböző eszközökön is megfelelően jelenik meg és használható marad.

Nyelvválasztás működése

A Profile oldal támogatja a többnyelvű megjelenítést. A felhasználó egy lenyíló menü segítségével választhat a különböző nyelvek közül. A kiválasztott nyelv eltárolásra kerül a LocalStorage-ben, és az oldal ennek megfelelően frissíti a szöveges elemeket.

Navigáció

A felső navigációs sáv tartalmaz egy kilépés gombot, amely visszairányítja a felhasználót a főoldalra. A navigáció eseménykezelők segítségével valósul meg, így a felhasználói interakciók gyorsan és hatékonyan kerülnek feldolgozásra.

Az oldal felépítése

A felület több logikai egységből áll, amelyek különböző típusú adatokat jelenítenek meg.

A felső szekció tartalmazza:

- a felhasználó profilképét,
- a felhasználó nevét,
- valamint a felhasználó Coin egyenlegét.

```
const picture = document.getElementById("profile-pic");
picture.src = `${userData.currentAvatarName}.png`;
const name = document.getElementById("name");
name.textContent = `${userData.userName}`;
const coins = document.getElementById("coins");
coins.textContent = `Coins: ${userData.coins}`;
```

96. ábra - Felhasználói alapadatok dinamikus megjelenítése

Ez alatt helyezkednek el a játékt statisztikák, amelyek külön kártyákban jelennek meg. Ezek az adatok a különböző játékokhoz tartozó eredményeket mutatják, például:

- Wordle teljesített szavak száma
- Memory Game legjobb idő és lépésszám
- Bunny Run maximális távolság
- Flappy Bird legjobb eredmény

```
const minflips = document.getElementById("min-flips");
minflips.textContent = `Min-Flips: ${userData.memoryGameStatus.minFlipping}`;

const maxdistance = document.getElementById("max-distance");
maxdistance.textContent = `Max-Distance: ${userData.bunnyRunStatus.maxDistance}`;

const maxpassedpipes = document.getElementById("max-passed-pipes");
maxpassedpipes.textContent = `Max-Passed-Pipes: ${userData.flappyBirdStatus.maxPassedPipes}`;
```

97. ábra - Játékt statisztikák megjelenítése a felhasználói adatok alapján

Az oldal további része tartalmazza:

- a felhasználó által megszerzett avatárokat,
- valamint a teljesített mérföldköveket.

```
userData.completedAchievements.forEach((achievement) => {
  achievements.innerHTML += `
<p>${achievement.nameEng}</p>`;
});
```

98. ábra - Megszerzett avatárok dinamikus megjelenítése

Adatbetöltés és inicializálás

Az oldal működésének alapját az inicializáló függvény biztosítja, amely az oldal betöltésekor automatikusan lefut.

Ez a folyamat a felhasználó adatainak lekérésével kezdődik, amely egy API hívás segítségével történik. A rendszer először lekéri a felhasználó nevét, majd ezt felhasználva további adatokat kér le a backendtől.

```
async function Start() {  
  const userData = await GetUserData();  
  
  userData !== undefined  
    ? await displayUserData(userData)  
    : (window.location.href = baseUrl);  
}
```

99. ábra - Az oldal inicializálása és a felhasználói adatok betöltése

Amennyiben az adatlekérés sikertelen, a rendszer visszairányítja a felhasználót a főoldalra.

```
export default async function GetUserData() {  
  let userName = await GetName();  
  const response = await fetch(`${baseUrl}/api/GetUserData`, {  
    method: "POST",  
    headers: {  
      "Content-Type": "application/json",  
    },  
    body: JSON.stringify({  
      userName: userName,  
    }),  
  });  
  
  if (!response.ok) {  
    return undefined;  
  }  
  const data = await response.json();  
  return data;  
}
```

100. ábra -Felhasználói adatok lekérése backend API segítségével

Adatkezelés és API kommunikáció

A Profile oldal működésének alapját az API kommunikáció biztosítja. Az adatok lekérése több lépésben történik:

- a felhasználónév lekérése hitelesített kérés segítségével,

```
async function GetName() {
  await refreshToken(sessionStorage.getItem("RefreshToken"));
  const response = await fetch(`${baseUrl}/api/GetUserName`, {
    method: "GET",
    headers: {
      Authorization: `Bearer ${sessionStorage.getItem("Token")}`,
    },
  });

  if (response.ok) {
    const data = await response.text();
    return data;
  }
}
```

101. ábra - Felhasználónév lekérése hitelesített API kérés segítségével

- a teljes felhasználói adat lekérése POST kérésen keresztül.

```
const response = await fetch(`${baseUrl}/api/GetUserData`, {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
  },
  body: JSON.stringify({
    userName: userName,
  }),
});
```

102. ábra - Felhasználói adatok lekérése POST kérésen keresztül

Az API kommunikáció során a rendszer token alapú hitelesítést használ, amely biztosítja, hogy csak jogosult felhasználók férhessenek hozzá az adatokhoz.

Avatar kezelés

A felhasználó által megszerzett avatárok külön listában jelennek meg. Minden avatar egy képként jelenik meg, amelyre kattintva a felhasználó kiválaszthatja azt aktuális karakterként.

Az avatar váltása egy API hívás segítségével történik, amely elküldi a kiválasztott avatar nevét a backend felé. A sikeres művelet után az oldal újratölti a felhasználói adatokat, így azonnal megjelenik az új profilkép.

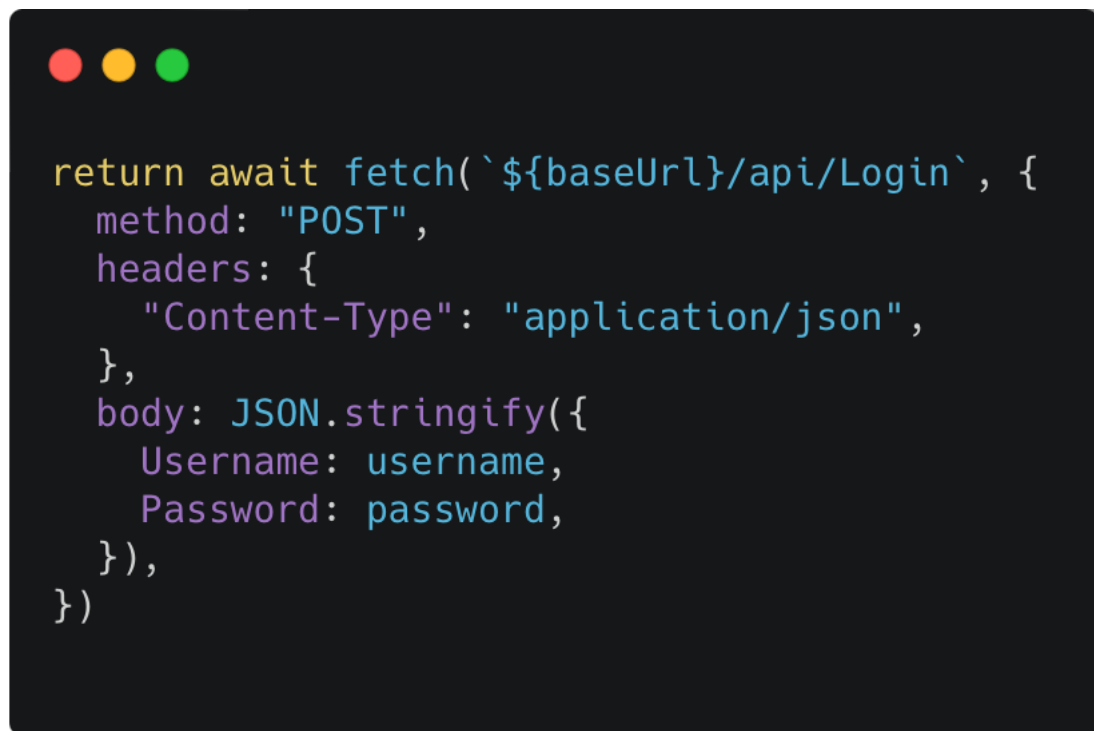
```
export default async function SwitchAvatar(avatarName) {
  const response = await fetch(`${baseUrl}/api/setCharacter`, {
    method: "POST",
    headers: {
      Authorization: `Bearer ${sessionStorage.getItem("Token")}`,
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      characterName: avatarName,
    }),
  });
}
```

103. ábra - Avatar módosítása API hívás segítségével

Bejelentkezés panel

Ez a dokumentáció egy kliensoldali bejelentkezési rendszer működését mutatja be, amely JavaScript alapokon, fetch API használatával kommunikál egy backend szolgáltatással. A megoldás két fő részből áll: az API hívást kezelő modulból és a felhasználói felület logikáját tartalmazó szkriptből. A rendszer célja a felhasználó hitelesítése, tokenek kezelése, valamint többnyelvű felület biztosítása.

A Login.js fájl egy aszinkron függvényt exportál, amely POST kérést küld a szerver /api/Login végpontjára. A kérés JSON formátumban tartalmazza a felhasználónevet és jelszót. Sikeres válasz esetén a rendszer eltárolja a kapott token és refreshToken értékeket a sessionStorage-ben, így azok a munkamenet során elérhetők maradnak. Hibás válasz esetén a függvény visszaadja a HTTP státuszkódot, amely alapján a hívó fél eldöntheti a további lépéseket. Hálózati hiba esetén a rendszer konzolra írja a hibát, majd tovább dobja azt.

A screenshot of a code editor window with a dark background and light-colored text. The window has three colored window control buttons (red, yellow, green) in the top-left corner. The code is a JavaScript function that uses the fetch API to send a POST request to the /api/Login endpoint. The request body is a JSON object containing 'username' and 'password'. The function returns the result of the fetch call, which is awaited.

```
return await fetch(`${baseUrl}/api/Login`, {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
  },
  body: JSON.stringify({
    Username: username,
    Password: password,
  }),
})
```

104. ábra - Login API hívás

A script.js fájl felel a felhasználói felület működéséért. A program dinamikusan generál egy animált háttéreffektet (részecskék), amely vizuális élményt nyújt. Emellett kezeli a nyelvválasztást is: a felhasználó kiválaszthatja a kívánt nyelvet, amelyet a rendszer localStorage-ben tárol, így az oldal újratöltése után is megmarad. A setLanguage függvény frissíti a DOM elemek szövegeit a kiválasztott nyelv alapján.

```
sessionStorage.setItem("Token", data.token);
sessionStorage.setItem("RefreshToken", data.refreshToken.token);
```

105. ábra - Tokenek mentése

A bejelentkezési folyamat az űrlap submit eseményére épül. Az eseménykezelő megakadályozza az alapértelmezett működést, majd meghívja a Login függvényt a felhasználó által megadott adatokkal. A válasz státuszkódja alapján a rendszer sikeres vagy sikertelen bejelentkezési üzenetet jelenít meg, és siker esetén átirányítja a felhasználót egy másik oldalra.

```
function setLanguage(lang) {
  localStorage.setItem("lang", lang);
  document.getElementById("login-h1").textContent = texts.loginH1[lang];
}
```

106. ábra - Nyelv beállítás

A rendszer alapértelmezés szerint az angol nyelvet használja, ha nincs korábban elmentett beállítás. A setLanguage függvény automatikusan lefut az oldal betöltésekor, így biztosítva a konzisztens felhasználói élményt.

Regisztráció panel

Ez a dokumentáció a kliensoldali regisztrációs rendszer működését mutatja be, amely szintén JavaScript alapokon, fetchAPI használatával kommunikál a backend szolgáltatással. A megoldás felépítése hasonló a bejelentkezési modulhoz: külön fájl kezeli az API hívást, míg a felhasználói felület logikája egy külön fájlban található. A rendszer célja új felhasználók regisztrálása, valamint a felhasználói élmény javítása többnyelvű támogatással és vizuális elemekkel.

A Register.js fájl egy aszinkron függvényt exportál, amely POST kérést küld a szerver /api/Register végpontjára. A kérés JSON formátumban tartalmazza a felhasználónevet és jelszót. A válasz feldolgozása egyszerűbb, mint a bejelentkezésnél: a rendszer kizárólag a HTTP státuszkódot adja vissza, amely jelzi a művelet sikerességét vagy sikertelenségét. Hálózati hiba esetén a rendszer naplózza a hibát, majd továbbdobja azt.

A code snippet is displayed within a dark-themed window with three colored window control buttons (red, yellow, green) in the top-left corner. The code is a JavaScript function that uses the fetch API to send a POST request to the /api/Register endpoint. The request body is a JSON object containing 'username' and 'password' fields. The function returns the response from the fetch call.

```
return await fetch(`${baseUrl}/api/Register`, {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
  },
  body: JSON.stringify({
    username: username,
    password: password,
  }),
})
```

107. ábra – Regisztrációs API hívás

A scripts.js fájl kezeli a felhasználói felületet és az interakciókat. A program itt is létrehoz egy animált háttéreffektet részecskékkel, amely egységes megjelenést biztosít az alkalmazás számára. A többnyelvű támogatás a setLanguage függvényen keresztül valósul meg, amely a kiválasztott nyelv alapján frissíti az oldal szöveges elemeit, és eltárolja a beállítást a localStorage-ben.

```
function setLanguage(lang) {  
    localStorage.setItem("lang", lang);  
    document.getElementById("regist-h1").textContent = texts.registH1[lang];  
}
```

108. ábra – Nyelvkezelés a regisztrációs oldalon

A regisztrációs folyamat az űrlap submit eseményéhez kapcsolódik. Az eseménykezelő először megakadályozza az alapértelmezett működést, majd kiolvassa a felhasználó által megadott adatokat. Egy fontos kliensoldali validáció történik: a rendszer ellenőrzi, hogy a jelszó és a jelszó megerősítése megegyezik-e. Ha nem, akkor hibaüzenetet jelenít meg, és nem küldi el a kérést a szerver felé.

```
if (password !== passwordAgain) {  
    alert("A jelszavak nem egyeznek!");  
}
```

109. ábra - Jelszó ellenőrzés

Amennyiben a jelszavak egyeznek, a rendszer meghívja a Register függvényt. A visszakapott státuszkód alapján eldönti, hogy sikeres volt-e a regisztráció. Siker esetén (204-es státuszkód) értesíti a felhasználót, majd átirányítja a bejelentkezési oldalra. Sikertelen próbálkozás esetén hibaüzenetet jelenít meg.



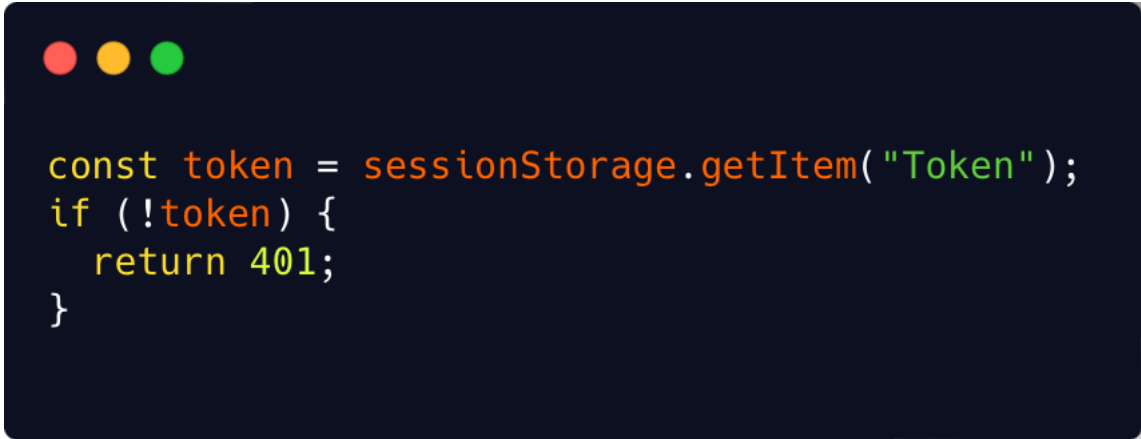
110. ábra - Sikeres regisztráció kezelése

A rendszer alapértelmezett nyelve itt is az angol, amely automatikusan betöltődik, ha nincs korábbi beállítás. Ez biztosítja a konzisztens működést és felhasználói élményt minden oldalbetöltéskor.

Landing Page

Ez a modul a felhasználói jogosultságok ellenőrzéséért és a főoldal (landing page) dinamikus működéséért felel. A rendszer két fő részből áll: a CheckUserRole.js, amely a felhasználó hitelesítési állapotát vizsgálja, valamint a landing.js, amely a felhasználói felületet és navigációt kezeli a jogosultságok alapján. A cél az, hogy a felhasználó csak a számára engedélyezett funkciókat érje el.

A CheckUserRole.js modul első lépésben ellenőrzi, hogy létezik-e token a sessionStorage-ben. Ha nem található, a rendszer azonnal 401-es státuszkóddal tér vissza, amely azt jelzi, hogy a felhasználó nincs bejelentkezve. Amennyiben van token, a rendszer először meghív egy token frissítő függvényt (refreshToken), majd egy GET kérést küld a backend felé a felhasználó ellenőrzésére.

A code editor window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. It contains the following JavaScript code:

```
const token = sessionStorage.getItem("Token");
if (!token) {
  return 401;
}
```

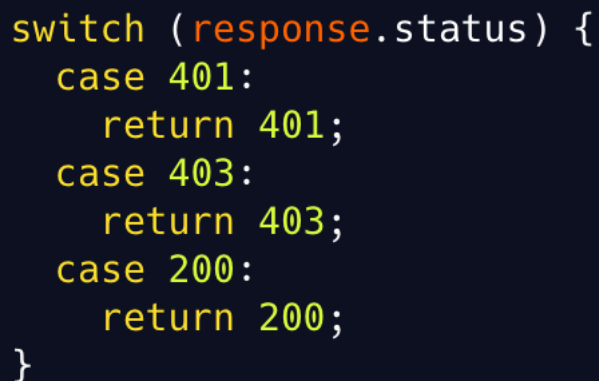
111. ábra - Token ellenőrzés



```
const response = await fetch(`${baseUrl}/getName`, {
  method: "GET",
  headers: {
    Authorization: `Bearer ${token}`,
  },
});
```

112. ábra - API kérés autorizációval

A válasz feldolgozása egy switch szerkezetben történik, amely a HTTP státuszkód alapján határozza meg a felhasználó jogosultságát. A 200-as kód sikeres hitelesítést, a 403-as tiltott hozzáférést, míg a 401-es jogosulatlan hozzáférést jelent.



```
switch (response.status) {
  case 401:
    return 401;
  case 403:
    return 403;
  case 200:
    return 200;
}
```

113. ábra - Státuszkód kezelés

A landing.js fájl felelős a főoldal működéséért. Ez magában foglalja a nyelvkezelést, a navigációt, valamint az egyes elemek megjelenítését a felhasználó jogosultsága alapján. A SetLanguage függvény frissíti az oldal szövegeit a kiválasztott nyelv szerint, és eltárolja a beállítást a localStorage-ben.



```
function SetLanguage(landing_lang) {  
    localStorage.setItem("lang", landing_lang);  
    document.getElementById("nyelvvalt").textContent =  
        textTranslate.nyelvvalt[landing_lang];  
}
```

114. ábra - Nyelv beállítás

A rendszer különböző eseménykezelőket rendel a gombokhoz, amelyek lehetővé teszik a navigációt az alkalmazás különböző részei között (pl. játékok, profil, ranglista). Ez egyszerű kattintás eseményekkel történik, amelyek átirányítják a felhasználót más oldalakra.



```
document.getElementById("flappy-bird-play-btn")  
    .addEventListener("click",  
        () => (window.location.href = "../flappyBird/index.html"));
```

115. ábra – Navigáció a főoldalon

A legfontosabb működési rész a felhasználói szerepkör (role) ellenőrzése. A rendszer meghívja a `CheckUserRole` függvényt, majd az eredmény alapján módosítja a felületet. Három fő eset különböztethető meg:

401 (nincs bejelentkezve): a felhasználó csak bejelentkezni tud, profil és admin funkciók rejtve maradnak

403 (normál felhasználó): elérhető a profil, de az admin panel nem

200 (admin): minden funkció elérhető, beleértve az admin panelt is

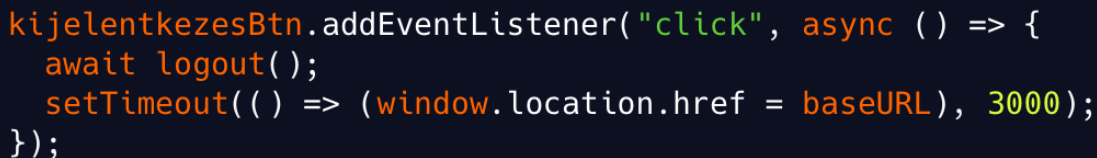


```
const role = await CheckUserRole();

if (role == 403) {
  document.getElementById("profile-btn").style.display = "block";
} else if (role == 200) {
  document.getElementById("admin-panel-btn").style.display = "block";
}
```

116. ábra - Jogosultság kezelés

A kijelentkezés kezelése szintén a jogosultságtól függ. Bejelentkezett felhasználók esetén a logout függvény kerül meghívásra, majd a rendszer visszairányít a kezdőoldalra. Nem bejelentkezett felhasználók esetén a gomb a login oldalra navigál.



```
kijelentkezesBtn.addEventListener("click", async () => {
  await logout();
  setTimeout(() => (window.location.href = baseUrl), 3000);
});
```

117. ábra – Kijelentkezés gomb

Achievemets Panel

Ez a modul a felhasználó által elérhető mérföldkövek (achievements) megjelenítéséért felel. A rendszer két fő részből áll: a GetAchievements.js, amely a backend API-ból lekéri az adatokat, valamint az achievements.js, amely a felhasználói felületen dinamikusan megjeleníti ezeket az adatokat. A cél egy átlátható, többnyelvű lista biztosítása a felhasználó számára.

A GetAchievements.js egy egyszerű aszinkron függvényt tartalmaz, amely egy GET kérést küld az /api/GetAchievements végpontra. Sikeres válasz esetén JSON formátumban visszaadja az adatokat, ellenkező esetben egy üres tömböt ad vissza. Ez biztosítja, hogy hiba esetén se omoljon össze a felület.

```
const response = await fetch(`${baseUrl}/api/GetAchievements`);
if (response.ok) {
  return await response.json();
}
return [];
```

118. ábra – Mérföldkövek lekérése

Az achievements.js fájl kezeli a felhasználói felületet és a megjelenítést. A SetLanguage függvény itt is a többnyelvűséget biztosítja: eltárolja a kiválasztott nyelvet, majd frissíti az oldal elemeit. Fontos, hogy a nyelv váltása után újra meghívja a DisplayAchievements függvényt, így az adatok is az adott nyelven jelennek meg.

```
function SetLanguage(lang_achiev) {
  localStorage.setItem("lang", lang_achiev);
  DisplayAchievements();
}
```

119. ábra - Nyelvkezelés és frissítés

```

async function DisplayAchievements() {
  var list = await GetAchievements();

  const itemList = document.getElementById("item-list");
  itemList.innerHTML = " ";
  var lang = localStorage.getItem("lang") ?? "Eng";
  var langa = lang.charAt(0).toUpperCase() + lang.slice(1);
  list.forEach((achievement) => {
    itemList.innerHTML += `
      <div class="item">
        <p class="achievement-name">${achievement['name${langa}']}</p>
        <p class="achievement-requirement">${achievement['requirement${langa}']}</p>
        <p class="achievement-prize">${texts.prize[localStorage.getItem("lang") ?? "eng"]}: ${achievement.prizeCoin} C</p>
      </div>`;
  });
}

```

104. ábra - Mérföldkövek megjelenítése

DisplayAchievements függvény felelős az adatok megjelenítéséért. Meghívja a GetAchievements függvényt, majd a visszakapott lista alapján dinamikusan generál HTML elemeket. A rendszer figyelembe veszi a kiválasztott nyelvet, és ennek megfelelően választja ki az objektum megfelelő mezőit (pl. nameEng, nameHun, stb.).

A megjelenítés során a rendszer template stringeket használ, amelyekkel könnyen generálható dinamikus HTML tartalom. Minden achievement külön blokkban jelenik meg, tartalmazva a nevét, követelményét és jutalmát.

A modul tartalmaz egy visszalépési funkciót is, amely lehetővé teszi a felhasználó számára, hogy visszatérjen a főoldalra. Ez egy egyszerű átirányítással valósul meg. Az oldal betöltésekor automatikusan lefut a DisplayAchievements és a SetLanguage függvény, így biztosított, hogy az adatok és a nyelvi beállítások azonnal megjelenjenek.

```

document.getElementById("exit-page-btn")
  .addEventListener("click", () => {
    window.location.href = baseUrl;
  });

```

121. ábra - Navigáció vissza főoldalra

4. Tesztelési dokumentáció

4.1 Bevezetés

A rendszer működése során a felhasználói interakciók kezelése, valamint az adatok lekérése és módosítása szerveroldali kommunikáción keresztül történik. A tesztelési folyamat célja az volt, hogy ellenőrizze az alkalmazás helyes működését különböző felhasználói helyzetekben, valamint biztosítsa a rendszer stabilitását és megbízhatóságát.

A fejlesztés során a tesztelés folyamatosan jelen volt, elsősorban manuális formában. A funkciók implementálását követően azonnali ellenőrzés történt, amely során különböző felhasználói műveletek kerültek kipróbálásra. A későbbi szakaszban a backend végpontok tesztelése strukturáltabb módon, Postman alkalmazás segítségével valósult meg, amely lehetővé tette a HTTP kérések és válaszok részletes vizsgálatát.

Létrehozott felhasználók és adataik

Felhasználónév	Jelszó	Jogosultsághör
<i>Test</i>	<i>Test</i>	<i>Felhasználó</i>
<i>test2</i>	<i>test123</i>	<i>Felhasználó</i>
<i>test3</i>	<i>test123</i>	<i>Felhasználó</i>
<i>admin1</i>	<i>admin123</i>	<i>Adminisztrátor</i>
<i>admin2</i>	<i>admin123</i>	<i>Adminisztrátor</i>
<i>admin3</i>	<i>admin123</i>	<i>Adminisztrátor</i>

4.2 Tesztelési módszerek

A tesztelési folyamat során több különböző módszer alkalmazására került sor annak érdekében, hogy a rendszer működése minden szempontból ellenőrizhető legyen. A fekete doboz tesztelés során a rendszer viselkedését a bemenetek és a kimenetek alapján vizsgáltam, anélkül hogy a belső működés részleteit figyelembe vettem volna. Ez a megközelítés különösen hasznos volt a felhasználói felület és az interakciók tesztelése során, ahol a cél az volt, hogy a rendszer a felhasználó szempontjából megfelelően működjön.

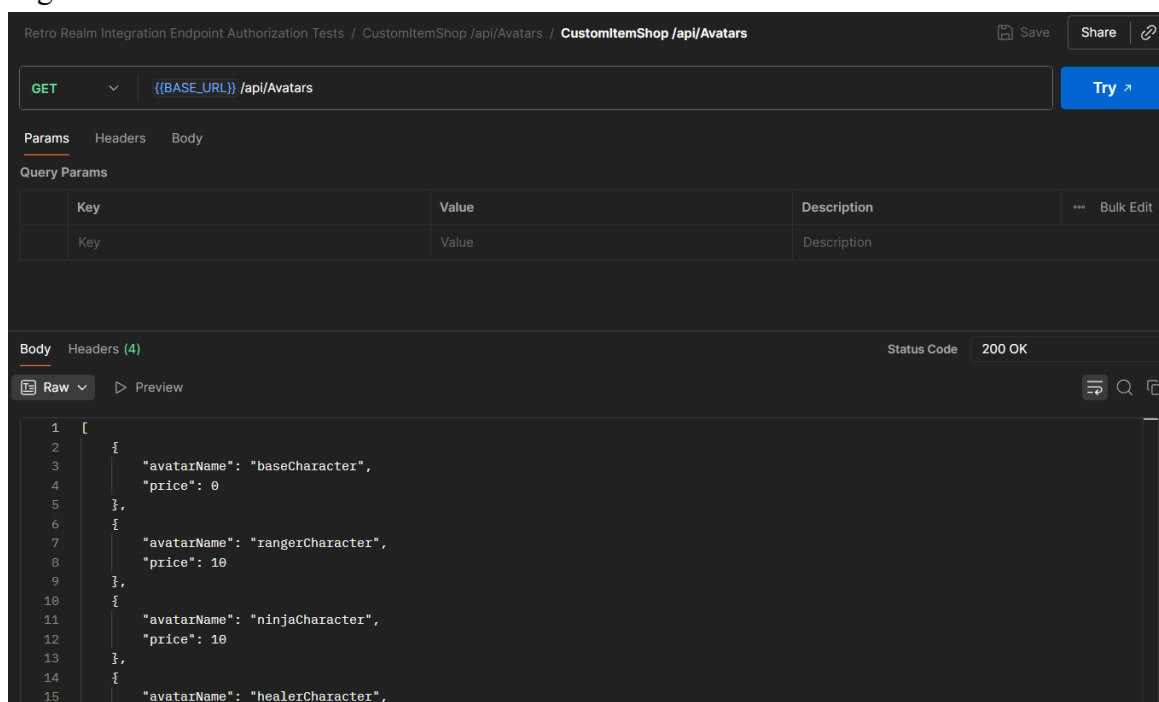
Ezzel párhuzamosan fehér doboz tesztelést is alkalmaztam, amely során a program belső logikáját és működését vizsgáltam. Ennek keretében ellenőriztem az adatkezelést, a feltételek helyes működését, valamint az egyes funkciók közötti kapcsolatokat. Ez a módszer lehetővé tette a rejtettebb, logikai jellegű hibák feltárását.

A fejlesztés során a manuális tesztelés folyamatosan jelen volt, amely során különböző felhasználói műveletek kipróbálásával ellenőriztem a rendszer működését. A backend funkcionalitások teszteléséhez Postman alkalmazást használtam, amely segítségével különböző HTTP kéréseket küldtem a szerver felé, és elemeztem a kapott válaszokat.

4.3 Tesztesetek bemutatása

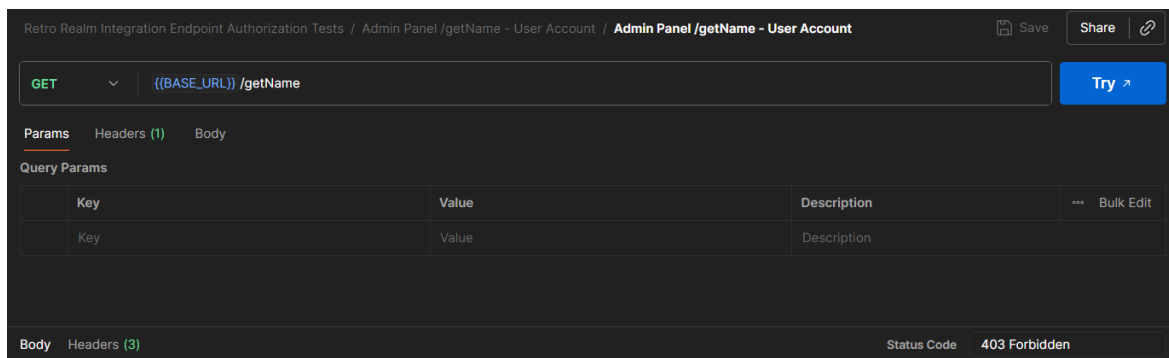
A tesztelés során különböző típusú tesztesetek kerültek végrehajtásra, amelyek közül kiemelhető egy normál működést vizsgáló eset, valamint egy extrém, úgynevezett bolondbiztos teszt.

A normál tesztet során a felhasználó a rendszer által elvárt módon használja az alkalmazást. Ebben az esetben a felhasználó elindítja az alkalmazást, majd végrehajt egy adott műveletet, például adatlekérést vagy egy interaktív funkció használatát. A rendszer a kérés feldolgozását követően megfelelő választ ad, amely a felhasználói felületen is megjelenik. A teszt során azt tapasztaltam, hogy a rendszer helyesen reagál a felhasználói műveletekre, az adatok megfelelően jelennek meg, és a működés folyamatos, megszakításmentes.



105. ábra - Bejelentkezés nélkül is elérhető végpont

Az extrém tesztelés célja az volt, hogy a rendszer viselkedését nem megfelelő vagy szélsőséges felhasználói műveletek esetén vizsgáljam. Ennek során például gyors egymás utáni műveleteket hajtottam végre, illetve hibás vagy hiányos adatokat adtam meg. A teszt során azt tapasztaltam, hogy a rendszer nem omlik össze, hanem megfelelő módon kezeli a hibás bemeneteket. A backend válaszok között megjelentek hibaüzenetek, például „Unauthorized” vagy „Bad Request”, amelyek egyértelműen jelezték a probléma okát.

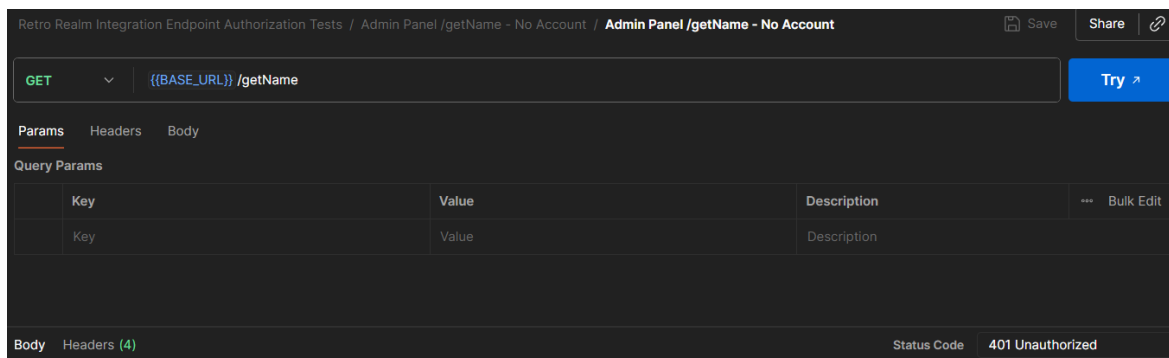


106. ábra - Admin név lekérésének elutasítása nem admin user esetén

4.4 API tesztelés

A backend működésének ellenőrzése Postman alkalmazás segítségével történt. A tesztelés során különböző végpontokat vizsgáltam, amelyek lehetővé tették az adatok lekérését és módosítását. A kérések során HTTP metódusokat használtam, mint például GET és PUT, valamint figyelembe vettem az autentikációs mechanizmust is.

Egy tipikus tesztelés során a rendszer egy GET kérés segítségével visszaadta a felhasználói adatokat. Amennyiben a kérés érvényes tokennel történt, a válasz sikeres volt, és a rendszer JSON formátumban visszaküldte az adatokat. Hibás vagy hiányzó token esetén a szerver 401-es hibakóddal válaszolt, amely jelzi, hogy a felhasználó nincs megfelelően hitelesítve.



107. ábra - Autorizáció nélküli kérés elutasítása

A tesztelés során adatfrissítési műveletek is vizsgálatra kerültek. Ezek során a rendszer megfelelően kezelte a beérkező adatokat, és sikeres esetben frissítette azokat a backend oldalon. Hibás adatok esetén a szerver 400-as hibakódot adott vissza, amely jelzi, hogy a kérés formátuma vagy tartalma nem megfelelő.

A backend tesztelése során a Postman alkalmazásban létrehozott tesztesetek nemcsak futtatásra kerültek, hanem dokumentálás céljából exportálásra is. Az exportált collection tartalmazza a különböző végpontokra küldött kéréseket, valamint azok válaszait és végkimeneteleit is, így átfogó képet ad a rendszer működéséről.

Az elkészített Postman collection export fájl a beadandó dokumentáció mellékleteként került csatolásra, amely lehetővé teszi a tesztelési folyamat utólagos ellenőrzését és reprodukálását. Ezáltal biztosított, hogy a tesztelési eredmények nemcsak leírás szintjén, hanem konkrét példákon keresztül is bemutatathatók.

4.5 Tesztelés során talált hibák

A tesztelési folyamat során több kisebb és egy jelentősebb, autentikációval kapcsolatos hiba is azonosításra került. Az egyik fontos probléma a JWT alapú jogosultságkezelés során jelentkezett. A rendszerben a felhasználói szerepkörök (role-ok) ellenőrzése az Authorize attribútum segítségével történt, például az [Authorize(Roles = "Admin")] használatával.

A hiba abból adódott, hogy a JWT tokenben a szerepkör a ClaimTypes.Role kulcs alatt került tárolásra, azonban ez nem működött minden esetben konzisztensen a backend oldali jogosultságellenőrzéssel. Ennek következtében előfordult, hogy egyes, admin jogosultsághoz kötött végpontok elérhetetlenek voltak akkor is, amikor a felhasználó rendelkezett a megfelelő jogosultsággal.

A probléma megoldása érdekében a tokenben használt claim típusa módosításra került, és a ClaimTypes.Role helyett egy egyszerű "Role" kulcs került alkalmazásra. Ezt követően a jogosultságellenőrzés már megfelelően működött, és az [Authorize(Roles = "Admin")] attribútum konzisztensen kezelte a felhasználói szerepköröket.

A hiba rávilágított arra, hogy a JWT token struktúrájának és a backend konfigurációnak összhangban kell lennie, különösen a jogosultságkezelés esetében.

5. Felhasználói dokumentáció

5.1 Játékok felhasználói dokumentációja

Bunny Run

A játék indítása és betöltése

A Bunny Run játék elindításakor a felhasználó egy rövid betöltési képernyővel találkozik, amelyen megjelenik a játékhoz használt játék motor, a Kaplay logója. Ez a logó a játék technikai hátterére utal, és a betöltési folyamat részeként jelenik meg. A betöltést követően a rendszer automatikusan a főmenübe navigálja a felhasználót, ahol a játék további funkciói elérhetővé válnak.



108. ábra - Kaplay logó a betöltés során

Ez a tartalom kizárólag nagy kijelzőkön jelenik meg, és minimum 1024 képpont szélességű megjelenítést igényel. Kisebb felbontású eszközökön (például tableteken vagy mobiltelefonokon) a tartalom nem érhető el vagy eltérő módon jelenhet meg a felhasználói élmény optimalizálása érdekében.

Főmenü felépítése

A főmenüben a felhasználó egy animált jelenetet láthat, amelyben a játék karaktere, a nyuszi folyamatos ugráló mozgást végez. Ez a vizuális elem a játék hangulatát és dinamikáját hivatott bemutatni. A képernyőn több navigációs lehetőség is elérhető. A bal felső sarokban található gomb segítségével a felhasználó visszatérhet a weboldal főoldalára. A középső részen elhelyezkedő két gomb közül az egyik a játék elindítására szolgál, míg a másik a Bunny Run ranglistájára irányítja a felhasználót, ahol a legjobb eredmények tekinthetők meg.



109. ábra - Játék menüje

Játékmenet és irányítás

A játék elindítását követően a nyuszi karakter automatikusan mozogni kezd, és a játékos feladata, hogy a folyamatosan érkező akadályokat elkerülje. A játéktérben kétféle ellenfél jelenik meg: földi ellenfelek, amelyek rókák formájában jelennek meg, valamint légi ellenfelek, amelyek keselyűként közelítenek a karakter felé. A játékos az akadályokat ugrással tudja kikerülni, amely az egér bal gombjának lenyomásával, illetve érintőképernyős eszközökön érintéssel hajtható végre. A játék folyamatos és egyre nagyobb figyelmet igényel, mivel az akadályok elkerülése gyors reakciót követel meg.



110. ábra - Játékmenet közben megjelenő akadályok

Pontozási rendszer

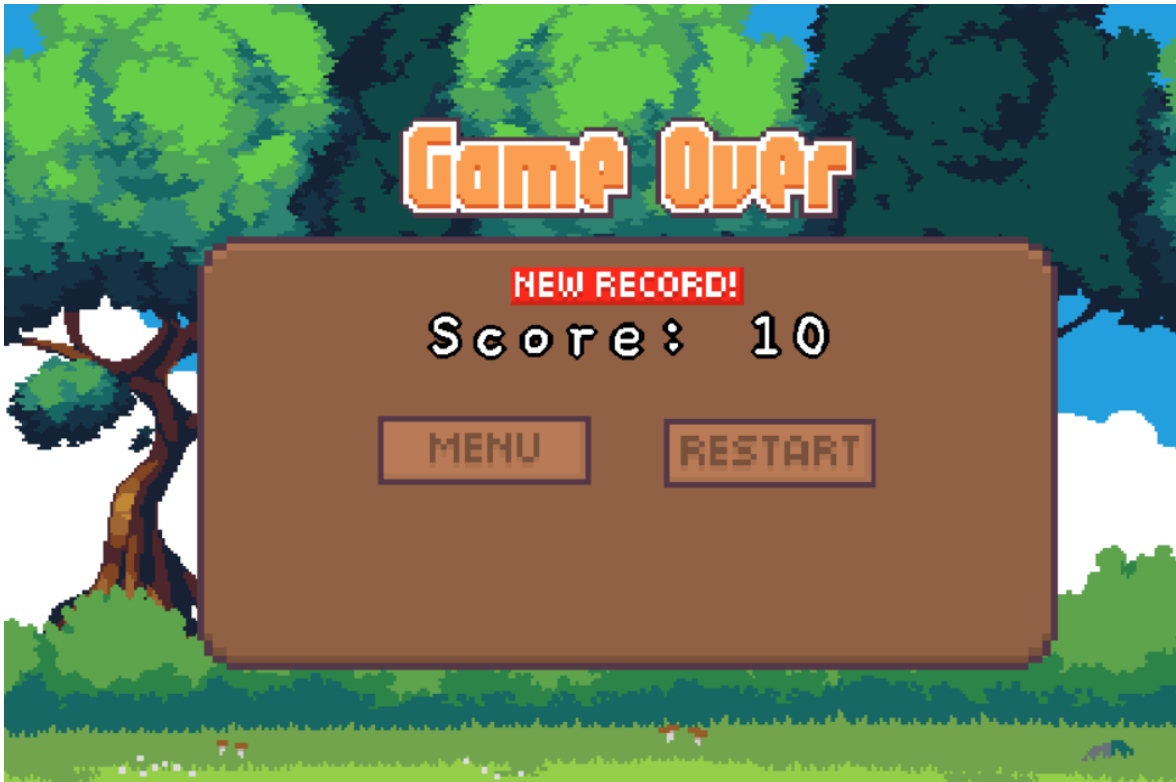
A játék során a pontozás a játékos teljesítményéhez igazodik, amely elsősorban a túlélési időn alapul. Minél hosszabb ideig képes a játékos ütközés nélkül haladni, annál magasabb pontszámot ér el. Az aktuális pontszám a képernyő bal felső sarkában kerül megjelenítésre, így a játékos folyamatos visszajelzést kap a teljesítményéről a játék során.



111. ábra - Pontszám megjelenítése játék közben a bal felső sarokban

Játék vége és eredmények

Amennyiben a játékos karaktere ütközik valamelyik akadállyal, a játék azonnal véget ér, és a rendszer egy záróképernyőt jelenít meg. Ezen a képernyőn a felhasználó megtekintheti az adott játék során elért pontszámát. Abban az esetben, ha a játékos be van jelentkezve a rendszerbe, a játék automatikusan összehasonlítja az aktuális eredményt a korábbi legjobb teljesítménnyel. Ha a játékos új rekordot ér el, ezt a rendszer egy jól látható „NEW RECORD” felirattal jelzi, valamint elmenti az új eredményt.



112. ábra – Záróképernyő a pontszámmal és "NEW RECORD" felirattal

A záróképernyőn a felhasználó számára további lehetőségek is rendelkezésre állnak. Lehetőség van visszatérni a főmenübe, illetve azonnal új játékot indítani. Ez a megoldás biztosítja, hogy a játékos gyorsan újra próbálkozhasson, és javíthassa korábbi eredményét anélkül, hogy hosszabb navigációs folyamaton kellene keresztülmennie.

Flappy Bird

A játék indítása és betöltése

A Flappy Bird játék elindításakor a felhasználó egy rövid betöltési képernyővel találkozhat, amelyen megjelenik a játékhoz használt grafikai motor, a Kaplay logója. Ez a logó a játék technikai hátterére utal, és a betöltési folyamat részeként jelenik meg. A betöltést követően a rendszer automatikusan a főmenübe navigálja a felhasználót.

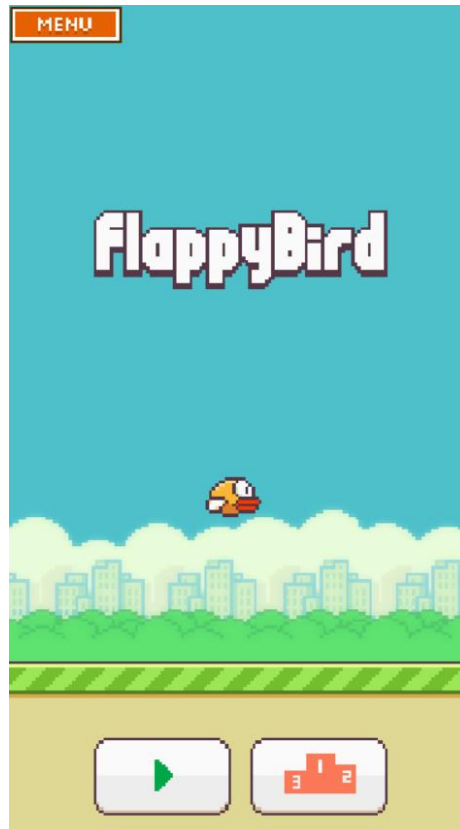


113. ábra - Kaplay logó a betöltés során

Ez a tartalom kizárólag nagy kijelzőkön jelenik meg, és minimum 1024 képpont szélességű megjelenítést igényel. Kisebb felbontású eszközökön (például tableteken vagy mobiltelefonokon) a tartalom nem érhető el vagy eltérő módon jelenhet meg a felhasználói élmény optimalizálása érdekében.

Főmenü felépítése

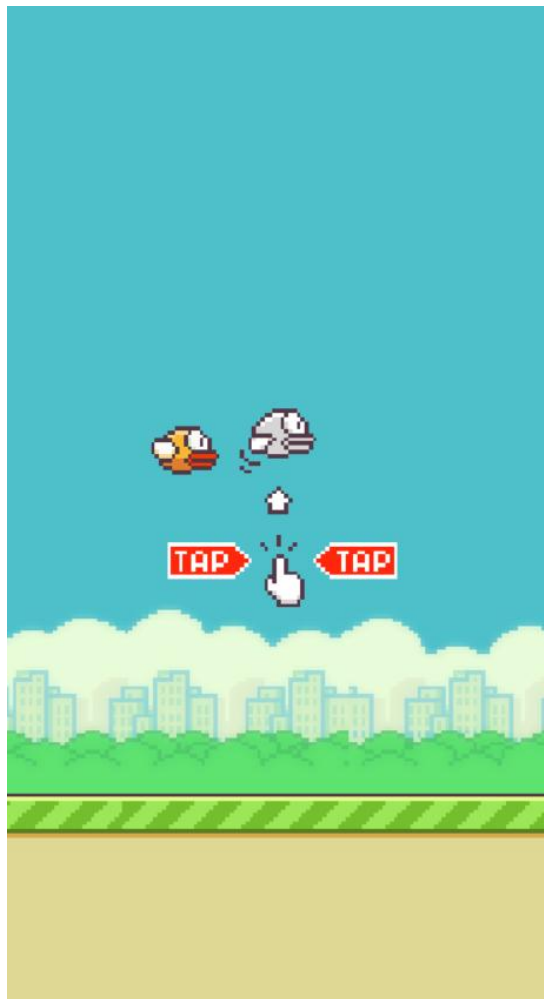
A képernyőn egy animált jelenet látható, amelyben a madár karakter repülő mozgást végez. A bal felső sarokban található gomb segítségével a felhasználó visszatérhet a főoldalra, míg az alsó részen elhelyezkedő két gomb közül az egyik a játék indítására, a másik pedig a ranglista megnyitására szolgál.



114. ábra - Menü felépítése és gombok

Játék kezdése és visszaszámlálás

A játék elindítása után egy rövid visszaszámlálás jelenik meg, amely felkészíti a játékost a kezdésre. Ezt követően a képernyőn egy „Tap Tap” ikon látható, amely jelzi, hogy a játék indításra kész állapotban van. A tényleges játékmenet csak akkor indul el, amikor a játékos az egér bal gombjával kattint, vagy ennek megfelelő érintőpad műveletet hajt végre.

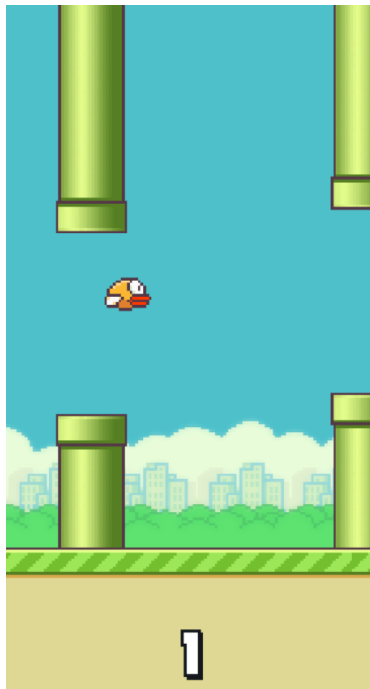


115. ábra - "Tap Tap" felirat a játék kezdésekor

Játékmenet és irányítás

A játék elindítását követően a játékos egy madarat irányít, amely folyamatos mozgásban van. A játék célja, hogy a madár a lehető legtöbb akadályon jusson át anélkül, hogy nekiütközne azoknak. Az akadályokat csövek alkotják, amelyek felülről és alulról közelítenek, közöttük egy szabad nyílással, amelyen keresztül a játékosnak át kell navigálnia a karaktert.

A madár mozgását a játékos az egér bal gombjának lenyomásával, illetve érintőpadon történő érintéssel szabályozhatja. Minden egyes kattintás hatására a madár felfelé mozdul, míg a gravitáció folyamatosan lefelé húzza. A megfelelő ritmus és időzítés elengedhetetlen a sikeres játékhoz.



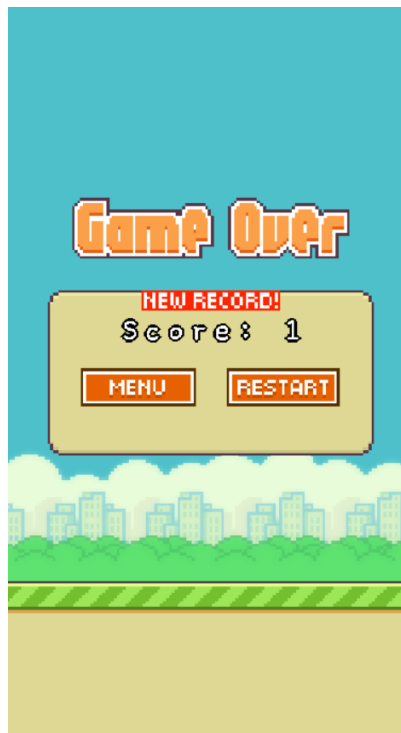
116. ábra - Játékmenet közben megjelenő csövek és pontszám

A játék során a játékos pontokat szerez minden egyes sikeresen kikerült akadály után. A pontszám folyamatosan növekszik, ahogy a játékos egyre több csövön jut át. Az aktuális pontszám a képernyő alsó részén kerül megjelenítésre, így a játékos folyamatosan nyomon követheti teljesítményét.

Játék vége és eredmények

A játék véget ér abban az esetben, ha a madár nekiütközik egy csőnek, elhagyja a játékteret felfelé, vagy a földre zuhan. Ilyenkor a rendszer egy záróképernyőt jelenít meg, ahol a játékos megtekintheti az elért pontszámát.

Amennyiben a játékos be van jelentkezve, a rendszer összehasonlítja az aktuális eredményt a korábbi legjobb teljesítménnyel. Ha a játékos új rekordot ér el, ezt a rendszer egy „NEW RECORD” felirattal jelzi, valamint elmenti az új eredményt.



117. ábra - Játék vége képernyő

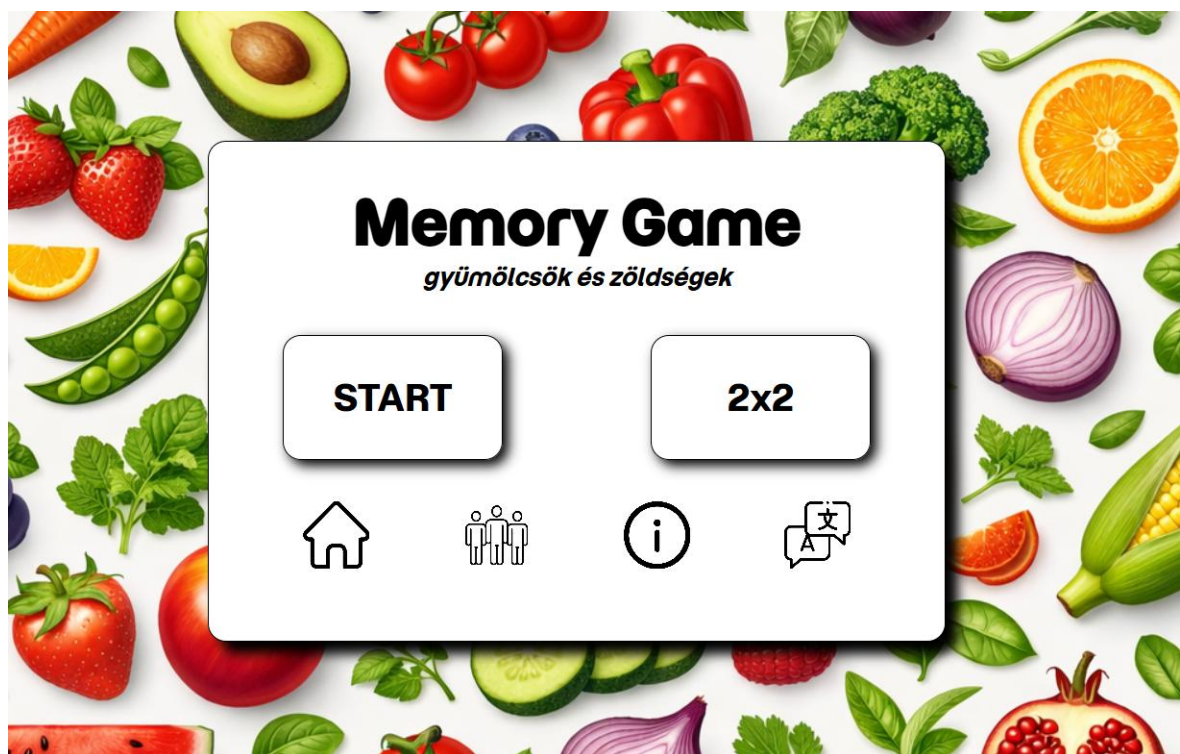
A záróképernyőn a felhasználó számára lehetőség nyílik a játék újraindítására, illetve a főmenübe történő visszatérésre. Ez biztosítja, hogy a játékos gyorsan újra próbálkozhasson, és javíthassa korábbi eredményét.

Memory Game

A játék indítása és menü

A játék indításakor a felhasználó a menübe érkezik, ahol egy gyümölcs- és zöldség témájú memóriajáték felülete fogadja. A képernyőn összesen hat gomb található: két gomb középen, valamint négy gomb az alsó részen. A középső bal oldali gombbal indítható el a játék, míg a jobb oldali gomb segítségével a játék mérete változtatható. A választható méretek: 2×2, 4×4, 6×6 és 8×8.

A képernyő alsó részén található négy gomb különböző funkciókat lát el. Az első gomb a weboldal főmenüjébe navigálja vissza a játékost. A második gomb a játékhoz tartozó ranglistát nyitja meg. A harmadik gomb egy kisegítő panelt jelenít meg, míg a negyedik gomb a nyelvváltásért felel. A játék három nyelven érhető el: magyarul, angolul és spanyolul. A nyelv megváltoztatása az egész felület azonnali frissülésével jár.

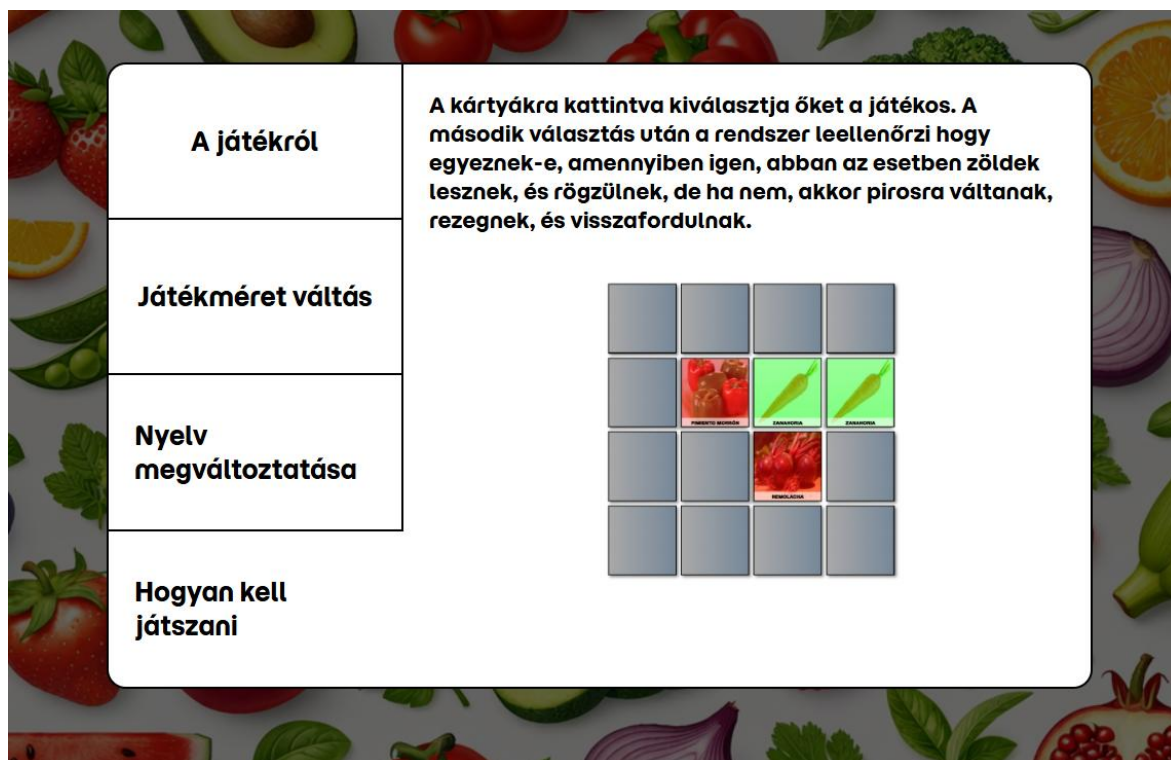


118. ábra - Játék menüje

Ez a tartalom kizárólag nagy kijelzőkön jelenik meg, és minimum 1024 képpont szélességű megjelenítést igényel. Kisebb felbontású eszközökön (például tableteken vagy mobiltelefonokon) a tartalom nem érhető el vagy eltérő módon jelenhet meg a felhasználói élmény optimalizálása érdekében.

Kisegítő panel

A kisegítő panel megnyitásával a játékos részletes információkat kap a játékról. A panel a kiválasztott nyelven jelenik meg, és tartalmazza a játékmenet leírását, a játékméret módosításának módját, valamint a nyelvváltás ismertetését. A panelen belül a bal oldali navigációs felületen lehet váltani az egyes információs részek között. Amennyiben a felhasználó a panelen kívül kattint, az automatikusan bezáródik.



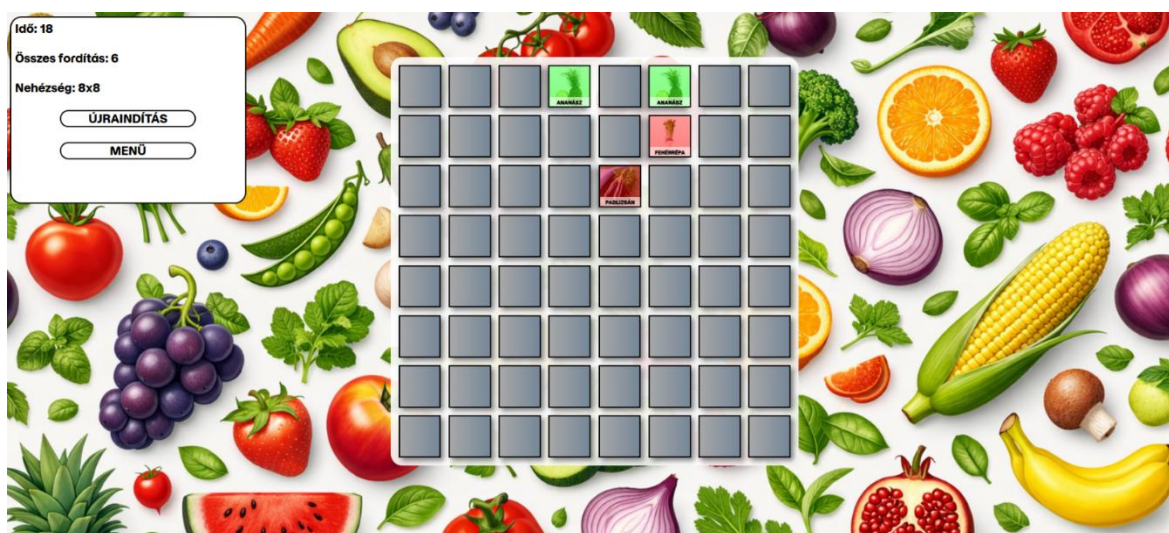
119. ábra - Kisegítő panel

A játék elindítása

A játék indítását követően a kiválasztott méretnek megfelelően jelennek meg a kártyák a játéktéren. A képernyő bal felső sarkában egy információs panel található, amely a játék aktuális állapotáról ad visszajelzést. Emellett két további gomb is megjelenik: az egyik a játék újraindítására szolgál, a másik pedig visszanyitja a főmenübe.

A játék célja a kártyák párhainak megtalálása. A játék során összesen 32 különböző gyümölcs és zöldség képe jelenhet meg. A kártyák kattintással fordíthatók fel, egyszerre legfeljebb kettő. Amennyiben a két kiválasztott kártya megegyezik, zöld színnel kiemelésre kerülnek és rögzítve maradnak. Ha nem egyeznek, piros színt kapnak, rövid ideig rezegnek, majd visszafordulnak. A rendszer folyamatosan számolja a felfordítások számát.

A játék egyik fontos eleme az oktatási jelleg. A háromnyelvű támogatásnak köszönhetően a játékos megismerheti a különböző gyümölcsök és zöldségek neveit több nyelven is. Ez lehetőséget biztosít a szókincs fejlesztésére játékos formában.



120. ábra - Játék felépítése

A játék vége

A játék akkor ér véget, amikor az összes kártya párja megtalálásra kerül. Ekkor a játéktér eltűnik, és megjelenik a játék végi összegző panel.

A záróképernyőn a játékos megtekintheti az elért eredményeit, amelyek közé tartozik a felfordítások száma, a játék ideje (óra–perc–másodperc formátumban), valamint a választott játékméret. Amennyiben a játékos be van jelentkezve, és a játék 8×8-as méretben zajlott, a rendszer ellenőrzi a korábbi rekordokat. Új rekord egy „Új rekord” felirat jelenik meg. A rendszer külön kezeli a legjobb időt és a legkevesebb lépésszámot.

A játék végi panelen két további gomb található. A bal oldali gombbal a játékos azonnal új játékot indíthat, míg a jobb oldali gombbal visszatérhet a menübe.



121. ábra - Játék vége panel

Wordle

Főmenü és navigáció

A játék indításakor a felhasználó a főmenübe érkezik, ahol összesen öt gomb található. A képernyő közepén helyezkedik el a „Start” gomb, amely a játék elindítására szolgál. A panel alsó részén négy további gomb található: az első a weboldal főmenüjébe navigál, a második a ranglistát nyitja meg, a harmadik a kisegítő panelt jeleníti meg, míg a negyedik a nyelv módosítására szolgál.



122. ábra - Játék menüje

Ez a tartalom kizárólag nagy kijelzőkön jelenik meg, és minimum 1024 képpont szélességű megjelenítést igényel. Kisebb felbontású eszközökön (például tableteken vagy mobiltelefonokon) a tartalom nem érhető el vagy eltérő módon jelenhet meg a felhasználói élmény optimalizálása érdekében.

Kisegítő panel és nyelvváltás

A kisegítő panel részletes információkat tartalmaz a játékról, beleértve a játékmenet leírását és a nyelvváltás működését. A panelen belül külön szekciók találhatók, amelyek között a felhasználó könnyen navigálhat. A nyelv módosításához a játékos a nyelvválasztó ikonra kattint, majd kiválasztja a kívánt nyelvhez tartozó zászlót. A rendszer ezt követően azonnal frissíti az oldal teljes nyelvezetét.



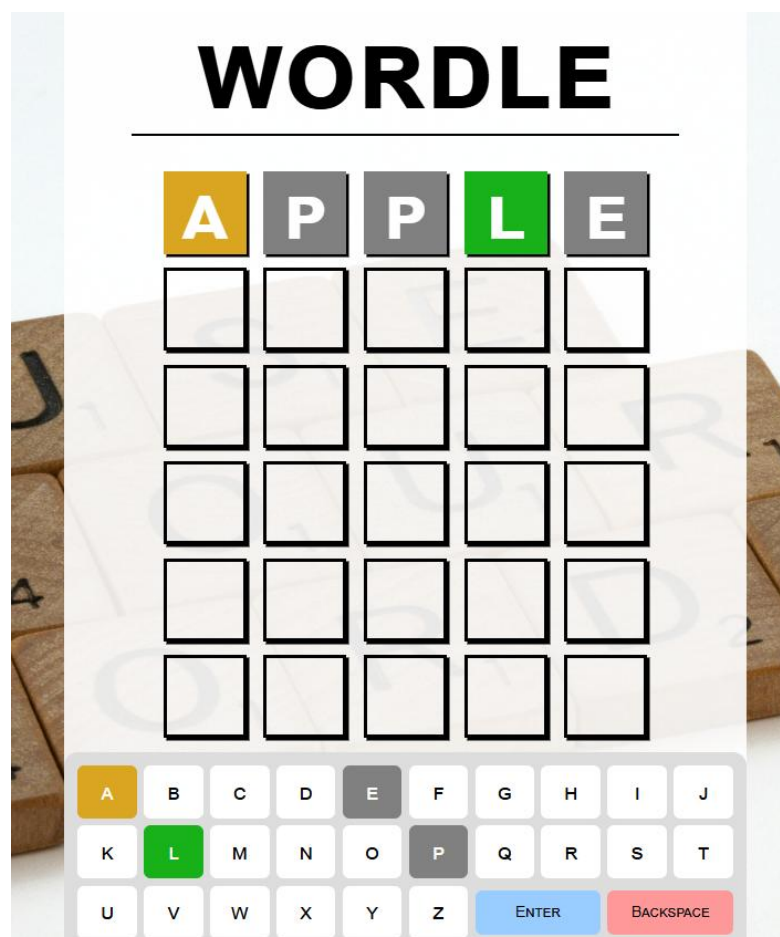
123. ábra - A játék menete szekció a kisegítő panelen

A játék elindítása

A játék indítását követően megjelenik a játéktér, amely hat sorból áll, soronként öt mezővel. Ezek a mezők a játékos által beírt betűket tartalmazzák. A játékos a billentyűzet segítségével tudja beírni a tippjeit, valamint törölni a karaktereket.

Amikor a játékos lenyomja az Enter billentyűt, a rendszer ellenőrzi a beírt szót. A szó akkor fogadható el, ha pontosan öt karakterből áll, létező angol szó, és szerepel a játék szótárában, amely körülbelül 1300 szót tartalmaz. Az ellenőrzést követően a betűk színekódolást kapnak: a szürke háttér azt jelzi, hogy a betű nem szerepel a keresett szóban, a sárga azt, hogy szerepel, de rossz helyen, míg a zöld szín a helyes betűt és pozíciót jelöli.

A játéktér alatt egy virtuális billentyűzet is található, amely segíti a játékost a már kipróbált betűk nyomon követésében. A billentyűk színe megegyezik a játéktéren megjelenő színekkel, így a játékos könnyebben tudja eldönteni, mely betűket érdemes még használni.



124. ábra - Játéktér felépítése és egy példa szemléltetése

Játék vége

Amennyiben a játékos helyesen kitalálja a szót a hat próbálkozás valamelyikében, a játék automatikusan a játék vége panelre navigál. Itt a rendszer gratulál a játékosnak. Ha a felhasználó be van jelentkezve, a rendszer rögzíti az eredményt, és növeli a teljesített játékok számát.

Ha a játékos nem találja ki a szót a hatodik próbálkozásig, a játék véget ér, és a rendszer a játék vége panelre irányítja. Itt egy „legközelebb sikerül” állapot jelenik meg, jelezve a sikertelen próbálkozást.

A játék végi panelen megjelenik a helyes megoldás, valamint – ha történt adatmentés – egy visszajelzés erről. A panelen két gomb található: a bal oldali gombbal új játék indítható, míg a jobb oldali gombbal a játékos visszatérhet a főmenübe.



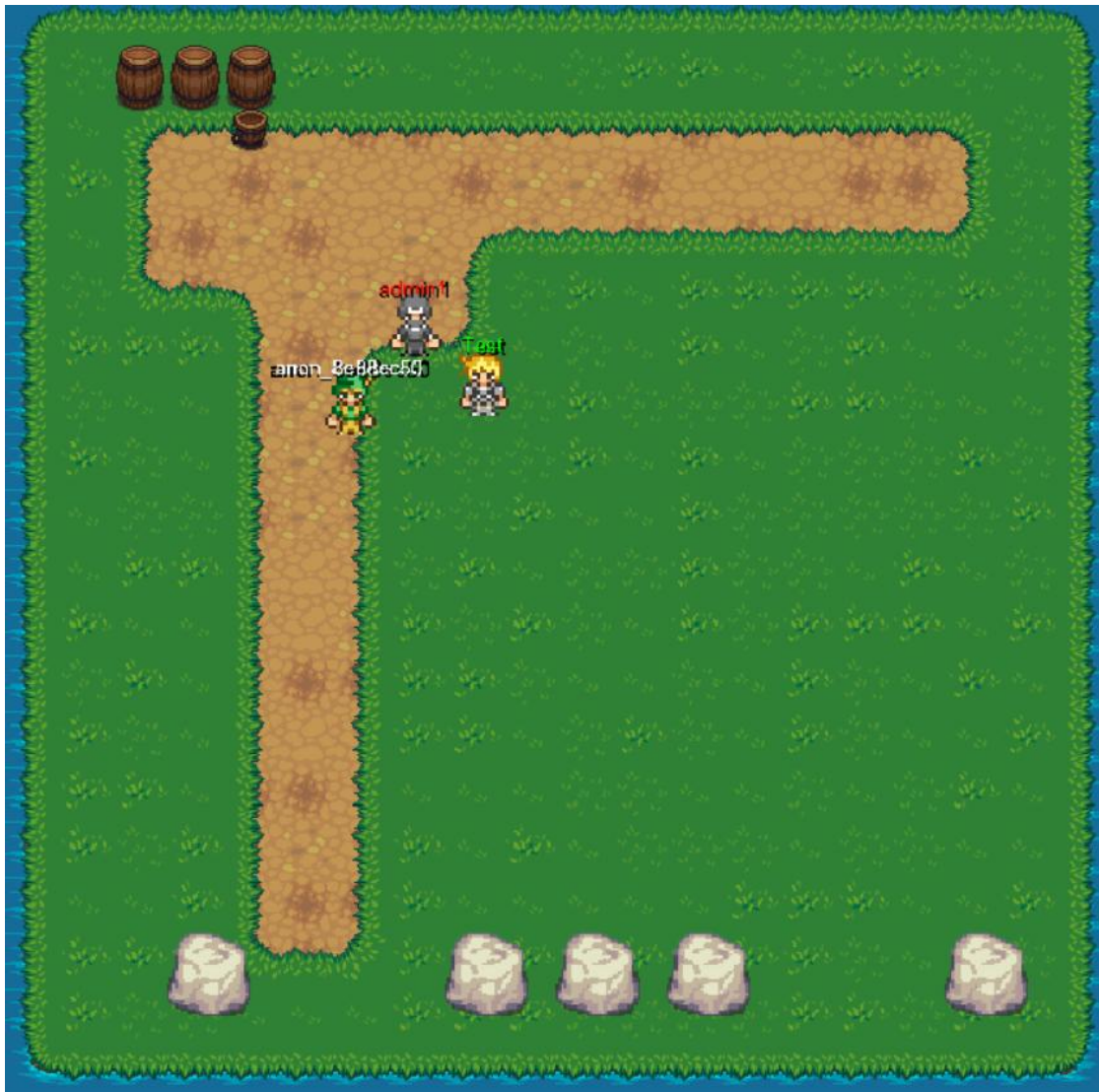
125. ábra - Sikertelen játék vége panel

Retro Realm

A játék célja és alapjai

A Retro Realm egy kétdimenziós, közösségi alapú játék, amelyben a felhasználók egy közös pályán mozoghatnak, beszélgethetnek és megjeleníthetik saját karakterüket. A játék elsődleges célja nem a verseny, hanem a közösségi interakció, vagyis a játékosok közötti kommunikáció és jelenlét megteremtése.

Ez a tartalom kizárólag nagy kijelzőkön jelenik meg, és minimum 1024 képpont szélességű megjelenítést igényel. Kisebb felbontású eszközökön (például tableteken vagy mobiltelefonokon) a tartalom nem érhető el vagy eltérő módon jelenhet meg a felhasználói élmény optimalizálása érdekében.



126. ábra - A játék pályája játékosokkal

Karakterek és megjelenés

A játékosok saját avatárjukkal és felhasználónevükkel jelennek meg a pályán. Amennyiben a felhasználó nincs bejelentkezve, egy „anon” kezdetű, véletlenszerűen generált nevet kap, és az alapértelmezett „baseCharacter” kinézetet használja. A játékos neve a jogosultsági szinttől függően különböző színnel jelenik meg: fehér szín jelzi a nem bejelentkezett felhasználót, zöld a normál felhasználót, míg piros az adminisztratori jogosultsággal rendelkező játékost.



127. ábra - Több játékos különböző névszínekkel

Mozgás és irányítás

A karakter irányítása a „W”, „A”, „S”, „D” billentyűkkel, illetve a nyílbillentyűkkel történik. A játékos szabadon mozoghat a pályán belül, azonban nem hagyhatja el annak határait. Emellett a karakter nem tud áthaladni tereptárgyakon vagy más játékosokon, így a mozgás során figyelembe kell venni a környezetet és a többi felhasználó helyzetét.

Üzenetküldés

A kommunikáció a játék egyik központi eleme. Az üzenetírást a „T” billentyű lenyomásával lehet megkezdeni, amely megnyit egy szövegbeviteli panelt. Ebben az állapotban a játékos mozgása letiltásra kerül. Az üzenet maximális hossza 20 karakter lehet. A felhasználó az „Escape” billentyű lenyomásával kiléphet az írásból anélkül, hogy elküldené az üzenetet, míg az „Enter” billentyűvel elküldheti azt.



128. ábra - Üzenet írása

Üzenetek megjelenése

Az elküldött üzenet a karakter feje felett jelenik meg, így a többi játékos számára is láthatóvá válik. Az üzenetek ideiglenesen jelennek meg, majd automatikusan eltűnnek, így a képernyő nem telítődik túl információval.



129. ábra - Játékos felett megjelenő szöveg buborék

Többjátékos működés

A játék valós idejű többjátékos élményt biztosít, ahol a felhasználók egyszerre vannak jelen ugyanazon a pályán. A rendszer folyamatosan frissíti a játékosok pozícióját és üzeneteit, így minden résztvevő valós időben láthatja a többiek mozgását és kommunikációját.



130. ábra - Több játékos beszélget

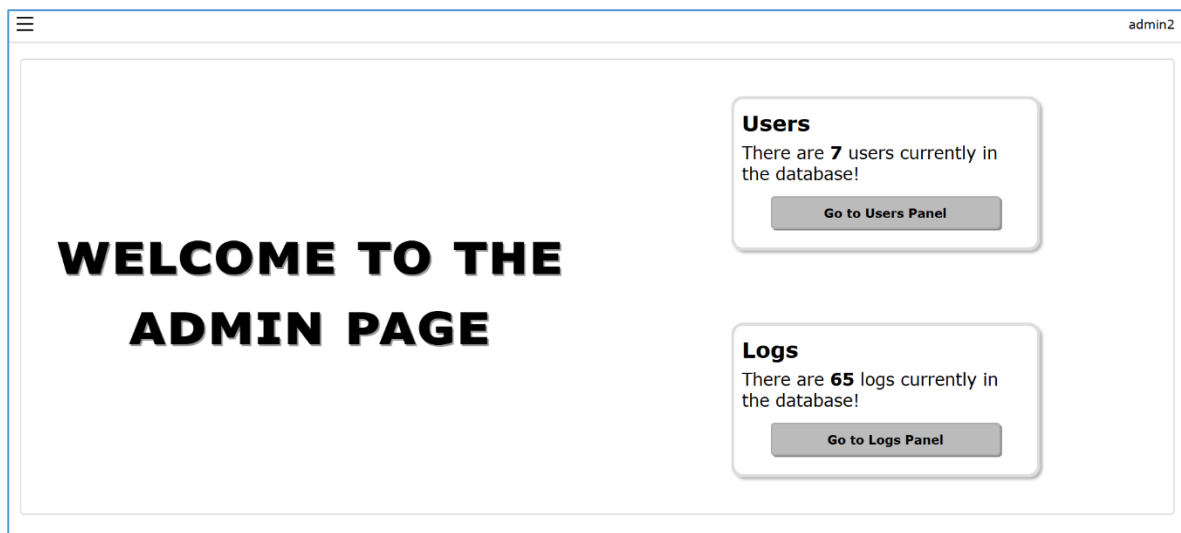
5.2 Az oldalak felhasználói dokumentációja

Admin Panel

Az adminisztrációs felület célja, hogy a rendszer üzemeltetői átfogó képet kapjanak a weboldal működéséről, valamint megtekintsék a felhasználói adatokat és a rendszerben keletkező naplóbejegyzéseket. A felület letisztult, könnyen navigálható kialakítással rendelkezik, így az adminisztrátor gyorsan hozzáférhet a számára fontos információkhoz. Az oldal teljes mértékben reszponzív kialakítású, így különböző képernyőméretek – asztali gépen, tableten és kisebb felbontású eszközökön – is megfelelően jelenik meg és használható marad.

Kezdőoldal és alapinformációk

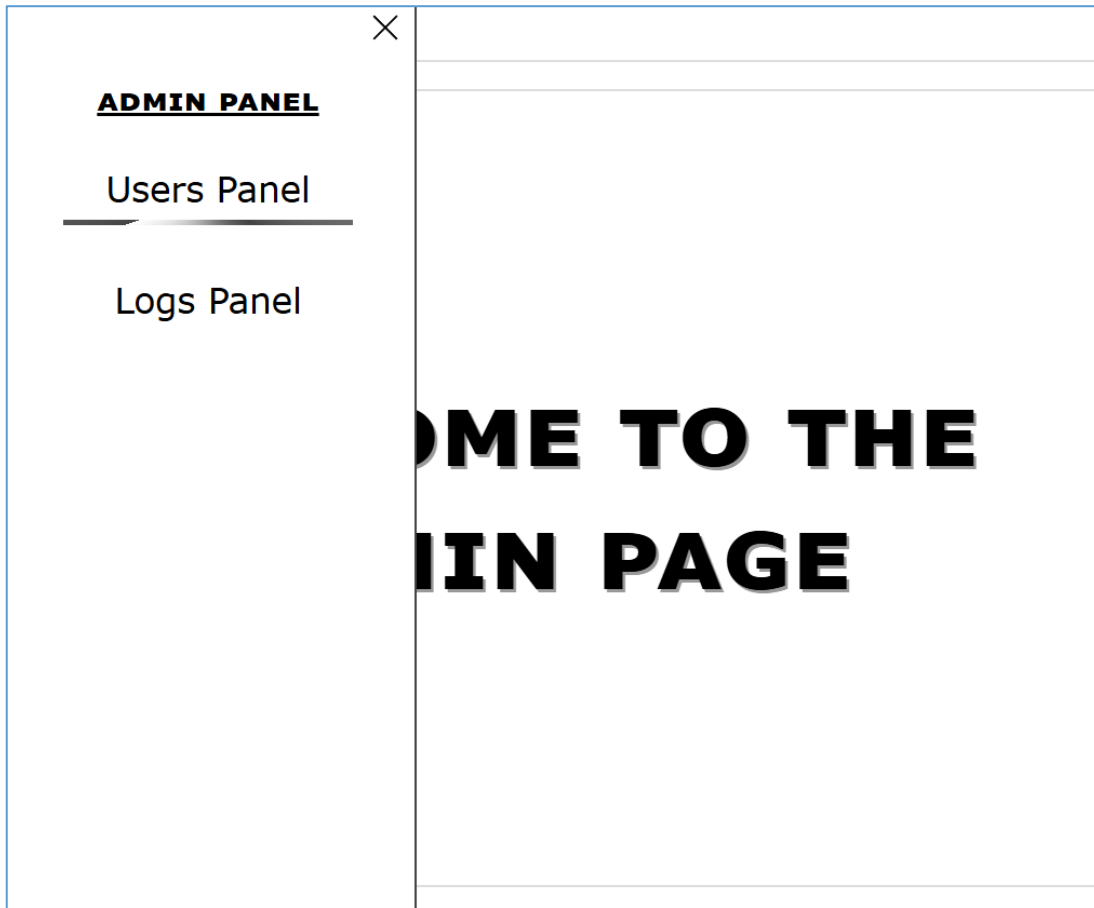
Az admin panel megnyitását követően a felhasználó egy kezdőoldalra érkezik, ahol az oldal köszönti az adminisztrátort. A jobb felső sarokban megjelenik az aktuálisan bejelentkezett adminisztrátor neve, amely visszajelzést ad a sikeres hitelesítésről. A központi részen két információs panel található, amelyek gyors áttekintést nyújtanak a rendszer állapotáról: az egyik a regisztrált felhasználók számát, míg a másik a naplóbejegyzések számát jeleníti meg. Ezek az adatok folyamatosan frissülnek, így az adminisztrátor mindig aktuális információkat lát.



131. ábra - Admin Panel kezdőlapja

Navigáció

A navigáció a bal felső sarokban található hamburger ikon segítségével történik. Ennek megnyomásával egy oldalsó navigációs panel jelenik meg, amely lehetővé teszi az adminisztrátor számára az egyes aloldalak közötti váltást. A rendszer kialakítása során fontos szempont volt, hogy a különböző funkciók gyorsan és egyszerűen elérhetők legyenek, ezért az aloldalak nemcsak a navigációs panelből, hanem a kezdőoldalon elhelyezett gyorsgombok segítségével is megnyithatók.

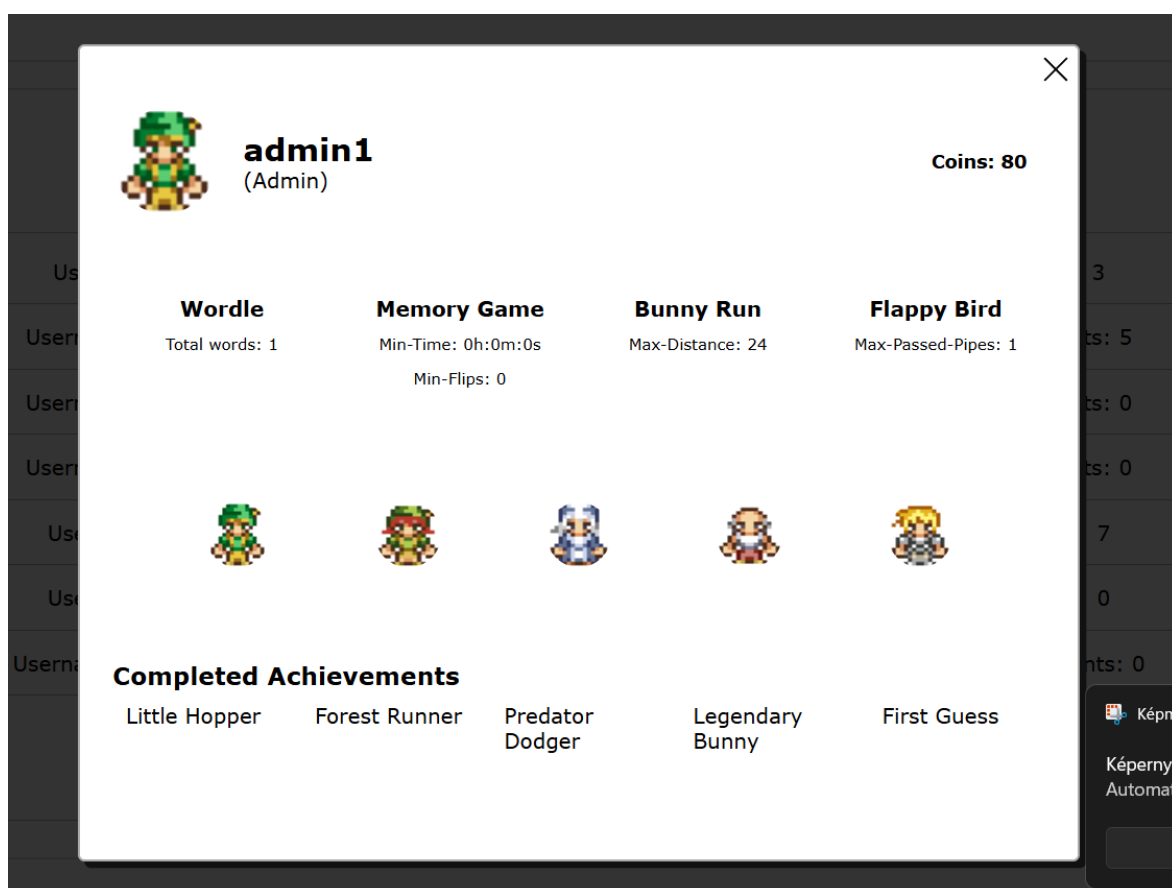


132. ábra - Navigációs panel

Felhasználók megtekintése

A felhasználók megtekintése egy külön panelen történik, ahol az adminisztrátor áttekintheti a rendszerben regisztrált összes felhasználót. A lista minden elemében alapvető információk jelennek meg, mint például a felhasználó neve, jogosultsági szintje, az aktuálisan használt avatar neve, valamint a teljesített kihívások száma. Ez a nézet gyors áttekintést biztosít a felhasználók állapotáról és aktivitásáról.

Amennyiben az adminisztrátor egy adott felhasználóra kattint, egy részletes információs panel nyílik meg. Ebben a panelben további adatok jelennek meg a kiválasztott felhasználóról, például a felhasználó neve és jogosultsága, az aktuálisan használt avatar képe, a rendelkezésre álló coin mennyiség, valamint a különböző játékokban elért eredmények. Ezen kívül megtekinthetők a felhasználó által megszerzett avatarok képei és a teljesített kihívások nevei is. A részletező panel a jobb felső sarokban található bezáró gomb segítségével zárható be.



133. ábra - Részletes felhasználói adatlap

Naplóbejegyzések megtekintése

A naplóbejegyzések megtekintése szintén egy külön panelen keresztül érhető el, ahol az adminisztrátor a rendszer működése során keletkezett eseményeket tekintheti át. A listában minden bejegyzéshez megjelenik annak típusa, rövid leírása, valamint az esemény időpontja, amely segíti az események időrendben történő követését.

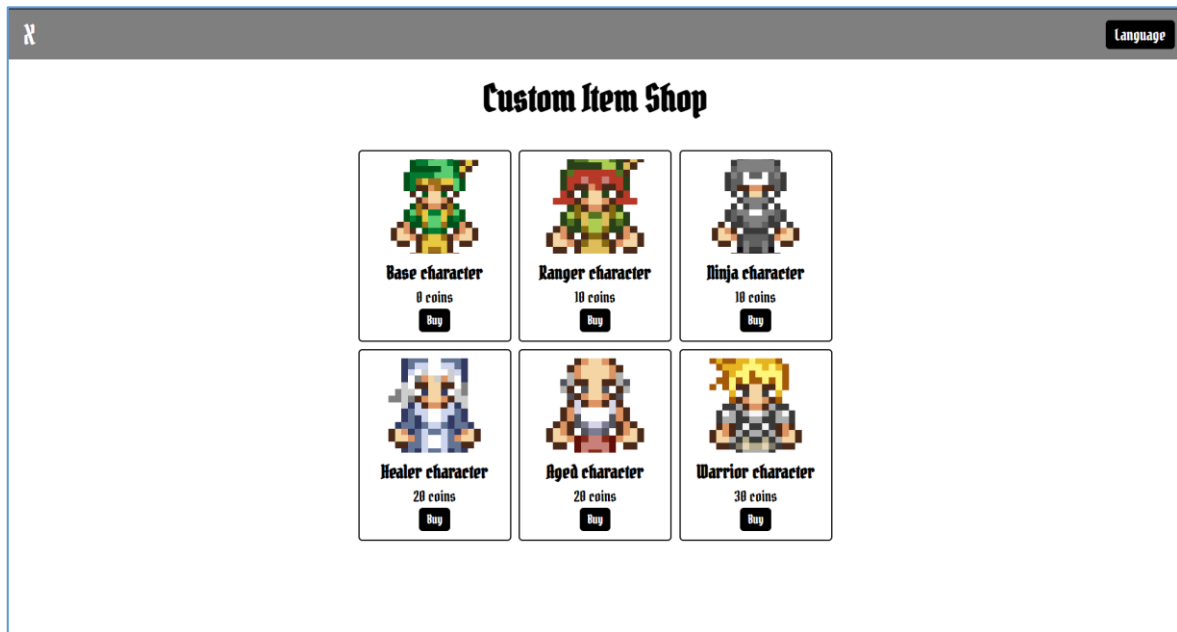
Egy adott naplóbejegyzés kiválasztásakor egy részletes nézet nyílik meg, amely további információkat tartalmaz. Itt látható a bejegyzés típusa, a pontos időpont, a hozzá tartozó leírás, valamint az esetleges hibaüzenet. Amennyiben a naplóbejegyzés egy felhasználóhoz kapcsolódik, annak neve is megjelenik, ellenkező esetben a rendszer „Unknown user” megnevezést használ. A részletes panel bezárása szintén a jobb felső sarokban található gomb segítségével történik.

LogType Update	DateTime 2026-04- 06T23:37:27.3716745	User Name test2
Description WordleStatus has been updated for User-5!		
Error		

134. ábra - Részletes naplóbejegyzés adatlap

Custom item shop

A Custom Item Shop (Egyedi tárgybolt) oldal célja, hogy a felhasználók különböző avatárokat vásárolhassanak a játékban megszerzett coinok felhasználásával.



135. ábra - A Custom Item Shop teljes felülete megvásárolható avatárokkal.

Az oldal lehetőséget biztosít avatárok megtekintésére, azok árának megjelenítésére, valamint a kiválasztott avatárok megvásárlására. Emellett a felhasználó nyelvet is válthat, így a felület több nyelven is használható.

Az oldal felépítése

A felső sáv két fő elemet tartalmaz: egy kilépés gombot (X), amely visszairányítja a felhasználót a főoldalra, valamint egy Language gombot, amely megnyitja a nyelvválasztó menüt. A felhasználó három különböző nyelv közül választhat: angol, magyar és spanyol. A kiválasztott nyelv elmentésre kerül a böngészőben, és azonnal frissíti az oldalon megjelenő szövegeket, például az avatárok neveit és a gombok feliratait.

Terméklista (Avatárok)

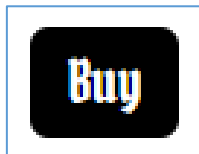
Az oldalon az összes megvásárolható avatár egy listában jelenik meg. Minden avatár egy külön kártyán látható, amely tartalmazza az avatár képét, nevét, árát coinban megadva, valamint egy vásárlás gombot.



136. ábra - Megvásárolható avatárok listája képpel, névvel és árral.

Vásárlás folyamata

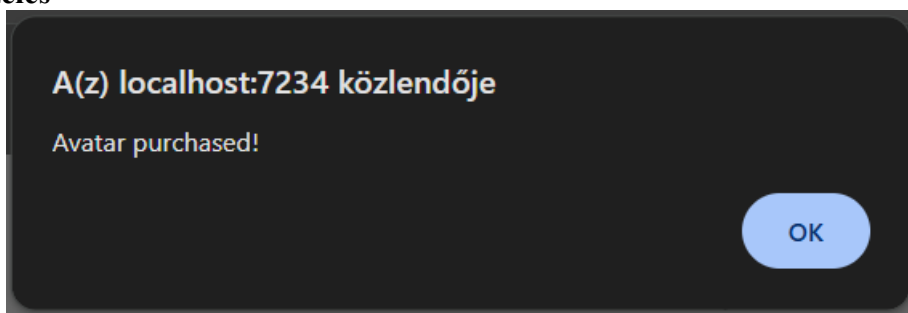
A felhasználó a kiválasztott avatar alatt található „Buy” (Vásárlás) gombra kattintva indíthatja el a vásárlási folyamatot. Ekkor a rendszer elküldi a szükséges adatokat a szervernek.



137. ábra - Vásárlás gomb egy avatar alatt.

A vásárlás eredményéről a rendszer egy felugró üzenetben (alert) ad visszajelzést. Ez lehet sikeres vásárlás esetén egy megerősítő üzenet, vagy hiba esetén egy tájékoztató szöveg, például ha a felhasználó nincs bejelentkezve vagy nem rendelkezik elegendő coin mennyiséggel.

Nyelvkezelés



138. ábra - Vásárlás utáni visszajelző üzenet.

Az oldal többnyelvű működést támogat, amely során nemcsak a kezelőfelület elemei, hanem az avatarok nevei és a vásárlás gomb szövege is a kiválasztott nyelvnek megfelelően jelenik meg.

Működés technikai háttere

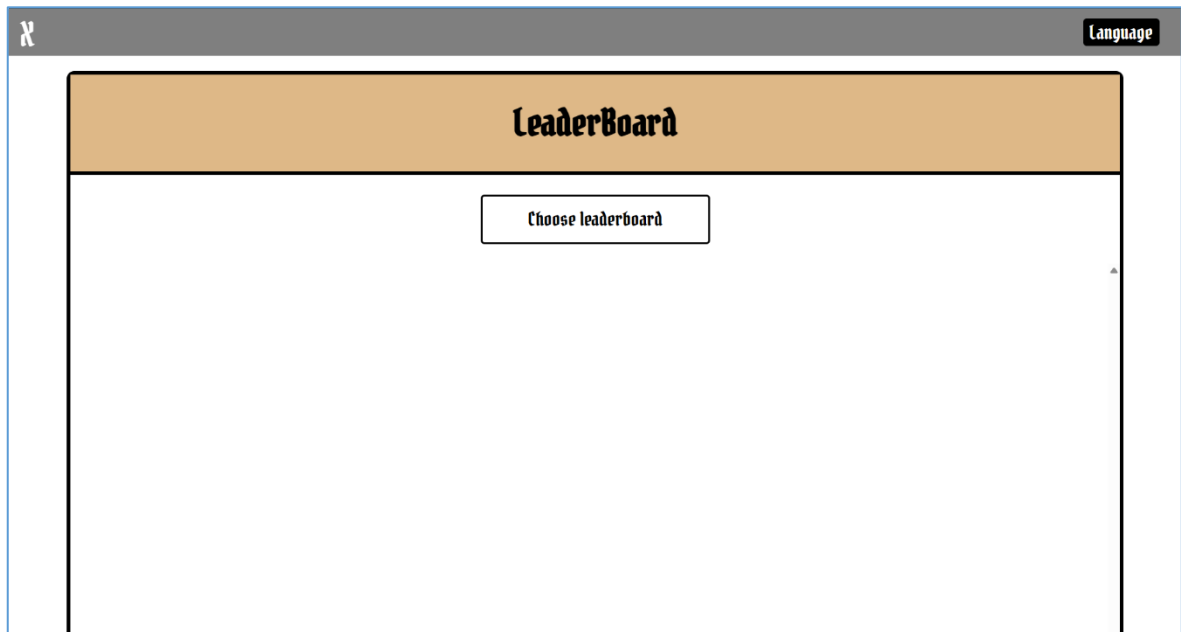
Az avatárok betöltése egy API híváson keresztül történik, amely az összes elérhető avatárt visszaadja a kliens számára. Ezek az adatok tartalmazzák az avatár nevét és árát. A vásárlás egy POST kérés segítségével történik, amely elküldi a kiválasztott avatár nevét a szervernek. A szerver ezután feldolgozza a kérést, és visszajelzést ad annak eredményéről. A rendszer ellenőrzi, hogy a felhasználó be van-e jelentkezve. Vásárlás előtt frissíti a hozzáférési tokent, amelyet a sessionStorage-ból olvas ki, így biztosítva a biztonságos kommunikációt a szerverrel.

Felhasználói folyamat

Az oldal betöltése után a rendszer lekéri az avatárokat a szerverről, majd megjeleníti azokat a felhasználó számára. A felhasználó ezután kiválaszthatja a kívánt nyelvet, böngészhet az avatárok között, majd a megfelelő gombra kattintva megvásárolhatja a kiválasztott elemet. A vásárlás után azonnali visszajelzést kap az eredményről.

Leaderboard felület

A LeaderBoard oldal célja, hogy a felhasználók megtekintsék a különböző játékok legjobb eredményeit, és összehasonlítsák saját teljesítményüket más játékosokéval.



139. ábra - A ranglista oldal teljes felülete különböző játékok eredményeivel.

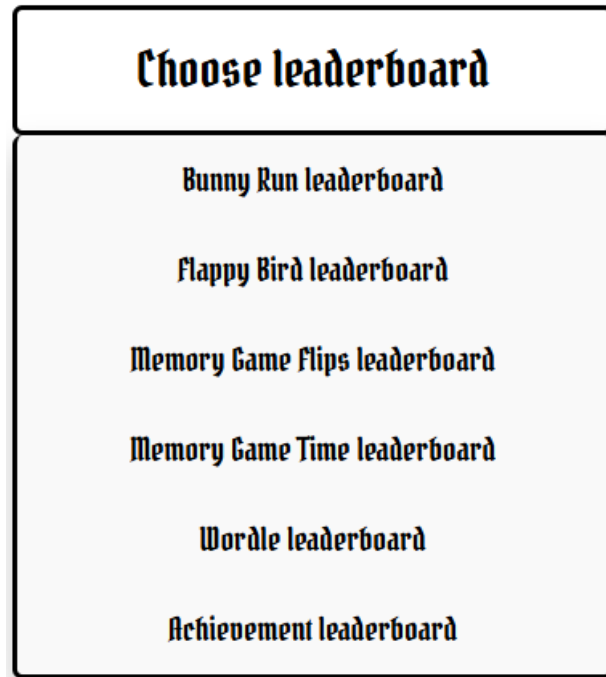
Az oldal felépítése

A felső navigációs sáv tartalmaz egy kilépés gombot (X), amely visszairányítja a felhasználót a főoldalra, valamint egy Language gombot, amely segítségével a felhasználó kiválaszthatja a kívánt nyelvet.

A nyelvválasztó menü három lehetőséget kínál: angol, magyar és spanyol. A kiválasztott nyelv elmentésre kerül a böngészőben, és azonnal frissíti az oldalon megjelenő szövegeket, beleértve a címeket, gombokat és oszlopneveket is.

Ranglista kiválasztása

A felhasználó egy lenyíló menü segítségével választhat a különböző ranglisták közül. A kiválasztott ranglista neve megjelenik a gombon, és a hozzá tartozó adatok betöltődnek az oldalra.



140. ábra - Lenyíló menü a különböző ranglisták kiválasztásához.

Ranglisták megjelenítése

A kiválasztott ranglista egy táblázatszerű listában jelenik meg. Az első sor fejléc, amely megmutatja az oszlopok jelentését (például felhasználónév és eredmény), ezt követik a játékosok eredményei. Minden játékhoz egyedi ranglista érhető el, illetve egy az elért mérföldkövekhez.

Bunny Run leaderboard	
<u>Username</u>	<u>Max Distance</u>
Test	58
admin1	24
test2	18
admin2	0
admin3	0
test3	0
UserForTest	0
Ujfelhasznalo	0

141. ábra - Ranglista megjelenítése felhasználónévvel és eredményekkel.

Működés technikai háttere

Az oldal több különböző API végpontról tölti be az adatokat, attól függően, hogy melyik ranglistát választja ki a felhasználó. Minden ranglista külön lekéréssel érkezik a szerverről. A kiválasztott ranglista adatai dinamikusan kerülnek be az oldalba, és minden váltáskor frissülnek. A fejléc és az adatok a kiválasztott típusnak megfelelően jelennek meg. Az oldal minden szöveges eleme több nyelven is elérhető, beleértve a ranglisták neveit, az oszlopneveket és a gombok feliratait.

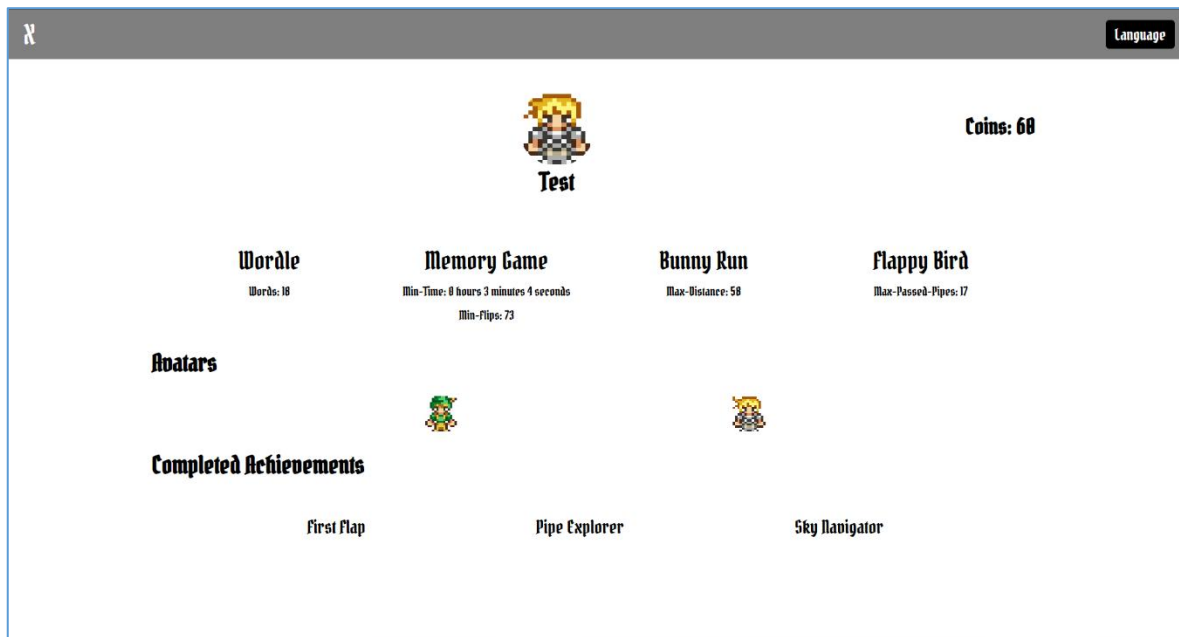
Felhasználói folyamat

Az oldal betöltése után a felhasználó kiválaszthatja a kívánt ranglistát a lenyíló menüből. Ezt követően megjelennek az adott játékhoz tartozó eredmények. A felhasználó bármikor válthat másik ranglistára, illetve megváltoztathatja a felület nyelvét is.

Profile oldal

A profil oldal célja, hogy a felhasználó számára egy áttekinthető felületet biztosítson saját adatai, statisztikái, avatárjai és elért mérföldköveit megtekintésére és kezelésére.

Az oldal dinamikusan tölti be az adatokat egy backend API segítségével, és lehetőséget biztosít a felhasználói adatok megjelenítésére, az avatár kiválasztására, a játékstatisztikák megtekintésére, valamint a nyelv váltására.



142. ábra - A profil oldal teljes felülete felhasználói adatokkal és statisztikákkal.

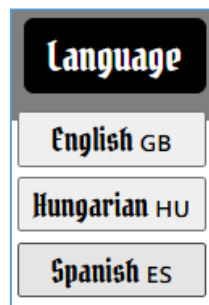
Az oldal felépítése

A felső sáv két fő elemet tartalmaz: a kilépés gombot (X), amely visszairányítja a felhasználót a főoldalra, valamint a Language gombot, amely megnyitja a nyelvválasztó menüt.



143. ábra - Navigációs sáv kilépés és nyelvválasztás lehetőséggel.

A felhasználó három nyelv közül választhat: angol (English), magyar (Hungarian) és spanyol (Spanish). A felhasználó három nyelv közül választhat:

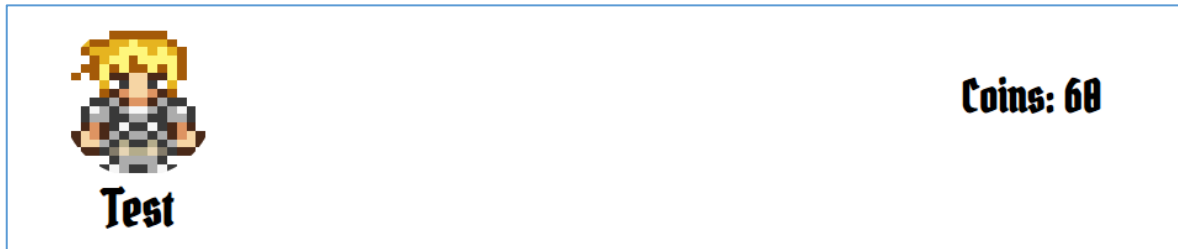


144. ábra - Nyelvválasztó menü három választható nyelvvel.

A kiválasztott nyelv elmentésre kerül a böngészőben, valamint azonnal frissíti a felület szövegeit az új nyelvi beállításnak megfelelően.

Felhasználói adatok megjelenítése

A profil oldal a felhasználó több alapadatát jeleníti meg, valamint részletes játékstatisztikákat is biztosít számára. Az alapadatok között szerepel a profilkép (az aktuálisan kiválasztott avatár), a felhasználónév, valamint a coin egyenleg, amely a játékban megszerzett fizetőeszközt mutatja.



145. ábra - Felhasználói alapadatok: profilkép, név és coin egyenleg.

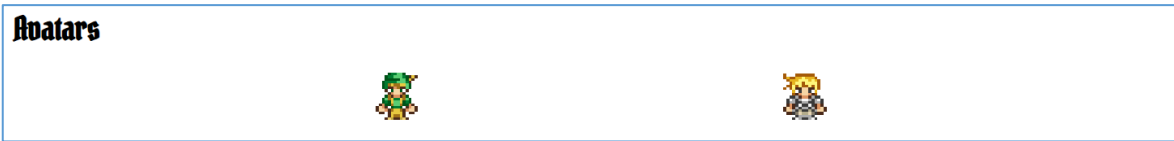
A játékstatisztikák részben a rendszer több különböző játék eredményeit jeleníti meg. A Wordle esetében a megoldott szavak száma látható. A Memory Game-nél a legjobb idő (óra, perc, másodperc bontásban), valamint a legkevesebb fordítás szám (flips) kerül megjelenítésre. A Bunny Run játék statisztikája a maximálisan megtett távolságot mutatja, míg a Flappy Bird esetében a maximálisan átjutott csövek száma jelenik meg.

Wordle	Memory Game	Bunny Run	Flappy Bird
Words: 18	Min-Time: 0 hours 3 minutes 4 seconds Min-Flips: 73	Max-Distance: 58	Max-Passed-Pipes: 17

146. ábra - Játékstatisztikák különböző játékokhoz.

Avatarok kezelése

A felhasználó összes megszerzett avatárja képként jelenik meg az oldalon, így könnyen áttekinthetővé válik a rendelkezésre álló karakterek listája.

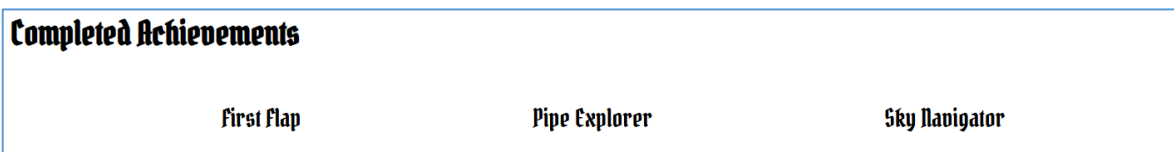


147. ábra - Megvásárolt avatárok, amelyek közül a felhasználó választhat.

Az avatár váltás során egy adott avatárra kattintva az azonnal kiválasztásra kerül, a kiválasztás pedig elküldésre kerül a szerver felé. Ezt követően az oldal automatikusan frissül, és a kiválasztott avatár lesz látható a profilban.

Mérföldkövek (Achievements)

Az eredmények (Achievements) részben a felhasználó által megszerzett mérföldkövek kerülnek listázásra, ahol minden elért achievement neve külön sorban jelenik meg, ezzel biztosítva az átlátható megjelenítést.



149. ábra - A felhasználó által elért eredmények listája.

Működés technikai háttere

A működés technikai háttere több lépésből áll, amelyek biztosítják az adatok helyes betöltését és az oldal dinamikus működését. Az adatlekérés két lépésben történik. Először a rendszer lekéri a felhasználó nevét a `GetUserName` végponton keresztül, majd ezt követően a teljes felhasználói adatokat a `GetUserData` végpontról. Amennyiben a lekérés sikertelen, a felhasználó automatikusan visszairányításra kerül a főoldalra. A token kezelés során a rendszer minden kérés előtt frissíti a hozzáférési tokent, amelyet a `sessionStorage`-ból olvas ki. Ez biztosítja a biztonságos és folyamatos hitelesítést a szerver felé. A nyelvkezelés a `lang.js` fájlban történik, ahol a szövegek kulcs-érték párok formájában vannak eltárolva. Minden támogatott nyelvhez külön fordítás tartozik, így az oldal dinamikusan képes váltani a kiválasztott nyelvre. Az avatár váltás API egy POST kérésen keresztül működik, amely az `/api/setCharacter` endpointot használja, és a kiválasztott avatár nevét küldi el paraméterként a szervernek. A felhasználói folyamat az oldal betöltésével kezdődik, ezt követi az adatok lekérése a szerverről, majd a profil megjelenítése. Ezt követően a felhasználó szabadon válthat nyelvet, választhat avatárt, valamint megtekintheti a statisztikáit. Végül a kilépés gomb segítségével visszatérhet a főoldalra.

Bejelentkezés Panel

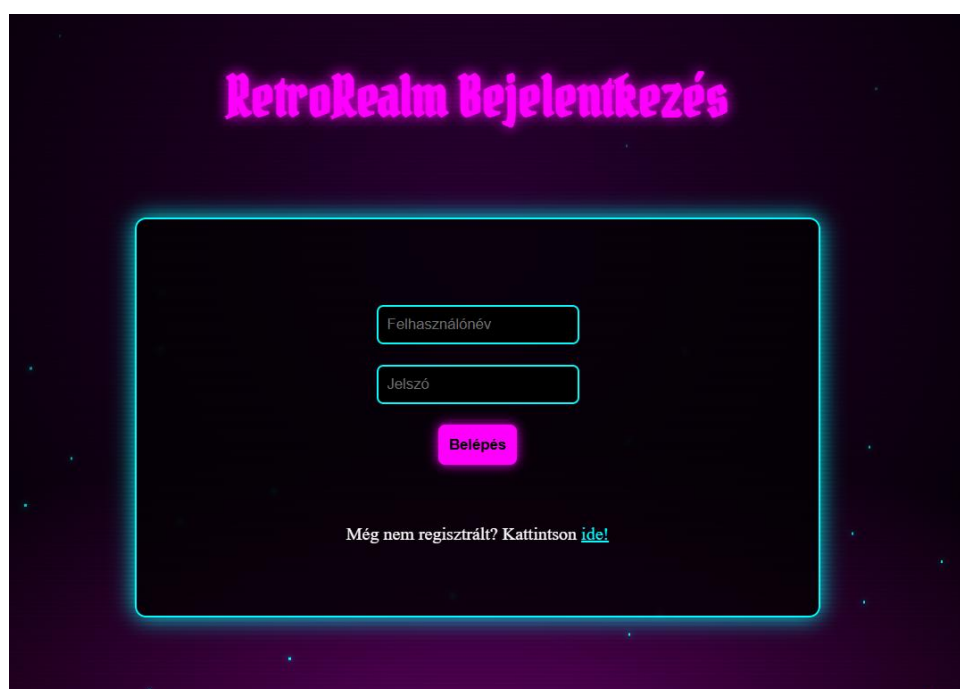
A bejelentkezési oldal célja, hogy a felhasználók biztonságosan hozzáférjenek az alkalmazás funkcióihoz. A rendszer lehetővé teszi a már regisztrált felhasználók számára, hogy azonosítsák magukat, és elérjék a személyre szabott tartalmakat, például a profiljukat, eredményeiket vagy egyéb funkciókat.

Az oldal felépítése

Az oldal központi részén található a bejelentkezési űrlap, amely egy letisztult, könnyen átlátható felületen jelenik meg. A háttérben animált elemek (részecskék) biztosítják a modern megjelenést, de nem zavarják a használatot. Az oldal tetején található egy nyelvválasztó gomb, amely lehetővé teszi a felhasználó számára a felület nyelvének módosítását.

A nyelvválasztó menü három lehetőséget kínál: angol, magyar és spanyol. A kiválasztott nyelv mentésre kerül a böngészőben, így az oldal újratöltése után is megmarad. A nyelv módosítása azonnal frissíti az összes szöveges elemet, beleértve a mezők helyőrző szövegét, a gombokat és az üzeneteket.

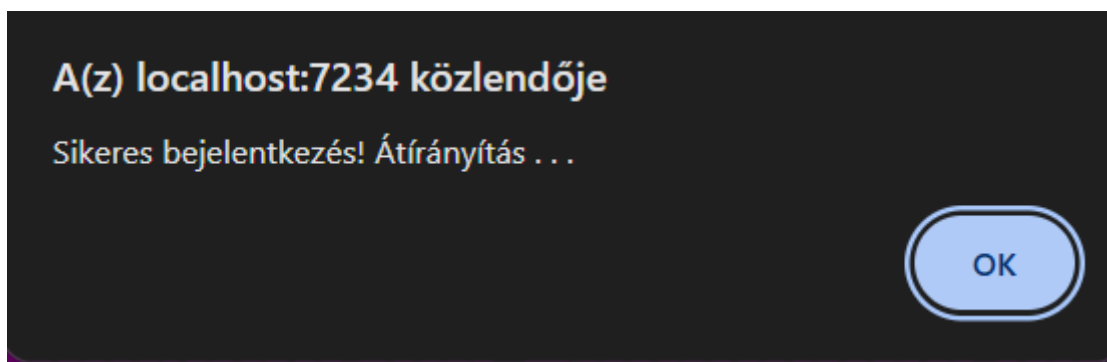
A felhasználónév és jelszó megadására két külön input mező szolgál. Ezek alatt található a bejelentkezés gomb, amely elindítja a hitelesítési folyamatot. Az oldalon emellett egy hivatkozás is található, amely a regisztrációs oldalra irányítja az új felhasználókat.



150. ábra - Bejelentkezés panel megjelenése

Működés technikai háttere

A bejelentkezés során a rendszer a felhasználó által megadott adatokat egy API végpontra küldi el POST kérés formájában. Az adatok JSON formátumban kerülnek továbbításra a szerver felé. A szerver ellenőrzi a megadott adatokat, majd visszaküld egy választ a kliensnek. Sikeres bejelentkezés esetén a szerver egy úgynevezett tokenet ad vissza, amely a felhasználó azonosítására szolgál. Ez a token a böngésző sessionStorage tárhelyében kerül eltárolásra, és a későbbi kérések során biztosítja, hogy a felhasználó hitelesített maradjon. Amennyiben a megadott adatok hibásak, vagy a szerver nem tudja feldolgozni a kérést, a rendszer egy sikertelen állapotot jelez vissza. Ilyenkor a felhasználó hibaüzenetet kap, és lehetősége van az adatok újbóli megadására.



151. ábra - Sikeres bejelentkezés felhívás

Felhasználói folyamat

Az oldal megnyitásakor a felhasználó egy azonnal használható felületet kap, ahol szükség esetén először kiválaszthatja a számára megfelelő nyelvet. Ezt követően a bejelentkezési űrlapon megadja a felhasználónevét és jelszavát.

A bejelentkezési gomb megnyomásával elindul a hitelesítési folyamat, amely során az adatok ellenőrzésre kerülnek a szerveren. Amennyiben az adatok helyesek, a rendszer sikeres belépést jelez, majd automatikusan továbbirányítja a felhasználót a főoldalra.

Ha a megadott adatok nem megfelelőek, a rendszer erről egyértelmű visszajelzést ad, így a felhasználó javíthatja az adatokat és újra próbálkozhat. A korábban kiválasztott nyelv minden esetben megmarad, így a későbbi használat során nem szükséges újra beállítani.

Regisztráció Panel

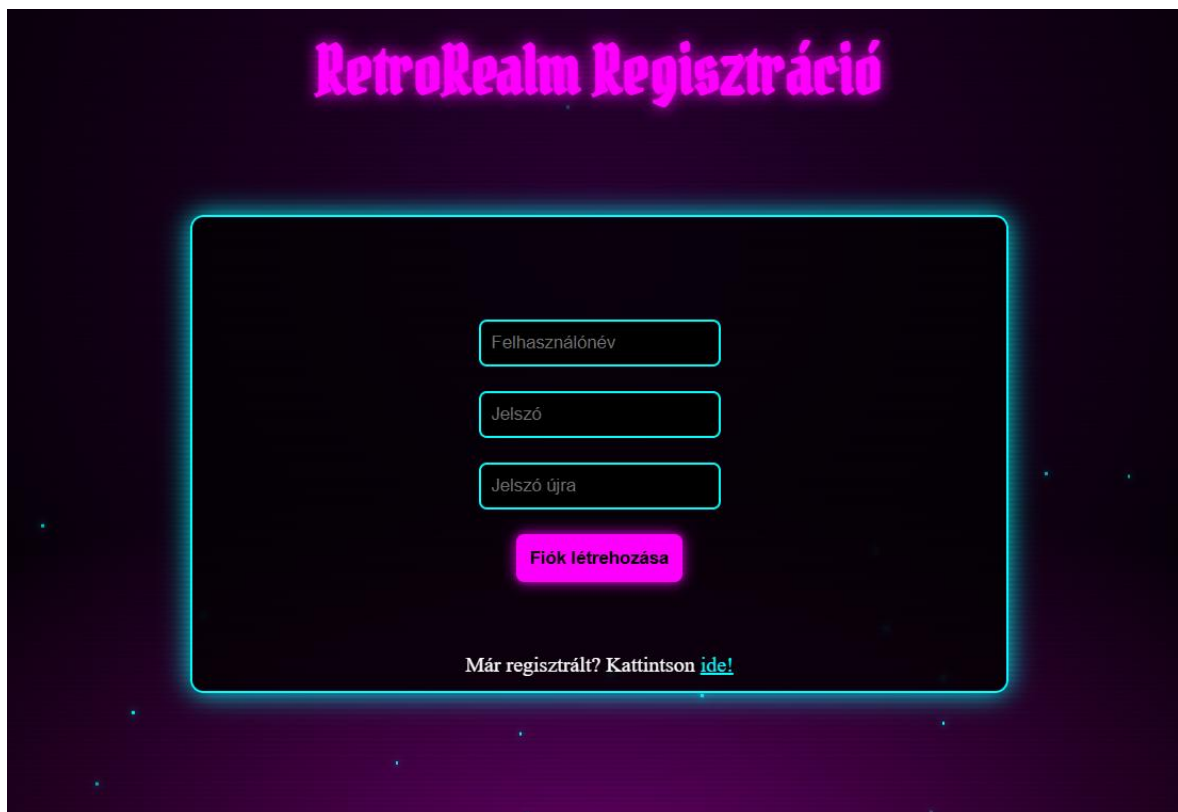
A regisztrációs oldal célja, hogy az új felhasználók létrehozassanak egy saját fiókot az alkalmazás használatához. A folyamat egyszerű és gyors, lehetővé téve, hogy a felhasználók néhány lépésben hozzáférjenek a rendszer funkcióihoz.

Az oldal felépítése

Az oldal központi részén egy regisztrációs űrlap található, amely letisztult és könnyen kezelhető felületen jelenik meg. A háttérben animált elemek (részecskék) biztosítják az egységes vizuális megjelenést. Az oldal tetején egy nyelvválasztó gomb helyezkedik el, amely lehetővé teszi a felület nyelvének módosítását.

A felhasználó három nyelv közül választhat: angol, magyar és spanyol. A kiválasztott nyelv mentésre kerül a böngészőben, így a későbbi látogatások során is automatikusan betöltődik. A nyelvváltás azonnal frissíti az oldal összes szöveges elemét.

A regisztrációhoz három mező áll rendelkezésre: felhasználónév, jelszó és jelszó megerősítése. Ezek alatt található a regisztráció gomb, amely elindítja a folyamatot. Az oldalon egy hivatkozás is található, amely visszairányít a bejelentkezési oldalra.

A screenshot of the RetroRealm registration panel. The title "RetroRealm Regisztráció" is displayed in a stylized, glowing pink font at the top. Below it, a central white rectangular box with a glowing blue border contains the registration form. The form consists of three input fields: "Felhasználónév", "Jelszó", and "Jelszó újra", each with a blue border. Below these fields is a pink button labeled "Fiók létrehozása". At the bottom of the white box, there is a link that says "Már regisztrált? Kattintson [ide!](#)". The background of the entire page is dark purple with a subtle pattern of small, glowing blue dots.

152. ábra - Regisztrációs panel

Működés technikai háttere

A rendszer a megadott adatokat egy API végpontra küldi el POST kérés formájában. Az adatok JSON formátumban kerülnek továbbításra a szerver felé, ahol feldolgozásra és ellenőrzésre kerülnek.

A regisztráció előtt a rendszer ellenőrzi, hogy a két megadott jelszó megegyezik-e. Ez a lépés csökkenti a hibás kérések számát és javítja a felhasználói élményt.

A szerver válasza egy státuszkód formájában érkezik vissza. Sikeres regisztráció esetén a rendszer ezt felismeri, és ennek megfelelően kezeli a további lépéseket.

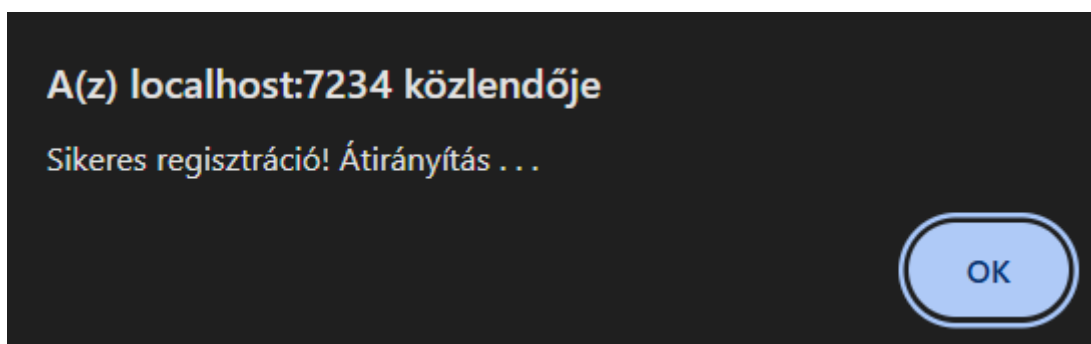
Felhasználói folyamat

Az oldal megnyitása után a felhasználó kiválaszthatja a kívánt nyelvet, ha szükséges. Ezt követően megadja a felhasználónevét, majd beírja a jelszót és annak megerősítését.

A regisztráció gombra kattintva a rendszer először ellenőrzi, hogy a két jelszó egyezik-e. Ha nem, akkor egy hibaüzenet jelenik meg, és a folyamat nem folytatódik.

Amennyiben az adatok megfelelőek, a rendszer elküldi azokat a szervernek. Sikeres regisztráció esetén visszajelzést ad, majd automatikusan átirányítja a felhasználót a bejelentkezési oldalra.

Hiba esetén a rendszer jelzi a sikertelen regisztrációt, így a felhasználó javíthatja az adatokat és újra próbálkozhat. A kiválasztott nyelv ebben az esetben is megmarad.



153. ábra - Sikeres regisztrációt követő válasz

Landing Page

A főoldal (landing page) az alkalmazás központi felülete, amely a felhasználók számára kiindulópontként szolgál a különböző funkciók és játékok eléréséhez. Az oldal dinamikusan alkalmazkodik a felhasználó bejelentkezési állapotához és jogosultságaihoz, így személyre szabott élményt biztosít.

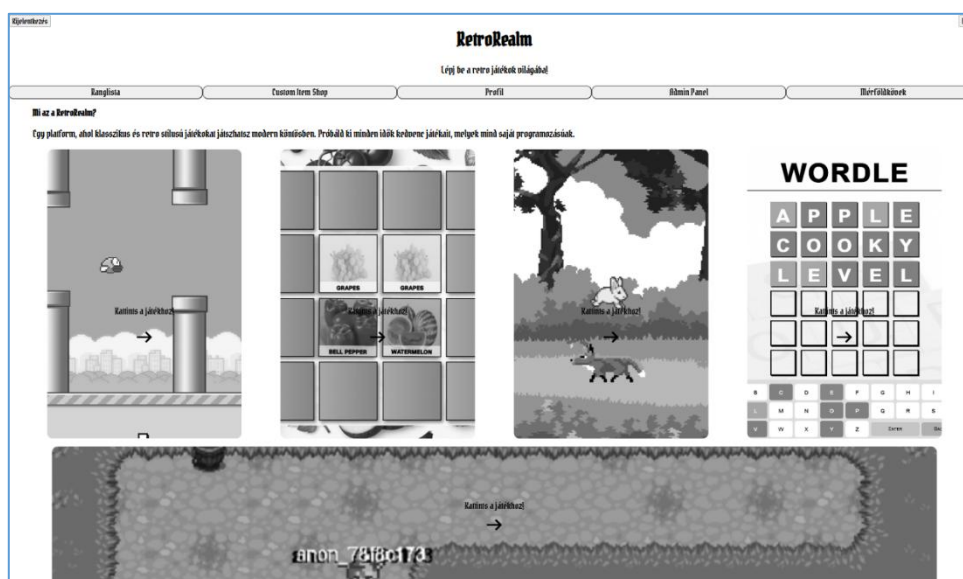
Az oldal felépítése

Az oldal felső részén található a nyelvválasztó gomb, amely lehetővé teszi a felület nyelvének módosítását. Emellett itt helyezkedik el a bejelentkezés/kijelentkezés gomb is, amely a felhasználó aktuális állapotától függően változik. Bejelentkezett felhasználó esetén kijelentkezési lehetőséget kínál, míg vendégként a login oldalra irányít.

A rendszer három nyelvet támogat: angol, magyar és spanyol. A kiválasztott nyelv mentésre kerül a böngészőben, így az oldal újratöltése után is megmarad. A nyelv módosítása azonnal frissíti az összes szöveges elemet, beleértve a gombokat, leírásokat és címeket.

A főoldalon különböző gombok találhatók, amelyek az alkalmazás egyes részeire navigálnak. Ide tartoznak például a játékok (pl. Flappy Bird, memóriajáték), a ranglista, a mérföldkövek (achievements), a profil oldal, valamint a bolt és a Retro Realm oldal.

Bizonyos elemek – például a profil gomb vagy az admin panel – csak akkor jelennek meg, ha a felhasználó rendelkezik a megfelelő jogosultsággal. Ez biztosítja, hogy mindenki csak a számára releváns funkciókat lássa.



154. ábra - Főoldal felépítése adminisztrátori jogkörrel

Működés technikai háttere

Az oldal betöltésekor a rendszer ellenőrzi, hogy a felhasználó rendelkezik-e érvényes bejelentkezési tokennel. Amennyiben igen, egy szerveroldali ellenőrzés történik, amely visszaadja a felhasználó jogosultsági szintjét.

A rendszer három fő állapotot különböztet meg:

- nincs bejelentkezve
- normál felhasználó
- adminisztrátor

Ezen státuszok alapján jelennek meg vagy tűnnek el bizonyos elemek (pl. admin panel csak admin esetén látható).

A háttérben a rendszer szükség esetén frissíti a felhasználó hitelesítési tokenjét, így biztosítva a folyamatos és biztonságos működést anélkül, hogy a felhasználónak újra be kellene jelentkeznie.

A különböző gombok eseménykezelőkkel vannak ellátva, amelyek kattintás esetén átirányítják a felhasználót a megfelelő aloldalra. Ez egyszerű és gyors navigációt tesz lehetővé az alkalmazáson belül.

Felhasználói folyamat

Az oldal betöltése után a rendszer automatikusan ellenőrzi a felhasználó állapotát, és ennek megfelelően jeleníti meg az elérhető funkciókat. A felhasználó első lépésként kiválaszthatja a kívánt nyelvet, ha szükséges.

Ezt követően a felhasználó a különböző gombok segítségével navigálhat az alkalmazásban. Például elindíthat egy játékot, megtekintheti a ranglistát vagy beléphet a profiljába. A műveletek egyszerű kattintással elérhetők, így az oldal könnyen kezelhető.

Amennyiben a felhasználó nincs bejelentkezve, bizonyos funkciók nem érhetők el, és a rendszer a bejelentkezési oldalra irányítja. Bejelentkezett állapotban további lehetőségek válnak elérhetővé, például a profil vagy – adminisztrátor esetén – az admin panel.

A kijelentkezés gomb használatával a felhasználó bármikor kiléphet a rendszerből, amely után visszairányításra kerül a kezdőoldalra.

Achievements Panel

Az Achievement (Mérföldkövek) oldal célja, hogy a felhasználók megtekintsék az alkalmazásban elérhető kihívásokat és jutalmakat. Az oldal segítségével a játékosok nyomon követhetik, milyen feladatokat teljesíthetnek, és milyen jutalmakat szerezhetnek meg.

Az oldal felépítése

Az oldal tartalmaz egy visszalépés gombot, amely lehetővé teszi a felhasználó számára, hogy visszatérjen a főoldalra. Emellett megtalálható a nyelvválasztó gomb is, amely segítségével a felhasználó módosíthatja a felület nyelvét.

A nyelvválasztó menü három lehetőséget kínál: angol, magyar és spanyol. A kiválasztott nyelv elmentésre kerül a böngészőben, és azonnal frissíti az oldalon megjelenő szövegeket, beleértve az achievementek neveit és leírásait is.

Az oldalon az achievementek egy listában jelennek meg. Minden elem tartalmazza az adott mérföldkő nevét, a teljesítés feltételét, valamint a megszerezhető jutalmat. Az elemek jól elkülönített blokkokban jelennek meg, így könnyen átláthatók.

M

Nyelv

Mérföldkövek

Kis Ugráló Érj el 5 pontot a Bunny Run játékban! Jutalom: 10 €	Erőfi Futója Érj el 10 pontot a Bunny Run játékban! Jutalom: 20 €	Ragadozókertülő Érj el 15 pontot a Bunny Run játékban! Jutalom: 30 €	Legendás Nyúl Érj el 20 pontot a Bunny Run játékban! Jutalom: 40 €
Első Szárnyasapás Érj el 5 pontot a FlappyBird játékban! Jutalom: 10 €	Cső Felfedező Érj el 10 pontot a FlappyBird játékban! Jutalom: 20 €	Ég Navigátora Érj el 15 pontot a FlappyBird játékban! Jutalom: 30 €	Csőnek Mestere Érj el 20 pontot a FlappyBird játékban! Jutalom: 40 €
Kíváncsi Elme Teljesítsd a Memória játékot 100 felfordítás alatt! Jutalom: 10 €	Memória Tanonc Teljesítsd a Memória játékot 100 felfordítás alatt! Jutalom: 20 €	Miniaadász Teljesítsd a Memória játékot 90 felfordítás alatt! Jutalom: 30 €	Memória Mestere Teljesítsd a Memória játékot 80 felfordítás alatt! Jutalom: 40 €
Első Találat Fejezz be 1 Wordle játékot! Jutalom: 10 €	Szóvadász Fejezz be 5 Wordle játékot! Jutalom: 20 €	Szókimés Szakértő Fejezz be 10 Wordle játékot! Jutalom: 30 €	Szavak Legendája Fejezz be 15 Wordle játékot! Jutalom: 40 €

155. ábra - Achievements oldal felépítése

Működés technikai háttere

Az oldal az achievement adatokat egy API végpontról tölti be. A lekérés egy GET kérés segítségével történik, és a szerver JSON formátumban küldi vissza az adatokat.

A lekért adatok dinamikusan kerülnek be az oldalba. A rendszer minden achievementhez automatikusan létrehoz egy HTML elemet, amely tartalmazza a megfelelő adatokat. A lista minden betöltéskor frissül.

Az achievementek neve és leírása több nyelven is elérhető. A rendszer a kiválasztott nyelv alapján választja ki a megfelelő mezőt (például angol vagy magyar elnevezést), így biztosítva a lokalizált megjelenést.

Felhasználói folyamat

Az oldal betöltésekor a rendszer automatikusan lekéri és megjeleníti az elérhető achievementeket. A felhasználó azonnal áttekintheti a különböző kihívásokat és a hozzájuk tartozó jutalmakat.

Szükség esetén a felhasználó megváltoztathatja a felület nyelvét a nyelvválasztó segítségével, amely azonnal frissíti az összes megjelenített adatot.

A felhasználó az oldalon böngészve kiválaszthatja, mely achievementeket szeretné teljesíteni, majd a visszalépés gomb segítségével visszatérhet a főoldalra.

6. A program általános specifikációja, célja

A Retro Realm egy webalapú, többfelhasználós platform, amely lehetőséget biztosít a felhasználók számára, hogy egy közös virtuális térben mozogjanak és valós időben kommunikáljanak egymással. A rendszer célja egy egyszerűen használható, mégis interaktív környezet biztosítása, amelyben a felhasználók karaktereken keresztül jelennek meg, és vizuálisan is érzékelhető módon léphetnek kapcsolatba egymással.

Az alkalmazás egyik kiemelt funkciója a saját, kosztümizálható karakterrendszer, amely lehetővé teszi, hogy a felhasználók különböző kinézetű karakterek közül válasszanak. Ez növeli az egyediség érzetét, valamint segíti a felhasználók megkülönböztetését a közös térben. A rendszer különböző felhasználói szerepköröket is kezel, amelyek vizuálisan is megjelennek, például a felhasználónév színében.

A platform nem csupán egy közösségi térként funkcionál, hanem több különálló játékelemet is tartalmaz. A Retro Realm összesen négy különböző játékot foglal magába a közösségi platformon kívül, amelyek a felhasználók számára további szórakozási lehetőségeket biztosítanak. Ezek a játékok ugyanazon rendszer részeként működnek, így egységes felhasználói élményt nyújtanak.

A rendszer része egy ranglista funkció is, amely a játékokban elért eredményeket tárolja és jeleníti meg. Ez lehetőséget ad a felhasználóknak arra, hogy összehasonlítsák teljesítményüket másokkal, ezáltal növelve a motivációt és a versenyszellemet.

Az alkalmazás fő funkciói közé tartozik a karakterek mozgatása egy kétdimenziós térképen, a többi felhasználó valós idejű megjelenítése, valamint a rövid szöveges üzenetek küldése és megjelenítése a karakterek felett. A rendszer támogatja több felhasználó egyidejű jelenlétét, amelyet WebSocket alapú kommunikáció biztosít, így az események és mozgások azonnal szinkronizálásra kerülnek a kliensek között.

A Retro Realm elsődleges célközönsége olyan felhasználók, akik egy egyszerű, böngészőben futó, telepítést nem igénylő közösségi platformot keresnek, ahol gyorsan és könnyen kapcsolatba léphetnek másokkal, valamint különböző játékokat is kipróbálhatnak egy egységes rendszer keretein belül. A rendszer különösen alkalmas oktatási vagy demonstrációs célokra, ahol a valós idejű kommunikáció és a többfelhasználós működés bemutatása a cél.

A program előnye, hogy nem igényel telepítést, bármely modern webböngészőből elérhető, valamint kis erőforrásigénnyel rendelkezik. Ugyanakkor a működéshez folyamatos internetkapcsolat szükséges, és a teljes funkcionalitás csak megfelelő képernyőmérettel rendelkező eszközökön (legalább 1024 pixel szélesség) érhető el.

7. Rendszerkövetelmények

7.1 Hardver követelmények

A rendszer web alapú alkalmazásként működik, így a felhasználói oldalon nem igényel speciális hardveres környezetet. Ennek ellenére a megfelelő működés és a gördülékeny felhasználói élmény érdekében bizonyos minimum és ajánlott konfigurációk meghatározása indokolt.

Felhasználói oldalon a minimális követelmény egy legalább kétmagos processzorral rendelkező számítógép, minimum 4 GB memóriával és stabil internetkapcsolattal. A rendszer futtatásához legalább 500 MB szabad tárhely szükséges, valamint egy legalább 1366×768 felbontású kijelző. Az alkalmazás modern webböngészőben fut, így külön telepítésre nincs szükség.

Az ajánlott konfiguráció ezzel szemben egy legalább négymagos processzor, 8 GB vagy több memória, valamint SSD alapú háttértár használata. A nagyobb felbontású kijelző (például Full HD) és a gyors, alacsony késleltetésű internetkapcsolat különösen fontos a valós idejű működés, például a WebSocket alapú kommunikáció esetén.

A rendszer webes jellegéből adódóan különböző eszközökön is elérhető, beleértve a mobiltelefonokat, tableteket, valamint asztali számítógépeket és laptopokat. A felhasználói felület reszponzív kialakítása lehetővé teszi, hogy az alkalmazás különböző képernyőméreteken is megfelelően jelenjen meg. Fontos azonban megjegyezni, hogy bár az oldal mobil eszközökön is megnyitható és használható, a játék funkciók jelenlegi formájukban elsősorban asztali környezetre lettek optimalizálva. Ennek megfelelően a játékok teljes funkcionalitása és megfelelő működése jelenleg laptopokon és asztali számítógépeken biztosított.

A szerveroldali környezet szintén nem igényel speciális hardvert, azonban a folyamatos működés érdekében stabil konfiguráció szükséges. Minimális követelményként egy kétmagos processzor, legalább 4 GB memória és néhány gigabájt szabad tárhely elegendő lehet. Ugyanakkor ajánlott legalább négymagos processzor, 8–16 GB memória és SSD alapú tárolás használata, különösen több egyidejű felhasználó esetén. A szerver folyamatos, stabil internetkapcsolatot és lehetőség szerint állandó rendelkezésre állást igényel.

7.2 Szoftver követelmények

A rendszer ASP.NET Core alapú webalkalmazásként került megvalósításra, ezért a működéshez szükséges szoftveres környezet ennek megfelelően alakul.

Felhasználói oldalon bármilyen modern operációs rendszer használható, például Windows, Linux vagy macOS. A rendszer eléréséhez egy korszerű webböngésző szükséges, amely támogatja a JavaScript futtatását és a WebSocket kommunikációt. Ilyen például a Google Chrome, a Microsoft Edge vagy a Mozilla Firefox. Mivel az alkalmazás webes felületen keresztül működik, külön kliens telepítése nem szükséges.

A szerveroldalon a rendszer futtatásához szükség van a megfelelő .NET futtatókörnyezetre, amely biztosítja az ASP.NET Core alkalmazás működését. A backend egy beépített webszerveren keresztül fut, amely igény esetén kiegészíthető reverse proxy megoldással. Az adatok tárolása SQLite adatbázis segítségével történik, amely nem igényel külön adatbázis-szerver telepítését. A rendszer működéséhez továbbá szükséges a WebSocket támogatás biztosítása is, amely a valós idejű kommunikáció alapját képezi.

Fejlesztői környezetben a rendszer karbantartásához és továbbfejlesztéséhez ajánlott valamilyen integrált fejlesztői környezet, például Visual Studio vagy Visual Studio Code használata, valamint a .NET SDK és egy verziókezelő rendszer, például Git alkalmazása.

8. A program telepítése

•A fejlesztett alkalmazás egy webalapú rendszer, amely nem igényel hagyományos értelemben vett telepítést a felhasználó részéről. A program használatához elegendő egy internetkapcsolattal rendelkező eszköz és egy korszerű webböngésző.

A felhasználó a programot egy megadott webcímen keresztül érheti el. A fejlesztés során az alkalmazás helyi környezetben (localhost) fut, amely a fejlesztő saját számítógépén elérhető szervert jelenti. Fontos megjegyezni, hogy a localhost cím kizárólag azon a gépen érhető el, ahol az alkalmazás fut.

A rendszer működéséhez ajánlott a legfrissebb verziójú böngészők használata, például Google Chrome, Mozilla Firefox vagy Microsoft Edge. A megfelelő működés érdekében a böngészőben engedélyezni kell a JavaScript futtatását, mivel az alkalmazás kliensoldali logikája erre épül.

A felhasználói élmény biztosítása érdekében az alkalmazás csak megfelelő képernyőméret esetén érhető el. Amennyiben a képernyő szélessége nem éri el az 1024 pixel szélességet, a rendszer automatikusan átirányítja a felhasználót egy tájékoztató oldalra, ahol információt kap arról, hogy az alkalmazás teljes funkcionalitása csak nagyobb kijelzőn érhető el.

A program használata előtt szükség lehet bejelentkezésre, amely során a rendszer hitelesíti a felhasználót. A sikeres autentikáció után a felhasználó hozzáfér a teljes funkcionalitáshoz.

A rendszer valós idejű kommunikációt használ, ezért stabil internetkapcsolat szükséges. A kapcsolat megszakadása esetén egyes funkciók, például a többjátékos interakciók nem működnek megfelelően.

Összességében a program használatba vétele nem igényel külön telepítési lépéseket, mivel az alkalmazás teljes mértékben böngészőben fut, így a használat gyors és egyszerű.

9. Összegzés

A projekt során kitűzött fő cél egy olyan webalapú, többfelhasználós platform létrehozása volt, amely valós idejű interakciót biztosít a felhasználók között, valamint egyszerűen elérhető böngészőből, telepítés nélkül. A megvalósított rendszer alapján elmondható, hogy ezek a célok sikeresen teljesültek.

A Retro Realm képes több felhasználó egyidejű kezelésére, akik egy közös térben mozoghatnak, valamint valós időben kommunikálhatnak egymással. A WebSocket alapú kommunikáció megfelelően biztosítja az adatok gyors és folyamatos szinkronizálását. Emellett a platform integrál több különálló játékot, valamint egy leaderboard rendszert is, amely tovább növeli a felhasználói élményt. A karakterek testreszabhatósága szintén hozzájárul az egyedi megjelenéshez és az azonosíthatósághoz.

A fejlesztés során ugyanakkor voltak olyan elképzelések, amelyek nem kerültek teljes mértékben megvalósításra, elsősorban időbeli korlátok miatt. Ilyen például egy fejlettebb karakter testreszabási rendszer, amely több vizuális elemet és kombinációs lehetőséget tartalmazna, valamint egy komplexebb chat rendszer, amely hosszabb beszélgetések kezelésére is alkalmas lenne (például külön chat felülettel vagy üzenetelőzményekkel). További terv volt különböző interaktív objektumok elhelyezése a pályán, amelyekkel a játékosok kapcsolatba léphetnek.

A jövőbeni fejlesztési lehetőségek közé tartozik a mobil eszközök teljes körű támogatásának megvalósítása. Jelenleg a rendszer csak legalább 1024 pixel szélességű eszközökön érhető el, amely technikai és felületbeli korlátokból adódik. A fejlesztés során azonban már elkezdődött a mobilra optimalizálás, és a játékok egy része már részben alkalmazkodik kisebb képernyőkhöz. A jövőben egy átdolgozott, modulárisabb rendszerrel a teljes platform elérhetővé válhat mobil eszközökön is.

További fejlesztési irány lehet különböző időszakos események (eventek), illetve évszakokhoz kapcsolódó vizuális és tartalmi változtatások bevezetése. Ez magában foglalhatja például ünnepi témájú pályák, egyedi dizájnelemek vagy időszakosan elérhető funkciók megjelenítését, amelyek növelik a felhasználói élményt és a platform dinamizmusát.

Egy barátrendszer kialakítása szintén jelentős fejlesztési lehetőséget jelent. Ennek segítségével a felhasználók megjelölhetik egymást barátként, ami lehetővé tenné a közvetlen kommunikációt, valamint a közös játékot és versengést bizonyos játékokban. Ez a funkció jelentősen erősítené a közösségi élményt és a felhasználók közötti kapcsolatokat.

Emellett érdemes lenne tovább bővíteni a játékok számát és komplexitását, valamint fejleszteni a leaderboard rendszert több kategóriával vagy időszakos ranglistákkal. A vizuális megjelenítés finomítása, animációk és effektek hozzáadása szintén hozzájárulhat a rendszer minőségének növeléséhez.

Összességében a projekt elérte a kitűzött célját, és egy stabil alapot biztosít további fejlesztésekhez. A rendszer jól bővíthető, így a jövőben számos új funkcióval egészíthető ki, amelyek tovább növelhetik a felhasználói élményt és a platform értékét.

Ezúton szeretnénk megköszönni a felkészítő tanárok támogatását és szakmai segítségét, valamint diáktársaink közreműködését és visszajelzéseit. Köszönettel tartozunk minden további közreműködőnek is, akik hozzájárultak a projekt megvalósításához.

10. Ábrajegyzék

1. ábra - Trello munkafelület	8
2. ábra - Adatbázis szelete a HeidiSQL-ben	9
3. ábra - Adatbázis ER-modell változata DBeaver-ben	10
4. ábra - Végpontok autorizációs tesztelése Postman-el	11
5. ábra - Adatbázisok és tábláik HeidiSQL-ben	13
6. ábra - Achievement modell	14
7. ábra - Avatar modell	15
8. ábra - Flappy Bird Status Modell	17
9. ábra - Memory Game Status Modell	18
10. ábra - Wordle Status Modell	19
11. ábra - Role Modell	20
12. ábra - User Modell	21
13. ábra - RefreshToken modell	23
14. ábra - Log Modell	24
15. ábra - /getName végpont válasza	26
16. ábra - /logs végpont válasza	27
17. ábra - /users végpont válasza	28
18. ábra - /api/Avatars végpont válasza	29
19. ábra - /api/BunnyRunStatus lekérés végpont válasza	30
20. ábra - /api/BunnyRunStatus frissítés végpont válasza	31
21. ábra - /api/DeleteAllRefreshToken végpont válasza	32
22. ábra - /api/FlappyBirdStatus lekérés végpont válasza	33
23. ábra - /api/FlappyBirdStatus frissítés végpont válasza	34
24. ábra - /api/GetAchievements végpont válasza	35
25. ábra - /api/GetUserData végpont válasza	36
26. ábra - /api/GetUserName végpont válasza	37
27. ábra - /bunnyrun végpont válasza	38
28. ábra - /flappybird végpont válasza	39
29. ábra - /memorygame/flips végpont válasza	40
30. ábra - /memorygame/time végpont válasza	41
31. ábra - /wordle végpont válasza	42
32. ábra - /achievement végpont válasza	43

33. ábra - /api/Login végpont válasza	44
34. ábra - /api/Logout végpont válasza	45
35. ábra - /api/MemoryGameStatus lekérés végpont válasza	46
36. ábra - /api/MemoryGameStatus frissítés végpont válasza	47
37. ábra - /api/PurchaseAvatar végpont válasza	48
38. ábra - /api/RefreshToken végpont válasza	49
39. ábra - /api/Register végpont válasza	50
40. ábra - /api/setCharacter végpont válasza	51
41. ábra - /api/WordleStatus lekérés végpont válasza	52
42. ábra - /api/WordleStatus frissítés végpont válasza	53
43. ábra - Log objektum létrehozása.....	54
44. ábra - Log szolgáltatás meghívása	55
45. ábra - JWT token generálás	56
46. ábra - Login DTO	57
47. ábra - Játékos állapot frissítése és tárolása a WebSocket rendszerben	58
48. ábra - Játékmotor inicializálása és beállítása	59
49. ábra - Loading Scene implementációja.....	60
50. ábra - Játékos karakterének osztálya.....	61
51. ábra - Háttér osztály impelementációja	62
52. ábra - Fox osztály implementációja	63
53. ábra - ScoreBoard osztály implementációja	64
54. ábra – UI- és Statikus elemek és Bunny sprite importálása.....	65
55. ábra - Felhasználónak rekordjának frissítésének kérése	66
56. ábra - Játékmotor inicializálása és beállítása	67
57. ábra - Játékot futtató metódus	68
58. ábra - Madár ugrásáért felelős metódus	69
59. ábra - Csövek generálásáért felelős metódus	70
60. ábra - Háttérelemek mozgatásáért felelős metódus	71
61. ábra - A pontszám megjelenítéséért felelős metódus.....	72
62. ábra - Pontszám vizsgálataért felelős metódus	73
63. ábra - Játék vége scen.....	74
64. ábra - Játék inicializálása	75
65. ábra - Kártyák összehasonlítása	76
66. ábra - Kártya eseménykezelés.....	76

67. ábra - Nyelvkezelés egy szelete	77
68. ábra - Rekord frissítés végpont meghívása	77
69. ábra - Szó kiértékelési algoritmus implementációja	79
70. ábra - Játékrács dinamikus renderelése	80
71. ábra - Animáció időzítési folyamat	81
72. ábra - Felhasználó Státuszának frissítése a Wordle játékban	82
73. ábra - Alkalmazás indító scriptje	84
74. ábra - Játéktér generáló metódus szelete	85
75. ábra - Karakterek frissítéséért felelős metódus	86
76. ábra - Gomb lenyomásokat érzékelő kódrészlet	87
77. ábra - Szövegbeviteli mező lehetséges inputjai	88
78. ábra - Kliens állapotküldése a szerverre	89
79. ábra - GetUserName metódus implementációja	90
80. ábra - Játékállapot frissítése	91
81. ábra - Belépés kérés elküldése	92
82. ábra - Napló esemény táblázatsorának létrehozása	93
83. ábra - Napló részletes adatlapjának felépítése	94
84. ábra - Az avatárok dinamikus megjelenítését végző DisplayItems függvény	96
85. ábra - A felület szövegeinek frissítését végző setLanguage függvény	97
86. ábra - Avatar vásárlásának megvalósítása HTTP POST kérés segítségével	98
87. ábra - Felhasználói hitelesítés ellenőrzése token segítségével	98
88. ábra - Navigáció megvalósítása a főoldalra történő visszalépéssel	99
89. ábra - Avatárok lekérése backend API végpontról	100
90. ábra - Token alapú hitelesítés használata az API kérés során	100
91. ábra - Ranglisták lekérése és összegyűjtése egy közös objektumba	101
92. ábra - Ranglista lekérése HTTP GET kérés segítségével	102
93. ábra - Ranglista fejléc és adatok dinamikus megjelenítése	103
94. ábra - Ranglista kiválasztása URL paraméter alapján	104
95. ábra - Navigáció megvalósítása eseménykezelők segítségével	105
96. ábra - Felhasználói alapadatok dinamikus megjelenítése	107
97. ábra - Játékstatisztikák megjelenítése a felhasználói adatok alapján	107
98. ábra - Megszerzett avatárok dinamikus megjelenítése	107
99. ábra - Az oldal inicializálása és a felhasználói adatok betöltése	108
100. ábra - Felhasználói adatok lekérése backend API segítségével	108

101. ábra - Felhasználónév lekérése hitelesített API kérés segítségével	109
102. ábra - Felhasználói adatok lekérése POST kérésen keresztül	109
103. ábra - Avatar módosítása API hívás segítségével	110
120. ábra - Mérföldkövek megjelenítése	121
122. ábra - Bejelentkezés nélkül is elérhető végpont.....	123
123. ábra - Admin név lekérésének elutasítása nem admin user esetén	124
124. ábra - Authorizáció nélküli kérés elutasítása	124
125. ábra - Kaplay logó a betöltés során.....	126
126. ábra - Játék menüje	127
127. ábra - Játékmenet közben megjelenő akadályok.....	128
128. ábra - Pontszám megjelenítése játék közben a bal felső sarokban	129
129. ábra – Záróképernyő a pontszámmal és "NEW RECORD" felirattal.....	130
130. ábra - Kaplay logó a betöltés során.....	131
131. ábra - Menü felépítése és gombok	132
132. ábra - "Tap Tap" felirat a játék kezdésekor	133
133. ábra - Játékmenet közben megjelenő csövek és pontszám	134
134. ábra - Játék vége képernyő.....	135
135. ábra - Játék menüje	136
136. ábra - Kisegítő panel	137
137. ábra - Játék felépítése.....	138
138. ábra - Játék vége panel.....	139
139. ábra - Játék menüje	140
140. ábra - A játék menete szekció a kisegítő panelen	141
141. ábra - Játéktér felépítése és egy példa szemléltetése	142
142. ábra - Sikertelen játék vége panel	143
143. ábra - A játék pályája játékosokkal.....	144
144. ábra - Több játékos különböző névszínekkel.....	145
145. ábra - Üzenet írása	146
146. ábra - Játékos felett megjelenő szöveg buborék	147
147. ábra - Több játékos beszélget.....	148
148. ábra - Admin Panel kezdőlapja	149
149. ábra - Navigációs panel.....	150
150. ábra - Részletes felhasználói adatlap	151
151. ábra - Részletes naplóbejegyzés adatlap	152

152. ábra - A Custom Item Shop teljes felülete megvásárolható avatárokkal.	153
153. ábra - Megvásárolható avatárok listája képpel, névvel és árral.	154
154. ábra - Vásárlás gomb egy avatár alatt.	155
155. ábra - Vásárlás utáni visszajelző üzenet.	155
156. ábra - A ranglista oldal teljes felülete különböző játékok eredményeivel.	157
157. ábra - Lenyíló menü a különböző ranglisták kiválasztásához.	158
158. ábra - Ranglista megjelenítése felhasználónévvel és eredményekkel.....	158
159. ábra - A profil oldal teljes felülete felhasználói adatokkal és statisztikákkal.	160
160. ábra - Navigációs sáv kilépés és nyelvválasztás lehetőséggel.	161
161. ábra - Nyelvválasztó menü három választható nyelvvel.	161
162. ábra - Felhasználói alapadatok: profilkép, név és coin egyenleg.	162
163. ábra - Játékstatisztikák különböző játékokhoz.....	162
164. ábra - Megvásárolt avatárok, amelyek közül a felhasználó választhat.	163
165. ábra - Az avatár kiválasztása után a profilkép azonnal frissül.....	163
166. ábra - A felhasználó által elért eredmények listája.	163
167. ábra - Bejelentkezés panel megjelenése.....	165
168. ábra - Sikeres bejelentkezés felhívás	166
169. ábra - Regisztrációs panel	167
170. ábra - Sikeres regisztrációt követő válasz.....	168
171. ábra - Főoldal felépítése adminisztrátori jogkörrel.....	169
172. ábra - Achievements oldal felépítése	171

11. Irodalomjegyzék, hivatkozásjegyzék

[1] ASP.NET Core – JWT Authentication

<https://learn.microsoft.com/en-us/aspnet/core/security/authentication/jwtbearer/>

A JSON Web Token (JWT) alapú hitelesítés hivatalos dokumentációja ASP.NET Core környezetben. Bemutatja a token alapú autentikáció működését és a biztonságos API-védelem kialakítását. A projekt során a felhasználók hitelesítésének és jogosultságkezelésének megvalósításához használtuk fel.

[2] ASP.NET Core – Microsoft Learn

<https://learn.microsoft.com/en-us/aspnet/core/>

Az ASP.NET Core keretrendszer hivatalos dokumentációja, amely részletes útmutatást nyújt webalkalmazások és REST API-k fejlesztéséhez. A projekt backend részének felépítéséhez, az alkalmazás architektúrájának kialakításához és az alapvető működés megértéséhez használtuk.

[3] ASP.NET Core – WebSockets

<https://learn.microsoft.com/en-us/aspnet/core/fundamentals/websockets/>

A WebSocket technológia használatát ismerteti ASP.NET Core alkalmazásokban, különös tekintettel a valós idejű kommunikációra. A projekt során valós idejű adatfrissítések és interaktív funkciók megvalósításának tanulmányozásához használtuk.

[4] Carbon – kódrészletek megjelenítése

<https://carbon.now.sh/>

Egy online eszköz, amely lehetővé teszi esztétikus és jól olvasható kódrészletek generálását képként. A projekt során a dokumentációhoz szükséges kódrészletek kiemelésére és vizuálisan rendezett formában történő bemutatására használtuk.

[5] GitHub

<https://github.com/>

Online verziókezelő és együttműködési platform, amely Git alapokon működik. A projekt során a forráskód tárolására, verziókövetésre és a fejlesztési folyamat rendszerezésére használtuk.

[6] Google Fonts

<https://fonts.google.com/>

Online betűtípus-gyűjtemény, amely ingyenesen használható webes fontokat kínál. A projekt során a felhasználói felület tipográfiájának kialakításához, az egységes és esztétikus megjelenés biztosításához használtuk.

[7] Google Material Icons

<https://fonts.google.com/icons>

A Google által biztosított ikonkönyvtár, amely könnyen beépíthető webalkalmazásokba. A projekt során vizuális elemek, ikonok és felhasználói felületi jelölések megjelenítésére használtuk.

[8] Mobiltelefon szimulátor – reszponzív webhely tesztelése (Chrome bővítmény)

https://chromewebstore.google.com/detail/mobiltelefon-szimul%C3%A1tor-ckejmhbmlajgoklhgbapkiccekfoccmk?hl=hu&utm_source=ext_sidebar

A Google Chrome böngészőhöz kapcsolódó fejlesztői eszköz, amely lehetővé teszi különböző mobil eszközök képernyőméretének és viselkedésének szimulálását. A projekt során a webalkalmazás reszponzív működésének tesztelésére és a felhasználói felület különböző kijelzőkön való ellenőrzésére használtuk.

[9] HeidiSQL

<https://www.heidisql.com/>

Adatbázis-kezelő eszköz, amely lehetővé teszi adatbázisok létrehozását, módosítását és lekérdezések futtatását. A projekt során az adatbázis struktúrájának kialakítására és az adatok kezelésére használtuk.

[10] KaplayJS

<https://kaplayjs.com/>

JavaScript alapú könyvtár 2D játékok fejlesztéséhez. A projekt során játékmechanikai megoldások és kliensoldali interakciók megvalósításának tanulmányozásához használtuk.

[11] OpenGameArt

<https://opengameart.org/>

Ingyenesen felhasználható grafikai és hanganyagokat biztosító platform. A projekt során vizuális elemek és grafikai erőforrások beszerzésére és inspiráció gyűjtésére használtuk.

[12] Postman

<https://www.postman.com/>

API tesztelésre szolgáló eszköz, amely lehetővé teszi HTTP kérések küldését és válaszok elemzését. A projekt során a backend végpontok tesztelésére és hibakeresésre használtuk.

[13] W3Schools Web Tutorials

<https://www.w3schools.com/>

Online tanulási platform, amely alapvető webfejlesztési technológiák elsajátítását segíti. A projekt során a HTML, CSS és JavaScript ismeretek megerősítésére és gyors referenciaforrásként használtuk.

[14] YouTube – oktatóvideók, tutorialok

Videómegosztó platform, amely számos programozási oktatóanyagot tartalmaz. A projekt során különböző technológiák gyakorlati használatának elsajátításához és problémák megoldásához használtuk.